

软件设计实践

C++ 与软件设计

从类型、对象与资源管理
到泛型编程、设计与工程实践

VON

课程参考：2025 春软件设计实践

授课教师：谢涛、马郢

目录

开始之前	v
第 1 章 类型、值与对象：先看一段程序怎样保存和修改数据	1
1.1 声明变量，然后使用它	1
1.2 名字、对象、引用与指针	2
1.3 初始化：对象怎样获得起点	3
1.4 const：限制哪一条访问路径	4
1.5 auto 与 decltype：推导什么，保留什么	5
1.6 动态存储：建立对象与归还资源	7
1.7 读代码时的三个检查	8
第 2 章 函数与可调用对象：把过程变成可组合的接口	9
2.1 函数的约定先于函数体	9
2.2 参数传递：复制一个值，还是访问原对象	10
2.3 默认参数与重载：编译器怎样选择入口	11
2.4 函数指针：把算法中的变化部分传进来	12
2.5 Lambda：把行为与所需状态放在一起	13
2.6 组合过程与类型擦除	14
2.7 从函数抽象走向设计模式	15
第 3 章 类、封装与生命周期：让对象始终处于有效状态	16
3.1 类是带操作约定的数据抽象	16
3.2 成员函数与 this	17
3.3 构造函数：先初始化，再进入函数体	18
3.4 析构顺序与成员对象	18
3.5 对象数组与临时对象	20
3.6 静态成员：属于类的共享状态	20
3.7 只读成员函数与逻辑只读	21
3.8 友元与协作接口	22
第 4 章 资源管理与复制：把责任写进对象语义	24
4.1 拥有资源与借用资源	24
4.2 RAII：利用析构规则自动清理	24
4.3 Buffer 的不变式与建立方式	25
4.4 复制构造与复制赋值	27
4.5 编译器生成的复制到底做什么	28
4.6 独占所有权：std::unique_ptr	29
4.7 共享所有权与 weak_ptr	30
4.8 把资源语义落实到接口	31
第 5 章 运算符与值语义：让自定义类型自然地参与表达式	32
5.1 重载不会改变语法结构	32
5.2 成员与非成员：二元操作是否对称	32

5.3	比较：相等与排序是两套要求	33
5.4	赋值与链式操作	34
5.5	输入输出：链式调用与失败状态	34
5.6	下标、检查访问与只读重载	35
5.7	前置与后置自增	36
5.8	调用操作与显式转换	37
第 6 章	继承与组合：表达关系，也承担约束	39
6.1	派生对象中的基类子对象	39
6.2	访问权限与继承方式	40
6.3	名字隐藏、重载与覆盖	40
6.4	构造与销毁顺序	41
6.5	切片与基类部分赋值	42
6.6	多继承与虚继承	43
6.7	组合与委派：复用功能而不承诺替换	44
第 7 章	多态与运行时行为：同一个接口，不同的实现	45
7.1	静态类型与动态类型	45
7.2	把查找、访问与分派分开	46
7.3	构造与析构期间的虚调用	47
7.4	虚析构与多态拥有	48
7.5	抽象类与可扩展集合	49
7.6	复制多态对象：显式 clone	50
7.7	RTTI：查询与检查动态类型	51
7.8	虚表是常见解释，不是语言强制布局	51
第 8 章	移动语义与类型转换：什么时候复制，什么时候交出资源	53
8.1	先用起来：保留一份副本，或把内容交给新对象	53
8.2	std::move 本身做什么：观察一次普通函数调用	54
8.3	右值引用的语法：T&& 不代表自动搬运	55
8.4	亲手实现资源交接：Buffer 的两个移动操作	56
8.5	值类别：给已经看到的调用规则命名	58
8.6	noexcept 与异常保证	59
8.7	复制消除：有时根本不需要复制或移动	59
8.8	四类显式转换分别表达什么	60
8.9	让类型变化服务清楚的设计	61
第 9 章	模板与编译期抽象：让算法依赖能力而非固定类型	62
9.1	函数模板与实参推导	62
9.2	转发引用与完美转发	63
9.3	模板与普通函数如何竞争	64
9.4	类模板、非类型参数与 CTAD	64
9.5	特化与类型特征	65
9.6	变参模板与折叠表达式	66
9.7	if constexpr 与按需实例化	67
9.8	依赖名字与定义可见性	68
9.9	泛型抽象的收益与代价	69
第 10 章	容器与迭代器：数据结构背后的使用契约	70

10.1 容器、迭代器与算法的分工	70
10.2 vector: 大小、容量与重新分配	70
10.3 不同容器的组织方式	72
10.4 关联容器: 键、比较器与哈希	73
10.5 半开区间与迭代器能力	74
10.6 失效规则: 修改结构后重新检查访问途径	75
10.7 元素构造、复制与移动	76
10.8 选容器时先回答实际问题	76
第 11 章 标准算法: 把意图写清楚, 也把前提说清楚	77
11.1 一个算法由什么组成	77
11.2 查找: 返回末端时不要解引用	77
11.3 复制、筛选与输出迭代器	78
11.4 排序: 比较器决定关系	79
11.5 算法能力: 为什么 list 不能直接用 std::sort	80
11.6 累计: 初始值也决定类型	81
11.7 可调用对象的状态与复制	82
第 12 章 Concepts 与 Ranges: 把泛型要求放到接口上	83
12.1 requires 表达式描述可用操作	83
12.2 用约束表达一个真正泛化的求和接口	83
12.3 范围不是新的容器继承体系	84
12.4 视图管道: 描述怎样访问, 而不立即建立结果数组	85
12.5 惰性求值与重复访问	86
12.6 拥有、借用与悬垂	87
12.7 视图也受算法能力限制	88
12.8 用现代接口减少错误, 而不是隐藏责任	88
第 13 章 多文件工程: 从一个程序文件到可构建的项目	89
13.1 翻译单元、编译与链接	89
13.2 把接口与实现分开	90
13.3 ODR、inline 与名字空间	91
13.4 用构建系统表达目标和依赖	92
13.5 文件与流: 状态检查属于处理过程	93
13.6 二进制存储不是随意写出对象字节	94
第 14 章 开发过程与设计原则: 控制变化带来的成本	95
14.1 从功能到可观察的完成标准	95
14.2 模块与架构: 先分责任, 再画框	95
14.3 SOLID: 把原则落到行为与变化上	96
14.4 依赖倒置与可测试性	97
14.5 计划、风险与版本控制	98
第 15 章 健壮性与测试: 把“应该正确”变成可检查的条件	99
15.1 规约、故障与可观察失败	99
15.2 输入边界与错误处理	99
15.3 异常路径与状态保证	100
15.4 黑盒、白盒与边界值	101
15.5 参数化测试与性质测试	101

15.6 建立短而可靠的反馈循环 102

第 16 章 复用与创建模式：给变化一个合适的位置 103

16.1 模式的名字应指向设计意图 103

16.2 策略与模板方法：变化算法，还是变化步骤 103

16.3 适配器：保留旧组件，提供新接口 105

16.4 装饰器与外观：组合行为，或简化入口 106

16.5 工厂：把创建决定与使用流程分开 107

第 17 章 维护与演化：扩展行为，并守住已有语义 110

17.1 观察者：把状态变化通知给多个客户 110

17.2 状态模式：让状态决定行为 112

17.3 访问者：给稳定的对象结构增加操作 113

17.4 重构：小步改变结构，保持外部行为 115

17.5 度量与审查：让判断有依据 116

第 18 章 性能与并发：先测量，再改变执行方式 117

18.1 测量与瓶颈 117

18.2 惰性计算与代理 117

18.3 单例：唯一入口不等于没有依赖 118

18.4 线程生命周期与数据竞争 119

18.5 用互斥量保护共同不变式 120

附录 A 贯穿案例与自检 122

A.1 对象、复制与移动 122

A.2 容器与视图 122

A.3 设计变化与行为契约 123

A.4 Buffer 的完整定义 123

A.5 配套程序的使用 124

参考文献 126

开始之前

一个程序能编译，并不意味着它的资源管理、边界处理和接口设计都正确。学习 C++ 时，要同时追踪三件事：表达式的类型、对象的生命周期，以及某项操作承担的责任。它们会在类型推导、复制移动、继承多态和模板中反复出现。

本讲义假设读者已认识变量、条件和循环等基本编程概念，但不预设已掌握本书介绍的 C++ 特性。内容结合课程课件与原始知识点笔记 [1, 2]，以 C++20 为主要编译基线 [3]。需要其他版本知识的地方会说明条件。示例中的标准库名称通常保留 `std::` 前缀，头文件使用标准名称。短片段突出局部规则，完整程序保存在配套示例目录中。

可以用下面的命令编译一个独立示例：

```
clang++ -std=c++20 -Wall -Wextra -Wpedantic example.cpp -o example
./example
```

编译诊断也是学习材料。标注为编译错误的片段，用来观察语言边界；标注为未定义行为的片段，用来解释错误原因，不能根据某次运行结果为它总结规律。运行输出只有在相关前提满足时才有意义。

核心概念 三种不同层面的结论

语言保证由 C++ 规则定义。**库契约**描述某个接口的前提、结果和复杂度。**实现方式**解释编译器或库通常怎样完成工作。理解机制时，不能把一种常见实现当成所有平台都必须采用的布局。

第 1 章 类型、值与对象：先看一段程序

怎样保存和修改数据

先从小问题开始：有三件商品，每件 2.5 元，怎样计算总价？数量适合用整数保存，价格和总价需要表示小数。变量的类型既决定能保存哪些值，也决定表达式中哪些操作有意义。

本章在这样的实际操作中介绍类型、初始化、引用和指针。读代码时先找到对象，再看每个名字访问了谁，最后追踪每条语句修改了什么。

1.1 声明变量，然后使用它

代码示例 类型、名字、初值和一次计算

```
#include <iostream>
int main() {
    int count = 3;           // 数量是整数
    double unit_price = 2.5; // 单价需要表示小数
    double total = count * unit_price; // 相乘时按相应数值转换规则计算
    using Count = int;       // 为已有类型起名字，不是建立一种新类型
    Count boxes = count;
    std::cout << boxes << ' ' << total << '\n'; // 预期：3 7.5
}
```

输出为 3 7.5。int count=3 中，int 是类型，count 是名字，3 是初值。count*unit_price 读取两者当前的值完成乘法，再用结果初始化 total。建立了 total 以后，再修改 count 不会让 total 自动重新计算。

std::cout 是标准输出流，连续的 << 把各项依次写出；' ' 写一个空格，'\n' 写换行。名字前的 std:: 表明它属于标准库命名空间。以后示例常用这种方式展示结果，也会用 assert(条件) 检查预期性质：条件不成立时，调试断言会报告失败。

值可以改变，声明的类型不会跟着改变

给 count 重新赋值为 4，只改变其中保存的整数。把 2.5 赋给整数变量，也不会把那个变量变成 double，而是触发相应的转换；这种转换可能丢失小数部分。auto 可以让编译器推导类型，推导完成后同样有一个确定的类型。

using Count=int; 给 int 起了一个别名。它方便表达用途，但没有建立一种能与 int 严格区分的新类型。若要让“米”与“秒”在编译时不能混用，需要以后介绍的类或包装类型提供进一步约束。

声明与定义先用函数来理解

```
int add(int a, int b); // 声明：告诉编译器怎样调用
int add(int a, int b) { return a + b; } // 定义：提供真正执行的函数体
```

看到声明以后，编译器知道参数和返回类型，能够检查 `add(2,3)` 这样的调用。程序最终还需要相应定义，才知道如何执行。变量也有声明与定义之分，但不能把所有声明都简单理解成“立刻分配一块存储”。多文件章节会把这两者与编译链接联系起来。

1.2 名字、对象、引用与指针

设 `x` 是一个整数对象，`r` 是绑定到它的引用，`p` 是保存它地址的指针对象。`x` 和 `r` 访问同一个整数；`p` 自己则保存一个指针值。

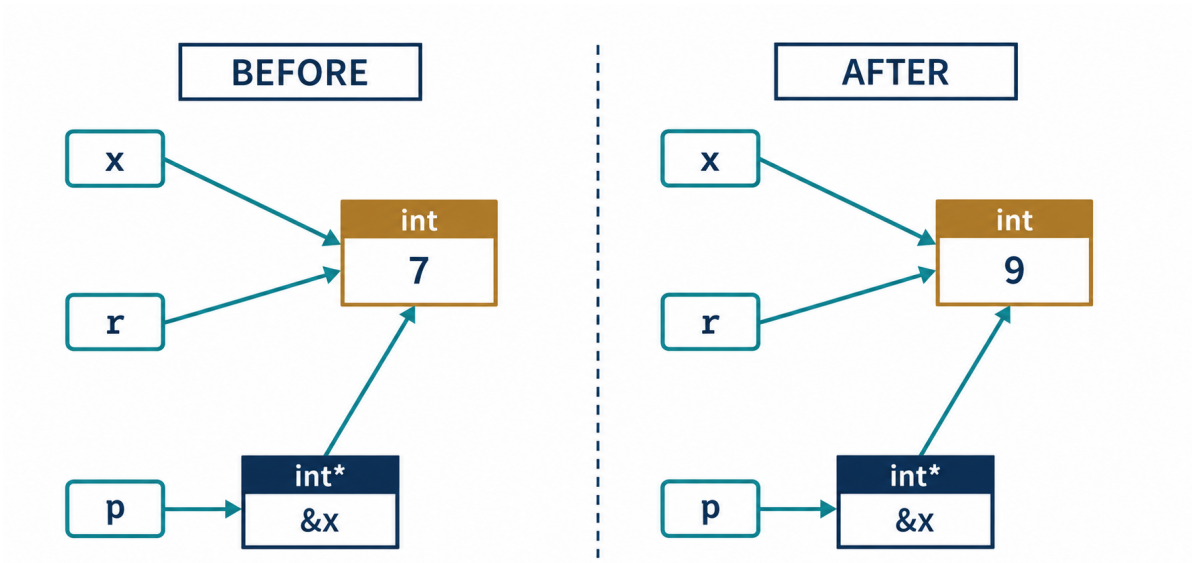


图 1.1 名字与存储的概念关系。通过 `*p` 将值改为 9 后，`x` 和 `r` 都观察到同一个变化。图中的连线表示语言层面的引用关系，不规定机器上的具体布局。

代码示例 引用赋值并不重新绑定

```
#include <iostream>
int main() {
    int x = 7;
    // r 不建立第二个整数，它是同一个 x 的别名。
    int& r = x;
    int* p = &x;
    // 先通过 p 找到 x，再修改 x；不是改变 p 保存的地址。
    *p = 9;
    // 这里建立新整数 copy，取出当前 x 的值作为初值。
    int copy = r;
    copy = 10;
    std::cout << x << ' ' << r << ' ' << *p << ' ' << copy << '\n';
    int y = 20;
    // 引用不能靠赋值重新绑定；实际执行的是 x = y。
    r = y;
```

```
std::cout << x << ' ' << y << '\n';  
}
```

运行追踪 沿对象追踪变化

```
9 9 9 10  
20 20
```

`copy` 是新建的整数对象，修改它不会修改 `x`。`r = y` 则把 `y` 的值写入原来绑定的 `x`；`r` 没有改为绑定 `y`。

引用表达别名，指针表达地址值

普通左值引用必须在建立时绑定目标。语言不提供把它重新绑定到另一个对象的赋值操作。指针则可以被重新赋值，也可以使用 `nullptr` 表示没有目标。

引用本身不保证目标永远有效。一个引用可能指向已经销毁的对象，成为悬垂引用。指针非空也不保证能够解引用：它可能已经悬垂，或指向数组末尾之后的位置。

易错点 非空不等于可访问

`if (p != nullptr)` 只排除了空指针这种情况。它不能检测目标是否已经被释放，也不能确认索引是否越界。有效访问同时依赖对象生命周期、地址范围和类型规则。

赋值作用于表达式所表示的对象

`p = &y` 修改指针对象；`*p = 8` 修改它当前指向的整数。理解一层间接访问以后，多层指针也可以逐层分析，而不必把星号当成一串难记符号。

不要用“能否写在等号左边”完整定义左值。一个只读对象的名字仍是左值表达式，却不能通过它赋值。值类别将在移动语义一章集中讨论。

1.3 初始化：对象怎样获得起点

初始化发生在对象建立时，赋值发生在已有对象上。即使语句中都出现等号，两者也可能调用完全不同的操作。

```
int a = 5;    // 建立 a，并给出初值  
int b{5};    // 同样是初始化，只是采用花括号语法  
b = 8;       // b 已经存在，现在对它赋值
```

对内置整数，花括号能明确给出初值。`int x{};` 得到零；局部的 `int x;` 则没有提供可供任意读取的初值。在 C++20 的常规情形中读取这种未初始化的 `int` 会导致未定义行为，不能把结果理解成一个可依赖的“随机数”。

列表初始化与窄化检查

`int x{3.5};` 会因窄化而被拒绝。`int x = 3.5;` 可能允许转换，但会丢失小数部分。编译器接受一种写法，并不代表数据损失符合业务要求。

对类类型，花括号与圆括号还可能选择不同构造函数：

代码示例 相同数字，不同构造意图

```
#include <cassert>
#include <vector>
struct Point { int x; int y; };
int main() {
    int zero{};
    // 未显式给值的剩余两个元素初始化为 0。
    int values[3]{1};
    Point p{2, 5};
    // 圆括号调用“数量、初值”构造：得到三个 7。
    std::vector<int> repeated(3, 7);
    // 花括号选择元素列表：得到 3 和 7 两个元素。
    std::vector<int> listed{3, 7};
    assert(zero == 0 && values[1] == 0);
    assert(p.y == 5);
    assert(repeated.size() == 3 && repeated[0] == 7);
    assert(listed.size() == 2 && listed[0] == 3);
}
```

`repeated(3,7)` 创建三个值为 7 的元素，`listed{3,7}` 则创建两个元素 3 和 7。统一的语法外观不意味着所有类型拥有相同初始化语义。

默认初始化不是统一清零

静态存储期对象、自动存储期标量和带构造函数的类，会遵循不同初始化步骤。类的默认构造函数也可能没有给所有内置成员赋值。最容易检查的写法，是在成员声明或构造初始化列表中明确写出初值。

工程实践 先建立有效状态

对象一旦交给其他代码使用，就应满足自己的基本约束。尺寸非负、索引在范围内、拥有的指针与记录的长度一致，都适合在构造阶段建立。把对象先放到半完成状态，再希望调用者补齐，会增加出错机会。

1.4 const：限制哪一条访问路径

先考虑一个函数只应读取分数，不应修改它。可以让函数接收 `const int& score`，也可以通过 `const int*` 读取。`const` 修饰的位置很重要：指针本身能否换地址，与能否通过它修改整数，是两项不同权限。

读 `const int* reader` 时，把星号右侧的 `reader` 看成指针，它指向 `const int`；读 `int* const fixed` 时，`const` 限制指针变量 `fixed` 本身。下面让两种指针都先指向 `x`，再分别修改整数和指针地址，观察区别。

`const` 的含义是限制修改，不必然表示该值在编译时已知。例如：

```
const int n = read_count();
```

这个值可以在运行时获得，只是初始化以后不能再通过 `n` 修改它。

还要区分对象是否本来就是只读，与某个访问路径是否只读。对一个可修改对象建立 `const int&`，只是不能通过这个引用修改；其他可修改路径仍然可以改变原对象。

声明	指针本身	通过该指针访问目标
<code>int* p</code>	可以重新指向	可以修改整数
<code>const int* p</code>	可以重新指向	不可通过它修改整数
<code>int* const p</code>	初始化后不可重新指向	可以修改整数
<code>const int* const p</code>	初始化后不可重新指向	不可通过它修改整数

表 1.1 从 `const` 所修饰的层次理解指针类型，比背“常量指针”等容易混淆的中文名称更可靠。

代码示例 指针与目标的只读性是两回事

```
#include <iostream>
int main() {
    int x = 3, y = 4;
    // 通过 reader 不能改整数，但 reader 自己可以改为指向别处。
    const int* reader = &x;
    // fixed 不能换目标，但可通过它修改原来的 x。
    int* const fixed = &x;
    *fixed = 8;
    reader = &y;
    // 这条访问路径既不换目标，也不允许修改目标。
    const int* const both = &x;
    std::cout << *reader << ' ' << *fixed << ' ' << *both << '\n';
}
```

输出为 4 8 8。`reader` 可以改指向 `y`，`fixed` 虽然不能改指向，却可以把 `x` 改为 8。

从 `int*` 转到 `const int*` 增加了限制，通常可以隐式进行；反过来会丢失限制，不能随意隐式转换。多层指针的限定转换还有额外规则，不能仅凭“给最里层加一个 `const`”推断一定安全。

易错点 去掉类型上的 `const`，不等于对象变得可写

如果对象原本就是 `const`，借助转换获得可写指针后再修改它，仍可能导致未定义行为。转换只改变表达式的类型视角，不能重写对象建立时的事实。

1.5 auto 与 decltype：推导什么，保留什么

平时写 `int n=3`，类型由程序员写出；写 `auto n=3`，类型由初始化表达式推导。先判断自己想要一份副本，还是原对象的别名：`auto a=x` 通常建立新值，`auto& a=x` 建立引用。

`decltype` 解决另一个问题：根据一个名字或表达式取得类型，而不执行它。示例中的 `std::is_same_v<A,B>` 在编译时判断 `A`、`B` 是否为同一类型，`static_assert` 要求判断为真。先根据注释预测结果，再用这些检查验证，不必把模板符号当成需要手算的输出。

`auto` 的常见值推导会丢掉引用与顶层 `const`。如果希望保留绑定关系，应在声明中写出 `auto&` 或 `const auto&`。

`decltype` 查询表达式的类型，但有一条重要区别：对未加括号的名字，常得到该实体声明的类型；对一般表达式，则会把值类别反映为引用类型。

代码示例 用静态断言检查推导结果

```
#include <cassert>
#include <type_traits>
int main() {
    const int c = 3;
    // 按值推导得到 int: a 是独立副本，不继承顶层 const。
    auto a = c;
    // 按引用绑定原对象，必须保留只读性质，所以 b 是 const int&。
    auto& b = c;
    int x = 7;
    // 括号使 decltype 检查表达式类别；这里得到 int&，因此是别名。
    decltype(auto) alias = (x);
    auto text = "hi";
    // 引用方式保留数组长度；字符串还包含结尾的零字符。
    auto& array = "hi";
    static_assert(std::is_same_v<decltype(a), int>);
    static_assert(std::is_same_v<decltype(b), const int&>);
    static_assert(std::is_same_v<decltype(x), int>);
    static_assert(std::is_same_v<decltype((x)), int&>);
    static_assert(std::is_same_v<decltype(text), const char*>);
    static_assert(std::is_same_v<decltype(array), const char (&)[3]>);
    alias = 8;
    assert(x == 8);
}
```

这里的 `decltype(x)` 是 `int`，而 `decltype((x))` 是 `int&`。额外括号让它进入一般表达式的规则。`decltype(auto) alias = (x)`；因而建立了别名。

原理剖析 值推导与精确保留服务不同目的

`auto` 适合“我需要由初值推导类型的新值”。`decltype(auto)` 适合“我需要保留这个表达式的准确返回类型，包括引用”。后一种写法更精确，也更容易无意中返回悬垂引用。

对字符串字面量，`auto text = "hi"`；得到指针，`auto& array = "hi"`；保留对数组的引用。数组在许多表达式中会退化为指针，但 `sizeof`、取地址和引用绑定等情形需要分别分析。

编译期常量需要明确语境

`constexpr` 对变量要求常量初始化，对函数则表示它可以在合适条件下参与常量求值，并不要求每次调用都在编译期完成。`constexpr` 则用于要求常量求值的立即函数。

代码示例 同一个 constexpr 函数的两种用法

```
#include <cassert>
constexpr int square(int x) { return x*x; }
constexpr int checked_capacity(int x) { return x > 0 ? x : throw "capacity"; }
int main() {
    // 这里要求编译期结果；编译器可以计算 square(3)。
    constexpr int fixed = square(3);
    static_assert(fixed == 9);
    int runtime = 6;
    // 同一个 constexpr 函数也可以在普通运行语境中调用。
    assert(square(runtime) == 36);
    // constexpr 要求调用在编译期完成，不能传入任意运行期输入。
    constexpr int capacity = checked_capacity(8);
    static_assert(capacity == 8);
}
```

编译器可能把普通计算优化掉，但这与语言是否要求常量表达式是两个问题。判断代码是否合法，应看语言语境，而不是猜优化器最后生成了什么。

1.6 动态存储：建立对象与归还资源

普通局部变量离开作用域时结束生命周期。若一块数据需要由程序显式决定何时释放，可以先用 new 建立对象，保留返回的指针，最后通过匹配的 delete 归还。这个例子故意采用最原始的接口，让资源责任可见；后面会用拥有型对象自动管理它。

int* one=new int{42} 同时涉及两个对象：指针变量 one，以及另行建立的整数 42。把 one 赋为 nullptr，只是清空指针变量；它不会代替 delete 释放那个整数。

new 表达式通常完成存储分配与对象初始化。delete 表达式对合适的目标执行销毁并释放存储。数组形式必须对应数组释放：

代码示例 匹配 new 与 delete 的形式

```
#include <cassert>
int main() {
    // new 建立整数对象，返回地址；one 只是保存这个地址的指针。
    int* one = new int{42};
    assert(*one == 42);
    // 释放目标后，原地址失效；后面不再解引用它。
    delete one;
    one = nullptr;
    int* many = new int[3]{};
    assert(many[0] == 0 && many[2] == 0);
    // 用 new[] 建立的数组必须按对应方式 delete[]。
    delete[] many;
    many = nullptr;
}
```

`new int{42}` 与 `new int[3]{}` 创建不同形式的对象。不能因为某个数组只有一个元素，就把 `delete[]` 改成 `delete`。空指针可以安全地交给相应 `delete` 表达式，但已经释放的非空地址不能重复释放。

指针消失，目标未必消失

自动变量 `p` 离开作用域时，指针对象自己会结束生命周期。它指向的动态对象不会因此自动释放。反过来，释放动态对象也不会自动找到并清空所有指向它的指针。

工程实践 把释放责任交给对象

实际代码通常优先使用 `std::vector`、`std::string` 或 `std::unique_ptr` 来管理资源。学习 `new/delete` 是为了理解责任如何产生；RAII 则把这份责任落实到有一个有确定生命周期的对象上。

1.7 读代码时的三个检查

先找对象：哪些声明建立了新对象，哪些只是建立别名？再找操作：赋值到底修改了哪一层对象？最后找生命周期：当前使用的目标是否仍然存在？

例如 `int& r = x`；没有复制整数，`int y = r`；则复制了当前整数值。`r = y`；修改原对象，`p = &y`；修改地址值。这些区分会直接影响下一章的参数传递，也会在后面资源管理中决定是否发生共享、复制或所有权转移。

第 2 章 函数与可调用对象：把过程变成可组合的接口

一个函数不仅是一段可以重复执行的代码。它还划出了一条边界：调用者提供什么，函数可以修改什么，结果通过什么方式返回。接口越清楚，调用者越不需要阅读实现细节。

本章从参数与返回值讲起，再讨论重载和默认参数。最后把“要执行的过程”本身作为数据传递，理解函数指针、Lambda 与可调用对象。这些机制会直接支撑标准库算法、回调和策略模式。

2.1 函数的约定先于函数体

先读一个函数的完整写法：

```
int add(int left, int right) { // int 是返回类型，括号里声明两个参数
    return left + right;      // 把计算结果交回调用者
}
int result = add(2, 3);      // 把 2、3 传入，result 得到 5
```

调用时，实参按参数类型进入函数；函数体使用参数名字完成计算；return 决定调用表达式的结果。理解这个过程后，再问接口允许哪些输入、是否修改外部状态，以及发生错误时怎样通知调用者。

一个求和函数至少应说明输入范围、返回结果、是否修改输入，以及溢出或非法输入怎样处理。函数名和类型能表达一部分约定，剩下的需要文档、断言或错误处理补充。

```
int add(int a, int b); // declaration
int add(int a, int b) { return a + b; } // definition
```

即使名字叫 add，也不能假设它对任意两个整数都安全。内置有符号整数溢出不等于自动得到正确的大整数。课程中的小例子会选用能表示结果的输入范围。

返回值与副作用

函数可以通过返回值给出结果，也可以修改传入对象。后一种行为应能从接口或命名看出来。把一个只读查询写成会悄悄改动对象的操作，会让程序难以组合和测试。

工程实践 让常见用法容易写对

只读且体积较大的输入，常用 `const T&`；小而廉价的值，常直接传值；需要修改调用者对象时，用 `T&` 明确表达。可选目标可以用指针或其他显式可选表示。具体选择还要考虑所有权与生命周期。

2.2 参数传递：复制一个值，还是访问原对象

假设想让一个函数把计数加 1。写成 `void increase(int x)` 时，调用 `increase(count)` 只把 `count` 的值交给新的参数变量 `x`；函数内部改变 `x`，不会改变 `count`。若参数写成 `int& x`，`x` 就成为 `count` 的别名。下面按调用顺序追踪 `a`，而不是只记“引用效率高”。

值参数是一个独立对象，用实参初始化它。引用参数绑定到实参表示的对象。指针参数本身按值传递，但通过复制过来的地址，仍可以访问调用者的目标。

代码示例 三种传参方式与引用返回

```
#include <iostream>
// x 是调用者数值的副本，修改它不影响原变量。
void by_value(int x) { ++x; }
// x 是调用者变量的别名，修改会反映到原变量。
void by_reference(int& x) { ++x; }
void by_pointer(int* x) { if (x != nullptr) ++*x; }
// 返回被选中的原变量的引用，因此返回结果还能用于赋值。
int& larger(int& a, int& b) { return a > b ? a : b; }
int main() {
    int a = 3, b = 4;
    by_value(a);
    by_reference(a);
    by_pointer(&a);
    // 此时 a 从 3 变为 5，比 b 大；所以这句把 a 改为 9。
    larger(a, b) = 9;
    std::cout << a << ' ' << b << '\n';
}
```

运行追踪 输出为何是 9 和 4

9 4

`by_value` 只改自己的整数副本；`by_reference` 将 `a` 从 3 改到 4；`by_pointer` 再改到 5。`larger` 返回对较大者 `a` 的引用，最后一次赋值将它改为 9。

“指针按值传递”的准确含义

```
void redirect(int* p, int* q) { p = q; }
void redirect_caller(int*& p, int* q) { p = q; }
```

第一个函数只改变自己的指针副本，不能重定向调用者的指针变量。第二个函数把“指针对象本身”作为引用传入，因此能够修改调用者保存的地址。两层对象分别是谁，是这段代码的关键。

数组参数会丢失长度信息

函数参数中的 `int a[]` 通常被调整为 `int* a`。在函数里对 `a` 求 `sizeof`，得到的是指针大小，不是原数组元素数。应显式传入长度，使用数组引用模板，或使用 C++20 的 `std::span` 表示一段借用范围。

易错点 返回引用必须追踪目标寿命

返回调用者仍持有的对象引用，与返回局部变量引用完全不同。局部对象在函数返回时结束生命周期，引用不会把它保留下来。返回值类型写成引用，并不能让已销毁的对象重新存在。

2.3 默认参数与重载：编译器怎样选择入口

默认参数在调用点补齐省略的实参。通常从参数列表右侧开始连续提供；声明与定义分开时，应避免重复提供同一默认值。

```
void print_point(int x, int y = 0, int z = 0);
print_point(5);           // 使用默认参数，相当于传入5、0、0
print_point(5, 2);        // 只省略最后一个参数，相当于传入5、2、0
```

默认参数不改变函数类型。一个带两个默认参数的三参数函数，不会因此自动成为三个不同函数，也不能直接赋给一参数函数指针。

重载关注参数列表

仅返回类型不同，不能构成普通函数重载。按值参数的顶层 `const` 也不区分重载：`f(int)` 与 `f(const int)` 表示同一个参数类型约定。`f(int&)` 与 `f(const int&)` 则可以区分。

代码示例 提升、转换与只读重载

```
#include <iostream>
void pick(int) { std::cout << "int\n"; }
void pick(double) { std::cout << "double\n"; }
void visit(int*) { std::cout << "mutable\n"; }
void visit(const int*) { std::cout << "read-only\n"; }
int main() {
    short s = 1;
    int x = 2;
    const int c = 3;
    // short 提升为 int，选择 int 版本。
    pick(s);
    pick(2.5);
    visit(&x);
    // const 对象不能通过 int* 访问，选择 const int* 版本。
    visit(&c);
}
```

运行追踪 选择结果

```
int
double
mutable
read-only
```

`short` 到 `int` 的提升比到 `double` 的普通转换更匹配。指针重载则区分能否通过指针修改目标。

不是按源码顺序试函数

编译器先形成候选集合，再判断哪些候选可行，最后比较转换序列和其他规则。若多个候选都可行，却没有唯一更好的候选，调用就有二义性。

```
void choose(long);
void choose(double);
// choose(1); // 预期二义性：两个候选都需要转换，无法选出唯一更优者
```

默认参数也可能引入二义性。例如同时存在 `f(int)` 与 `f(int, int=0)` 时，`f(1)` 可能无法选择。接口设计应尽量减少这种需要调用者猜测的重叠。

原理剖析 重载让同一意图适应不同类型

好的重载应让调用者感到它们在做同一种事，只是输入类型不同。如果重载之间的含义相差很大，即使编译器总能选出一个，接口仍可能难以理解。

2.4 函数指针：把算法中的变化部分传进来

求 $1 + \dots + n$ 与求 $1^2 + \dots + n^2$ 的循环相同，只有“第 k 项怎么算”不同。可以把每项公式写成函数，再把这个函数交给统一的 `summation`。

`int (*term)(int)` 中，括号里的 `*term` 表示 `term` 是指针；右侧 `(int)` 表示它指向的函数接收一个 `int`；最左侧 `int` 是返回类型。`using Term=...` 只是给这串类型写法起一个容易使用的名字。函数指针也能像函数一样用 `term(k)` 调用。

求自然数和、平方和与立方和，循环结构相同，变化的是每一项如何计算。函数指针允许把这个变化点从循环里抽出来。

代码示例 同一个求和过程，三种项函数

```
#include <iostream>
// Term 是函数指针：可以保存接收 int、返回 int 的函数地址。
using Term = int (*)(int);
int summation(int n, Term term) {
    int total = 0;
    // 循环不决定每项公式，而是调用传入的函数。
    for (int k = 1; k <= n; ++k) total += term(k);
    return total;
}
int natural(int x) { return x; }
int main() {
    // natural 给出第 k 项为 k，所以求的是 1+2+3+4+5。
    std::cout << summation(5, natural) << '\n';
    std::cout << summation(5, [](int x) { return x*x; }) << '\n';
    std::cout << summation(5, [](int x) { return x*x*x; }) << '\n';
}
```

运行追踪 输入为 5 时

15
55
225

循环负责遍历和累加，`term` 负责定义每一项。两部分可以分别理解和验证。

`using Term = int (*)(int);` 表示接受一个整数、返回一个整数的函数指针类型。调用时可以写 `term(k)`，也可以写 `(*term)(k)`。函数名在相应语境中可以转换为函数指针。

这种形式不携带额外对象状态。若需要同时传入阈值、配置或计数器，可以增加参数，也可以使用带状态的可调用对象。

函数表与输入验证

函数指针可以放进数组，让一个操作编号选择函数。但操作编号必须在范围内，函数还可能有自己的定义域，例如开方要求实数输入非负。把分支改成查表，并不会自动消除输入验证责任。

2.5 Lambda：把行为与所需状态放在一起

如果一个小操作只用一次，没有必要为了它单独取一个全局函数名。Lambda 可以在使用位置直接定义：

```
auto square = [](int x) { return x * x; }; // 建立一个可调用对象
int result = square(4);                  // 调用它，得到 16
```

方括号是捕获列表，说明怎样使用外部局部变量；圆括号是参数列表；花括号是函数体。空的 `[]` 表示不捕获外部局部状态。下面再把外部变量 `x` 放入捕获列表，比较保存副本与保存引用的区别。

Lambda 表达式产生一个闭包对象。它有一个编译器生成的类类型，并提供调用操作。捕获列表决定这个对象怎样携带外部局部状态。

```
auto add_three = [step = 3](int x) { return x + step; };
int answer = add_three(5);
```

可以把它概念上理解为一个保存 `step` 成员、提供 `operator()` 的小类。这个模型有助于理解捕获，但不规定闭包的确切内存布局。

按值捕获与按引用捕获

按值捕获保存相应值的副本；按引用捕获继续访问外部对象。默认情况下，Lambda 的调用操作不允许修改按值捕获的成员。加上 `mutable` 才能修改闭包内部的副本。

代码示例 闭包中的状态与外部状态

```
#include <iostream>
int main() {
    int x = 10;
    // 闭包保存 x 的一份副本；mutable 允许修改闭包内的副本。
    auto snapshot = [x]() mutable { return ++x; };
    // 这里保存对外部 x 的引用，调用时会修改外部变量。
```



```
auto alias = [&x] { return ++x; };  
// 第一次 snapshot 把内部副本从 10 改为 11，外部 x 仍为 10。  
std::cout << snapshot() << ' ' << x << '\n';  
std::cout << alias() << ' ' << x << '\n';  
std::cout << snapshot() << ' ' << x << '\n';  
}
```

运行追踪 逐次调用

```
11 10  
11 11  
12 11
```

snapshot 内部保存自己的计数，第一次变为 11，第二次变为 12。alias 修改外部的 x，因此外部最终为 11。

易错点 引用捕获不会延长局部对象寿命

把引用捕获的闭包返回、保存到容器或交给异步任务之前，必须确认被捕获对象会活得足够久。需要独立保存状态时，可改为值捕获；需要表达资源所有权时，可使用初始化捕获和合适的拥有类型。

捕获列表不是“把所有变量全部复制”

捕获默认项指定需要隐式捕获时采用的方式，并不意味着作用域中每个变量都会无条件成为成员。全局变量和静态存储期变量通常不通过普通局部捕获来访问。

[this] 保存的是对当前对象的访问途径，不是自动复制整个对象。对象若先销毁，闭包中的访问途径就会悬垂。C++17 的 [*this] 可以复制当前对象，但是否适合复制，仍取决于类的语义。

什么时候能转成函数指针

没有捕获的适当 Lambda 可以转换成匹配的函数指针。需要捕获状态时，普通函数指针无法独自携带这份状态，所以通常应保留闭包类型、使用模板参数，或使用类型擦除包装。

2.6 组合过程与类型擦除

把“先乘 2，再加 3”写成两个函数时，可以手工写 $g(f(x))$ 。compose 要做的就是返回一个新函数对象，使调用者以后只写一次调用。这里的 F,G 让编译器保留两个可调用对象的具体类型；先读 main 中的实际调用，再看返回的 Lambda 怎样保存它们。

上一节的闭包既是行为，也是普通对象，因此可以被另一个闭包保存。这让函数组合变得自然。

代码示例 安全返回闭包并组合操作

```
#include <cassert>
#include <functional>
auto make_adder(int step) {
    // 把 step 的值保存在返回的闭包中，函数结束后仍可使用。
    return [step](int x) { return x + step; };
}
template<class F, class G>
auto compose(F f, G g) {
    // 调用返回的对象时，先算 f(x)，再把结果交给 g。
    return [f, g](auto x) { return g(f(x)); };
}
int main() {
    auto twice_then_add = compose([](int x) { return 2*x; }, make_adder(3));
    assert(twice_then_add(5) == 13);
    // 用统一的可调用接口保存具体闭包；调用形式仍是 operation(参数)。
    std::function<int(int)> operation = make_adder(10);
    assert(operation(2) == 12);
}
```

`make_adder` 按值保存 `step`，不会引用已结束的函数局部变量。组合函数先执行 `f`，再执行 `g`，因此输入 5 先变成 10，再变成 13。

模板与 `std::function` 的分工

模板参数可以保留具体可调类型，让编译器针对它生成代码。`std::function<int(int)>` 则把不同可调类型放到同一种外部接口后面，这称为类型擦除。

类型擦除便于把不同函数保存到同一个容器，代价可能包括间接调用、额外存储或动态分配。具体开销由实现和目标对象决定，不能一概说一定分配堆内存。

在 C++20 中，`std::function` 对所存目标有可复制等要求。捕获独占资源的移动专用闭包，不应直接假定能装进去。若所有调用点都能保留具体类型，模板接口通常更直接。

2.7 从函数抽象走向设计模式

求和函数中可替换的 `term`，已经展示了策略的基本思想：稳定的流程调用可变化的行为。后面可以用虚函数、模板或 `std::function` 表达同一类变化点。

这里最值得记住的不是某一种写法，而是分离职责：循环负责何时执行，项函数负责执行什么。先找到变化部分，再选择合适的语言机制，通常比先套一个模式名称更清楚。

第 3 章 类、封装与生命周期：让对象始终处于有效状态

把数据和函数写进同一个类，只是面向对象的起点。更重要的是让类承担一项清楚的责任：哪些状态有效，哪些操作允许，建立和销毁对象时分别要完成什么。

一个好的类能减少调用者需要记住的步骤。例如调用者不必先创建一个“尚未初始化”的复数，再分别设置实部与虚部。构造函数可以直接建立有效对象，成员函数则维护它的约束。

3.1 类是带操作约定的数据抽象

复数需要把实部和虚部一起保存，还需要计算模长或修改实部。class 把这些数据和相关操作组织到一个类型里。Complex z{3,4} 建立一个具体对象，z.magnitude() 调用这个对象的成员函数。

先看下面 main 中的三步：建立 z，求模长，再连续两次增加实部。类体中的 private 数据负责保存状态，public 函数提供允许的使用方式；不要先把“封装”只记成一句抽象口号。

代码示例 一个简单的复数值类型

```
#include <cassert>
#include <cmath>
class Complex {
    double real_{};
    double imag_{};
public:
    Complex() = default;
    // 冒号后直接初始化两个成员；构造函数没有返回类型。
    Complex(double r, double i = 0) : real_(r), imag_(i) {}
    double real() const noexcept { return real_; }
    double imag() const noexcept { return imag_; }
    double magnitude() const { return std::hypot(real_, imag_); }
    // 返回当前对象的引用，所以可以继续调用下一个成员函数。
    Complex& add_real(double x) { real_ += x; return *this; }
};

int main() {
    // 先建立一个对象，再通过“对象.成员函数”使用它的操作。
    Complex z{3, 4};
    assert(z.magnitude() == 5);
    z.add_real(1).add_real(2);
    const Complex& view = z;
    assert(view.real() == 6);
}
```

成员使用 `real_` 与 `imag_` 命名，访问函数使用 `real()` 与 `imag()`。这样可以清楚地区分存储与接口，也避免名字冲突。

`class` 与 `struct` 都可以包含构造函数、私有成员、继承和虚函数。它们的关键语法区别是默认成员访问权限和默认继承方式。选择哪一个，不应被误解成“一个是对象，一个只是数据”。

封装隐藏的是决定，而不只是字段

把所有字段设为私有，再为每个字段机械地生成 `getter/setter`，未必真正建立了封装。若调用者仍必须知道字段之间的内部关系，修改表示时仍会影响外部代码。

例如账户类可以提供 `withdraw(amount)`，由类检查余额与金额。若只公开 `set_balance`，所有调用者都要重复执行正确性检查。类应该封装维持约束所需的操作，而不是只把赋值换个名字。

核心概念 类不变式

类不变式是对象在约定的可观察时刻必须满足的条件。例如缓冲区的长度与存储一致，区间的起点不大于终点。构造函数建立不变式，公共操作在完成时保持不变式，析构函数按既定责任释放资源。

3.2 成员函数与 `this`

执行 `z.add_real(1)` 时，函数需要知道应修改哪一个 `Complex`。编译器通过隐含的 `this` 指针把当前对象传给成员函数。函数体中的 `real_` 可以理解为当前对象的 `this->real_`。

`return *this;` 返回当前对象本身；若返回类型是 `Complex&`，调用者拿到的是同一个对象的引用，所以紧接着的 `.add_real(2)` 仍然修改 `z`。

调用 `z.add_real(1)` 时，成员函数访问的是 `z` 的成员。`this` 提供对当前对象的访问途径，`*this` 表示当前对象。

返回 `Complex&` 并返回 `*this`，允许链式调用继续操作同一个对象。返回值若改成 `Complex`，就可能产生新值，后续调用作用的对象也会不同。

```
Complex& add_real(double amount) {  
    // 修改当前对象，再返回同一个对象，供下一次链式调用。  
    real_ += amount;  
    return *this;  
}
```

访问权限按类检查

一个类的成员函数可以访问同类其他对象的私有成员。私有并不是“只有这一个对象自己能看”，而是以类的定义和友元关系为边界。

在类外，只有被允许的公共接口可用。`obj.member`、`ptr->member` 与 `ref.member` 不会绕过访问检查。改用指针语法并不能让私有数据突然变成公共数据。

3.3 构造函数：先初始化，再进入函数体

构造函数与类同名，没有返回类型。执行 `Complex z{3,4}` 时，实参 3、4 进入构造参数 `r`、`i`；冒号后的 `real_(r)`、`imag_(i)` 直接给两个成员初值，随后才执行花括号里的函数体。

因此，初始化列表不是普通函数调用列表。对于引用成员、`const` 成员，以及没有默认构造函数的成员对象，它尤其重要：这些成员需要在函数体开始之前就建立好。

成员初始化发生在构造函数体执行之前。类成员若没有默认构造方式，就必须通过合适的初始化器建立。不能等进入函数体后，再用一次赋值代替本来缺失的构造。

```
struct Engine {  
    // Engine 需要功率参数，因此外层 Car 必须在初始化列表中提供它。  
    explicit Engine(int power) : power_(power) {}  
    int power_;  
};  
class Car {  
    Engine engine_;  
public:  
    explicit Car(int power) : engine_(power) {}  
};
```

对内置成员，初始化列表也更直接。`const` 成员和引用成员尤其需要在初始化阶段处理，不能通过普通赋值完成其初次建立。

初始化顺序由声明决定

直接基类按基类列表顺序构造，非静态成员按类内声明顺序初始化，最后执行构造函数体。写在初始化列表里的顺序不会重新安排成员的实际构造顺序。

这意味着，若一个成员的初始化依赖另一个成员，声明顺序必须支持这种依赖。更好的设计通常是减少成员初始化之间的隐含耦合。

易错点 默认构造函数可能不存在，也可能被删除

声明了其他构造函数以后，不应再假定编译器仍提供可用的无参构造函数。即使显式写了 `T() = default;`，若某个成员不能被默认构造，这个默认构造函数仍可能被定义为删除。

默认成员初始化器与构造参数

默认成员初始化器提供常用起点；构造函数的成员初始化列表可以为该次构造选择别的值。若成员既未出现在列表中，也没有默认成员初始化器，就进入该成员类型相应的默认初始化规则。

不能据此断言所有成员此时都有一个可以安全读取的值。没有明确初始化的内置整数成员，仍可能处于不可读取的状态。

3.4 析构顺序与成员对象

构造与销毁的顺序，是理解资源责任的基础。析构函数体先执行，之后按相反顺序销毁成员，再销毁基类部分。

代码示例 跟踪一个汽车对象的建立与销毁

```
#include <iostream>
#include <string>
#include <utility>
struct Trace {
    std::string name;
    // 这里把参数 s 的字符串内容复制到成员 name。
    explicit Trace(std::string s) : name(s) {
        std::cout << "+ " << name << '\n';
    }
    ~Trace() { std::cout << "- " << name << '\n'; }
};
struct Car {
    // 成员按声明顺序建立: engine 先于 tyre。
    Trace engine{"engine"};
    Trace tyre{"tyre"};
    Car() { std::cout << "+ car body\n"; }
    // 先执行 Car 析构函数体, 再按相反顺序销毁成员。
    ~Car() { std::cout << "- car body\n"; }
};
int main() { Car car; }
```

运行追踪 确定的调用顺序

```
+ engine
+ tyre
+ car body
- car body
- tyre
- engine
```

车身构造函数体执行时，两个成员已经建立。车身析构函数体执行时，成员尚未被自动销毁，因此仍能完成合适的收尾操作。

默认析构不是“什么都不做”

即使没有手写析构函数，成员对象和基类部分仍会按规则被销毁。`std::string` 成员会执行自己的析构。普通裸指针成员的销毁则不会自动 `delete` 它指向的对象。

因此，一个类是否需要手写析构函数，取决于它是否直接承担了编译器无法从成员类型中知道的释放责任。把资源包装成合适的成员类型后，外层类常可以依靠默认行为。

构造失败时销毁谁

若某个成员构造失败，已经完成构造的成员会被清理；尚未完成构造的外层对象不会像一个完整对象那样调用自己的析构函数。这是使用 RAII 成员的重要理由：部分完成的状态也能按已建立的责任安全清理。

3.5 对象数组与临时对象

对象数组按元素顺序初始化，销毁时按相反顺序处理。指针数组则只是一组指针对象；销毁指针数组，不会自动销毁所有被指向的对象。

代码示例 数组元素的生命周期

```
#include <iostream>
struct Item {
    int id;
    explicit Item(int i = 0) : id(i) { std::cout << "+" << id << '\n'; }
    ~Item() { std::cout << "-" << id << '\n'; }
};
int main() {
    // 按下标从小到大建立三个元素；离开作用域时逆序销毁。
    Item items[3]{Item{1}, Item{2}, Item{3}};
}
```

输出先是 +1,+2,+3，再是 -3,-2,-1。示例采用 C++20；这里不应再凭旧的临时对象复制模型，额外添加并不发生的复制构造输出。

作用域与完整表达式

普通自动对象通常在离开其作用域时销毁。许多临时对象在创建它们的完整表达式结束时销毁，例如一条完整语句的末尾；但也存在引用绑定延长临时对象生命周期等特殊规则。

一个局部引用获得生命周期延长，不代表把它再从函数返回后，临时对象就能继续无限存在。生命周期延长有明确语境，不能被任意传递。

工程实践 少让调用者承担隐含时序

若接口返回借用的引用、指针或视图，文档应说明其有效期。例如“在容器发生重新分配前有效”与“对象下一次修改前有效”是不同约定。能够返回廉价值时，值返回通常更容易组合。

3.6 静态成员：属于类的共享状态

如果要统计当前一共有多少个对象存活，计数器应由整个类共享。若每个对象各存一个计数器，就无法得到全体数量。下面用构造加 1、析构减 1，并通过 `Counted::live()` 查询；双冒号表示按类名访问这个共享接口。

非静态成员为每个对象提供状态；静态数据成员通常为相应类提供一份共享状态。静态成员函数没有隐式当前对象，但仍可以通过显式传入的对象访问允许访问的成员。

在 C++17 及以后，`inline static` 数据成员可以在类内定义。传统写法则常把声明放在类内，把唯一的定义放在某个源文件中。

代码示例 跟踪当前存活的对象数

```
#include <iostream>
class Counted {
    // 整个类共享一个计数器，不是每个对象各保存一个计数器。
    inline static int live_ = 0;
public:
    Counted() { ++live_; }
    // 复制也建立新对象，所以存活数量同样增加。
    Counted(const Counted&) { ++live_; }
    ~Counted() { --live_; }
    static int live() { return live_; }
};

int main() {
    {
        Counted a;
        // 内层作用域结束后 b 析构，计数会从 2 回到 1。
        { Counted b = a; std::cout << Counted::live() << '\n'; }
        std::cout << Counted::live() << '\n';
    }
    std::cout << Counted::live() << '\n';
}
```

运行追踪 建立与销毁必须对称

```
2
1
0
```

复制构造也建立了一个新对象，所以同样要增加计数。普通赋值没有建立新对象，不应无故增加存活数。

类模板的不同特化是不同类型，它们的静态成员也需要按具体特化理解。函数局部静态对象则具有另一套初始化时机：第一次执行到相应定义时进行初始化，随后调用不会反复重建。

共享状态带来的代价

共享计数便于观察，却也让对象之间产生联系。并发修改普通静态整数还可能形成数据竞争。将变量放进类内部并不会自动使它线程安全。

3.7 只读成员函数与逻辑只读

写在成员函数圆括号后面的 `const`，例如 `int value() const`，限制的是函数通过当前对象进行的修改，不是说返回值的类型变成 `const int`。一个 `const` 对象只能调用满足相应只读条件的成员接口。

同一个类可以提供可写和只读两个版本。执行 `c.value()=8` 时需要返回可写引用；通过 `const Cell& view` 读取时，则选返回只读引用的版本。下面的例子把这两个入口并排放在一起。

`const` 成员函数承诺不通过当前对象这条路径修改其普通成员。它可以与非 `const` 版本重载，为只读对象和可修改对象提供不同访问接口。

代码示例 同一个访问函数的读写版本

```
#include <cassert>
#include <type_traits>
#include <utility>
class Cell {
    int value_ = 0;
    mutable int reads_ = 0;
public:
    // 非 const 对象可得到可写引用，允许 c.value() = 8。
    int& value() noexcept { return value_; }
    // const 成员函数提供只读访问；mutable 计数仍可更新。
    const int& value() const noexcept { ++reads_; return value_; }
    int reads() const noexcept { return reads_; }
};
int main() {
    Cell c;
    c.value() = 8;
    static_assert(std::is_same_v<decltype(c.value()), int&>);
    static_assert(std::is_same_v<decltype(std::as_const(c).value()), const int&>);
    // view 与 c 是同一个对象，但经 view 选择只读重载。
    const Cell& view = c;
    assert(view.value() == 8 && view.reads() == 1);
}
```

非 `const` 版本返回 `int&`，允许写入；`const` 版本返回 `const int&`，只允许读取。`std::as_const` 可以显式选择只读视角。

mutable 与缓存

`mutable` 成员可在 `const` 成员函数中修改，常用于缓存或统计信息。这可以支持“逻辑内容不变，但内部实现状态变化”的设计。

不过，`const` 不自动保证线程安全。两个线程同时更新同一缓存，仍需要同步。指针成员也带来浅层只读问题：指针自身只读，并不一定让指向的对象只读。

原理剖析 只读承诺应对调用者有意义

用户关心的是一次查询会不会改变对象代表的值，而不是每一个实现字节是否都不变。缓存可以是合理实现，但不能借 `mutable` 隐藏本来会改变业务含义的副作用。

3.8 友元与协作接口

友元允许特定外部函数或类访问私有成员。它是类主动授予的一种权限，不能被理解成绕过封装的万能通道。

代码示例 同类访问与隐藏友元

```
#include <cassert>
class Account {
    int balance_;
public:
    explicit Account(int b) : balance_(b) {}
    bool same_balance(const Account& other) const {
        // 成员函数可以访问另一个同类对象的私有成员。
        return balance_ == other.balance_;
    }
    // 显式授权这个非成员函数访问私有余额。
    friend int total(const Account& a, const Account& b) {
        return a.balance_ + b.balance_;
    }
};

int main() {
    Account a{3}, b{4};
    assert(!a.same_balance(b));
    // total 不是成员，使用普通函数调用，而不是 a.total(b)。
    assert(total(a, b) == 7);
}
```

`same_balance` 可以读取另一个 `Account` 的私有成员。`total` 则是非成员函数，通过友元声明获得访问权。其定义写在类内，但不会因此成为带 `this` 的成员函数。

友元关系不自动对称，也不自动传递。A 信任 B，不代表 B 信任 A，更不代表 B 信任的其他类也能访问 A。

何时使用友元

需要两个对象对等参与的操作，如某些二元运算和流输出，常适合非成员接口。友元可以让这些接口直接访问必要表示。若一个外部模块需要几乎全部私有细节，则应重新检查职责是否划分合理。

对象的复制、赋值和资源释放会让这些规则进一步交汇。下一章将用一个拥有动态数组的 `Buffer`，把生命周期从调用顺序推进到真正的资源管理。

第 4 章 资源管理与复制：把责任写进对象语义

一个整数可以直接复制。一个保存动态数组地址的对象却不同：复制地址以后，究竟由谁释放数组？两个对象是否允许共享这份存储？修改其中一个时，另一个应不应该改变？

这些问题不是编译器可以从一个裸指针自动推断的。类必须给出自己的资源语义。本章用一个拥有整数数组的 `Buffer`，区分成员复制、独立复制与所有权管理。

4.1 拥有资源与借用资源

设一个函数得到一块动态数组地址。仅看 `int* p`，无法知道它应该在结束时 `delete`，还是只临时访问别人提供的数据。若双方都释放，会重复释放；若双方都以为对方负责，又会泄漏。拥有与借用首先是在回答这个实际问题：谁负责决定资源何时归还。

一个局部 `unique_ptr` 可以拥有整数，另一个普通指针可以短期借用它。两者可能指向同一地址，却承担不同责任。因此，不能只数程序中有几份指针值，就判断有几个拥有者。

拥有者承担释放责任，借用者只在约定的有效期内访问。它们都可能在内部使用地址，但职责不同。类型和接口应尽量表达这种区别。

`std::unique_ptr<T>` 表达独占拥有；普通 `T*` 或 `T&` 常用来表达借用，但裸指针本身不携带完整所有权信息。`std::shared_ptr<T>` 则表示参与共享拥有，最后一个相应拥有者释放资源。

核心概念 资源不只包括内存

文件、锁、网络连接和系统句柄都需要在合适时机释放。RAII [4] 把资源责任绑定到对象生命周期，使正常返回和异常传播都能走过已定义的清理路径。

4.2 RAII：利用析构规则自动清理

先看一种容易遗漏的写法：函数申请资源以后，可能正常返回，也可能因错误提前返回或抛出异常。若每条出口都手工写清理语句，很容易漏掉某一条路径。RAII 把清理放入拥有对象的析构函数，让“局部对象离开作用域”的规则统一处理这些出口。

下面的 `Guard` 用打印消息代替真正的资源申请与释放。正号表示建立守卫，负号表示析构。虽然没有分配真实文件或数组，调用顺序与管理资源时需要遵守的生命周期规则相同。

代码示例 正常离开与异常离开

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>
```

```

struct Guard {
    std::string name;
    explicit Guard(std::string s) : name(s) {
        std::cout << "+ " << name << '\n';
    }
    ~Guard() { std::cout << "- " << name << '\n'; }
};

void work(bool fail) {
    // 先建立 A, 再建立 B; 正常返回和抛异常都要处理它们的析构。
    Guard a{"A"}, b{"B"};
    // 异常离开 work 时先销毁局部对象, 再进入外层 catch。
    if (fail) throw std::runtime_error("failed");
}

int main() {
    work(false);
    try { work(true); }
    catch (const std::exception&) { std::cout << "handler\n"; }
}

```

运行追踪 两条路径都先清理 B, 再清理 A

```

+ A
+ B
- B
- A
+ A
+ B
- B
- A
handler

```

work(true) 抛出异常后, 已经完成构造的局部对象按逆序销毁, 然后进入外层处理器。

为什么手动清理容易漏

一个函数有多个 return、异常路径和条件分支时, 每一条离开路径都手写释放代码很难维护。RAII 把这些路径汇到同一个析构责任上。代码新增一条返回路径时, 不必再复制一段清理逻辑。

RAII 也有边界。直接终止进程等情形不等于正常栈展开, 不能假定所有局部析构总会发生。析构函数本身还应避免向外传播异常, 尤其是在已经处理另一个异常的过程中。

4.3 Buffer 的不变式与建立方式

接下来自己写一个拥有整数数组的类型。它的最小状态是“长度 + 数组地址”: size_ 说明有几个元素, data_ 指向第一个元素。长度为零时允许空指针; 长度非零时必须拥有足够大的数组。类的各个操作都要维持这组关系。



图 4.1 正常返回与异常传播都利用对象销毁完成清理。这里假设两个局部对象均已成功构造，异常被外层处理器接住。

```
class Buffer {
    std::size_t size_ = 0; // 保存元素数量
    int* data_ = nullptr; // 保存唯一负责释放的数组地址
public:
    // 构造、复制、移动和访问函数都围绕这两个成员实现
};
```

这是下面分步实现的类骨架，完整可用定义集中放在附录。读构造函数时，把每一次分配与这两个成员的状态对应起来。

我们的教学缓冲区采用明确约定：`size_` 记录元素数，非空对象独占一段对应长度的数组；空状态使用长度 0 和空指针。下标访问要求索引有效，`at` 则主动检查范围。

代码示例 构造与销毁

```
// 按数量建立数组；这里的成员初始化器在构造函数体之前执行。
explicit Buffer(std::size_t n = 0)
    : size_(n), data_(n ? new int[n]{} : nullptr) {}
// 花括号元素列表先确定长度，再把每个初始值放入数组。
Buffer(std::initializer_list<int> values) : Buffer(values.size()) {
    if (size_ != 0) std::copy(values.begin(), values.end(), data_);
}
// 对象离开作用域时归还自己拥有的数组；空指针可安全删除。
~Buffer() { delete[] data_; }
```

`Buffer(3)` 创建三个零值元素。`Buffer{1,2,3}` 使用初始化列表创建三个指定值。注意 `Buffer{3}` 表示一个值为 3 的元素，不是三个元素；圆括号与花括号的区分依然存在。

析构函数只应释放自己的资源

`delete[] data_;` 在空指针上也成立。真正的危险是两个互不协调的对象都认为自己拥有同一块数组。两次析构可能重复释放，同步修改也可能违背值语义。

如果希望两个 `Buffer` 像两个整数或两个字符串那样独立，就应为复制创建新存储，并复制元素值。这通常称为深复制。

4.4 复制构造与复制赋值

先用两条调用区分它们：

```
Buffer a{1, 2, 3}; // a 拥有三个整数
Buffer b = a;      // b 还不存在：调用复制构造
b = a;             // b 已经存在：调用复制赋值
```

复制的目标是两份独立值。b 修改自己的元素时，a 不应跟着改变。对于裸数组指针，复制一个地址无法实现这个目标；复制构造需要分配新数组并复制元素。

赋值还多一个问题：b 原来可能已经有数组。下面先创建临时副本，再交换状态。复制成功之前不改 b；交换之后，临时对象拿到 b 的旧资源，离开作用域时负责释放。按这三步读代码，就能理解 `copy-and-swap` 为什么既处理旧资源，又提供失败时的保证。

复制构造建立一个新对象；复制赋值修改一个已经存在的对象。后者可能已经拥有旧资源，因此需要额外决定旧资源何时释放。

代码示例 独立复制与 `copy-and-swap`

```
// 复制构造先申请自己的新数组，再复制元素，双方互不共享所有权。
Buffer(const Buffer& other) : Buffer(other.size_) {
    if (size_ != 0) std::copy_n(other.data_, size_, data_);
}

Buffer& operator=(const Buffer& other) {
    if (this != &other) {
        // 先复制成功再改目标；若分配失败，目标原内容保持不变。
        Buffer temporary(other);
        // 交换后 temporary 拿到目标的旧数组，随后由它的析构函数释放。
        swap(temporary);
    }
    return *this;
}

void swap(Buffer& other) noexcept {
    std::swap(size_, other.size_);
    std::swap(data_, other.data_);
}
```

复制构造先委托给长度构造函数，再复制整数元素。对于长度为零的状态，显式跳过复制，避免把空指针参与不必要的范围运算。

复制赋值先构造临时副本，再交换资源。临时对象最后销毁时，释放原来属于目标对象的旧资源。这种安排把“准备新状态”与“替换旧状态”分开。

原理剖析 先完成可能失败的准备，再提交改变

如果临时副本的分配失败，目标对象尚未被修改。交换操作不抛异常，就可以安全地提交新状态。对这个整数缓冲区，这种结构提供了强异常保证：失败时目标保持原值。

自赋值为什么需要考虑

`a = a` 的两边是同一个对象。若先删除目标数组，再从源数组复制，源数据也已经被删除。这会形成释放后访问，问题不是自动发生“无限递归”。

显式的身份检查是一种处理方式。合理的 copy-and-swap 设计也可以天然支持自赋值，只是可能做一次额外复制。应根据实现分析，不必把某一行判断当成所有赋值运算符的固定模板。

代码示例 用行为检查复制是否独立

```
#include "buffer.hpp"
#include <cassert>
#include <iostream>
int main() {
    Buffer a{1, 2, 3};
    // b 正在建立，调用复制构造，得到独立数组。
    Buffer b = a;
    b[0] = 9;
    Buffer c{7};
    // c 已经存在，调用复制赋值，替换它原来的内容。
    c = a;
    const Buffer& self = a;
    // 来源与目标是同一个对象，用来检查自赋值仍然正确。
    a = self;
    assert(a.data() != b.data() && a.size() == c.size());
    std::cout << a[0] << ' ' << b[0] << ' ' << c[0] << '\n';
}
```

输出为 1 9 1。修改副本 b 不影响 a，给 c 赋值后它获得同样的内容，自赋值也保持原值。

4.5 编译器生成的复制到底做什么

默认复制按成员和基类分别执行它们自己的复制操作。`int` 成员复制值，裸指针成员复制地址，`std::string` 成员执行字符串的复制，`std::unique_ptr` 成员则可能使外层复制被删除。

所以“默认复制总是浅复制”过于笼统。关键是成员类型的复制语义。设计良好的成员类型，可以让外层类获得正确的默认行为。

代码示例 零法则：让成员管理自己的资源

```
#include <cassert>
#include <string>
#include <vector>
struct Record {
```

```

    std::string title;
    std::vector<int> values;
};
int main() {
    Record a{"sample", {1, 2, 3}};
    // 编译器逐个复制成员；string 和 vector 已实现各自正确的资源语义。
    Record b = a;
    // 副本中的 vector 是独立值，修改它不会修改 a 的元素。
    b.values[0] = 9;
    assert(a.values[0] == 1 && b.values[0] == 9);
}

```

这个类没有手写析构、复制或移动操作，却可以安全复制字符串与动态数组。资源责任已经由标准库成员承担。

工程实践 三法则、五法则与零法则

若一个类直接拥有需要手动释放的资源，应一起考虑析构、复制构造和复制赋值；加入移动后，还要考虑移动构造与移动赋值。更常见的工程目标是零法则：选择正确的拥有类型，让外层类不再手写这些特殊成员。

这些是设计指南，不是机械语法要求。有些类型故意不可复制，有些只允许移动，有些代表共享实体。先确定语义，再选择默认、删除或自定义操作。

4.6 独占所有权：std::unique_ptr

如果不需要自己实现数组类，可以把资源交给标准库拥有者。std::make_unique<int>(42) 建立一个保存 42 的整数，并返回负责释放它的对象。通过 *owner 访问整数，通过 owner.get() 临时借出地址。

unique_ptr 不允许复制出第二个独立拥有者；需要把责任交给函数时，用 std::move(owner) 让接收方接管。下面的 consume 按值接收拥有者，所以函数结束时释放整数，原 owner 随交接变为空。这里先掌握使用结果，移动构造的具体实现会在移动语义章展开。

独占指针不能随意复制，因为复制会产生两个互不协调的唯一拥有者。它可以转移所有权，也可以通过 get() 提供一个不拥有的观察指针。

代码示例 借用访问与接收所有权

```

#include <cassert>
#include <memory>
#include <utility>
// 按值接收 unique_ptr，函数承担这个整数的所有权。
int consume(std::unique_ptr<int> value) { return *value; }
int main() {
    auto owner = std::make_unique<int>(42);
    // get 只借出地址；observer 不负责释放整数。
    int* observer = owner.get();
}

```

```

    assert(*observer == 42);
    // 把所有权交给参数；consume 返回时参数析构并释放整数。
    int result = consume(std::move(owner));
    assert(result == 42 && owner == nullptr);
    // observer is now dangling; do not dereference it.
}

```

`consume` 按值接收独占指针，表示接收所有权。调用结束后，参数对象销毁并释放整数。原来的 `observer` 不会自动被置空，但它已经不能再解引用。

get、release 与 reset

`get()` 只查看地址，不放弃所有权。`release()` 放弃所有权并返回地址，之后调用者必须接手释放责任。`reset()` 则释放当前资源并接管新资源或变为空。

这三个操作看起来都与指针有关，却有完全不同的责任变化。尤其不要把 `get()` 得到的地址再次交给另一个独立的拥有者。

4.7 共享所有权与 `weak_ptr`

假设两个组件都需要让同一个对象继续存活，且很难指定谁一定最后离开，可以共享所有权。`make_shared<int>(7)` 建立一个共享对象；复制 `shared_ptr` 会增加拥有者，而不是复制整数。

如果另一个组件只是想“有对象时观察它”，但不应因此阻止销毁，则使用 `weak_ptr`。访问前先调用 `lock`：成功返回临时强引用，失败返回空 `shared_ptr`。下面用嵌套作用域与 `reset` 逐步减少拥有者，观察对象何时过期。

共享指针通过共同的控制信息管理资源。当最后一个相应强拥有者离开时，目标对象销毁。弱指针可以观察共享对象，但不会让目标一直存活。

代码示例 在使用前尝试获得临时拥有权

```

#include <cassert>
#include <iostream>
#include <memory>
int main() {
    auto owner = std::make_shared<int>(7);
    // 弱引用观察目标，不增加负责延长目标寿命的强引用计数。
    std::weak_ptr<int> observer = owner;
    {
        // 复制 shared_ptr 共享同一个整数，不是复制出第二个整数。
        auto another_owner = owner;
        assert(owner.use_count() == 2);
    }
    // lock 成功时临时取得强引用，使用期间保证目标仍存在。
    if (auto locked = observer.lock()) std::cout << "alive\n";
    // 最后一个强拥有者离开后整数销毁，weak_ptr 随后显示过期。
    owner.reset();
}

```

```
    assert(observer.expired());
    std::cout << "expired\n";
}
```

输出先是 `alive`，再是 `expired`。`lock()` 成功时返回一个共享拥有者，让这次访问期间对象保持有效。

共享不等于线程安全，也不等于没有生命周期问题

多个共享指针可以安全管理共同的拥有计数，并不意味着被指向对象的所有成员操作都线程安全。若两个对象通过强共享指针互相拥有，还可能形成循环，导致外部拥有者消失后仍无法释放。

此时需要重新检查关系：谁真正拥有谁，谁只是反向观察？适当的 `weak_ptr` 或明确的层次拥有关系，通常比给所有边都套上共享指针更清楚。

4.8 把资源语义落实到接口

接口形态	常见意图
<code>const T&</code>	借用，只读访问
<code>T&</code>	借用，允许修改目标
<code>T*</code>	常用作可空借用；具体约定需明确
<code>std::unique_ptr<T></code>	接收独占所有权
<code>std::shared_ptr<T></code>	接收或保存共享拥有关系
<code>std::span<T></code>	借用一段连续元素，携带长度

表 4.1 类型表达了意图的一部分，生命周期与失败条件仍属于接口契约。

一个只想读取对象的函数，不应仅为方便而要求共享指针参数。反过来，一个需要在调用结束后保存对象的组件，也不应假装只做短期借用。

后面的运算符重载会讨论如何让复制、赋值和访问表现得像一个自然的值类型。移动语义则会在保持对象有效的前提下，避免某些不必要的资源复制。

第 5 章 运算符与值语义：让自定义类型自然地参与表达式

运算符重载的目标，是让一个类型按照清楚的语义参与表达式。复数相加、区间比较、流输出与数组下标，都可以拥有简洁的写法。但简洁语法也会放大误解：看起来像普通加法的操作，如果悄悄修改了两个输入，就很难使用。

本章从一个复数类型出发，讨论成员与非成员运算符、赋值、输入输出、下标与自增。重点不是列出所有可以重载的符号，而是让操作的行为与调用者的直觉一致。

5.1 重载不会改变语法结构

已经有复数加法成员函数时，我们可以写 `add(a,b)`。若类型像普通数值一样使用，写成 `a+b` 更自然。运算符重载做的是把这类符号表达式连接到一个函数，并不是发明新的运算优先级。

例如 `Complex operator+(Complex a,const Complex& b)` 是一个函数，名字是 `operator+`；`a+b` 在相应规则下调用它。下面先实现会修改左边的 `+=`，再利用它实现返回新值的 `+`。

运算符重载本质上定义了具有特定名字的函数。它不能创造新的运算符，也不能随意改变已有优先级和结合方式。大多数运算符至少要有一个类或枚举类型的操作数，不能把两个内置整数的加法整体改写。

在 C++20 中，赋值、下标、调用和箭头运算符具有必须作为相应成员函数等限制。成员访问的点运算符、作用域解析和三目条件运算符等不能通过普通运算符重载改写。

核心概念 先定义操作语义，再选择语法

`a+b` 通常产生新值，`a+=b` 通常修改左操作数，`a==b` 通常比较值是否等价。这些常规行为不是所有返回类型都被语法强制规定，却是一个好接口应认真维护的约定。

5.2 成员与非成员：二元操作是否对称

成员加法 `a.operator+(b)` 隐含一个当前对象。非成员加法 `operator+(a,b)` 则把两个操作数都放在参数列表中，更适合表达对等关系。

代码示例 复合赋值作为核心实现

```
#include <cassert>
#include <iostream>
class Complex {
    double real_, imag_;
public:
    Complex(double r = 0, double i = 0) : real_(r), imag_(i) {}
    // 复合赋值修改当前对象，最后返回当前对象的引用。
```

```

double squared_norm() const { return real_*real_ + imag_*imag_; }
Complex& operator+=(const Complex& rhs) {
    real_ += rhs.real_;
    imag_ += rhs.imag_;
    return *this;
}
// 左操作数按值传入：先得到副本，在副本上做 +=，不修改原 lhs。
friend Complex operator+(Complex lhs, const Complex& rhs) {
    return lhs += rhs;
}
// C++20 的 default 比较按成员比较 real_ 和 imag_。
friend bool operator==(const Complex&, const Complex&) = default;
friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
    return os << '(' << z.real_ << ',' << z.imag_ << ')';
}
};

int main() {
    Complex z{1, 2};
    assert(z.squared_norm() == 5);
    assert(2 + z == z + 2);
    // 把相加所得临时复数交给输出重载，预期显示 (4,6)。
    std::cout << z + Complex{3, 4} << '\n';
}

```

`operator+=` 修改当前对象并返回引用。非成员 `operator+` 接收一个左值副本，在副本上累加，再返回结果。于是加法与复合加法共用一份主要逻辑。

为什么 $2+z$ 和 $z+2$ 都成立

这个示例允许用一个 `double` 隐式构造复数。非成员函数的两个参数都可以参与适当转换，因此整数放在左边或右边都能得到相同意图。

如果只提供成员 `Complex::operator+(int)`，它不会自动让左操作数为整数的 $2+z$ 成立。需要从候选函数与参数转换的角度分析，而不是认为“数学上可交换，所以编译器会交换两边”。

隐藏友元仍是非成员

定义写在类里的友元运算符可以直接访问私有表示，并通过相关名字查找参与表达式。这种写法便于把类型及其对等操作放在一起，但它没有隐式 `this`。

5.3 比较：相等与排序是两套要求

相等比较描述两个值是否等价。排序比较则要满足算法要求的不等关系。例如许多排序和有序容器要求比较器形成严格弱序，不能随意写成 `a <= b`。

对复数，按实部、按模长和按字典序排序代表不同任务。与其给类型强加一个含糊的“自然小于”，通常更适合把排序策略作为显式比较器传给算法。

```

auto by_norm = [](const Complex& a, const Complex& b) {

```



```
// 比较类提供的模长平方；相同模长视为排序等价。  
return a.squared_norm() < b.squared_norm();  
};
```

这里的 `norm_squared` 是相应领域查询接口的示意。若两个不同复数具有相同模长，它们可以在排序关系下等价，却不一定满足复数相等比较。

易错点 比较器必须在多次调用中保持一致

若比较器改变被比较元素，或者根据不断变化的外部状态返回相互矛盾的结果，排序算法的前提就可能被破坏。结果错误不一定发生在比较函数内部，可能在算法深处才表现出来。

C++20 可以为合适类型默认生成相等比较。需要顺序比较时，还可使用三路比较等机制，但返回的比较类别应符合数据本身的关系，不能把所有值都当成完整全序。

5.4 赋值与链式操作

赋值发生在已有对象上。`T b = a;` 是初始化，`b = a;` 才是赋值表达式。前者通常涉及构造，后者通常涉及赋值运算符。

经典复制赋值签名为 `T& operator=(const T&)`。返回左对象引用，使 `a=b=c` 可以继续使用赋值结果。语法允许某些不同返回类型，但偏离惯例需要很充分的理由。

别把复制责任分散到多个不一致的入口

如果同时提供从字符串构造、从字符串赋值以及从同类对象赋值，这几条路径都应维护同一组不变式。尤其要考虑源字符串是否可能位于目标自己的存储中。

例如给字符串类执行 `s = s.c_str()+1` 时，若先释放旧存储，就把源字符一起释放了。先准备新值，再替换旧值，能同时处理这种别名风险与分配失败。

工程实践 让操作复用同一个正确的核心

加法可以复用复合赋值，复制赋值可以复用复制构造与交换。减少重复实现不只是缩短代码，还能减少不同入口之间出现行为差异的机会。

5.5 输入输出：链式调用与失败状态

想让 `in >> p` 读取一个坐标，可以重载输入运算符。输入流是左操作数，`Point` 是右操作数，所以函数接收 `std::istream&` 和 `Point&`，并返回同一个流的引用。这样 `in >> p >> q` 才能把第一次调用的结果继续作为左操作数。

例子要求格式为“整数、逗号、整数”。不要边读边修改 `p`：先读入局部变量，确认逗号与两个整数都合法，最后一次性提交新坐标。这样坏输入不会让 `p` 落在只更新一半的状态。

流输出通常返回同一个输出流的引用，让下一次插入继续作用于它。不要在重载中无条件写向 `std::cout`，否则把类型写入文件或字符串流时就会失败。

输入更需要约定：读到一半失败时，目标对象处于什么状态？下面的实现先把数据读到临时变量，验证格式后再更新目标。

代码示例 输入失败时保持原对象

```

#include <cassert>
#include <istream>
#include <sstream>
struct Point { int x{}, y{}; };
std::istream& operator>>(std::istream& in, Point& p) {
    // 先读到临时变量；整个格式确认成功后才修改目标 p。
    int x, y;
    char comma;
    if (in >> x >> comma >> y) {
        // 输入必须使用逗号；错误分隔符设置流的失败状态。
        if (comma == ',') p = {x, y};
        else in.setstate(std::ios::failbit);
    }
    // 返回同一个输入流，使 in >> p >> q 可以继续链式读取。
    return in;
}
int main() {
    Point p{9, 9};
    std::istringstream good{"1,2"};
    good >> p;
    assert(good);
    assert(p.x == 1 && p.y == 2);
    std::istringstream bad{"3;4"};
    // 格式错误不覆盖 p 里已保存的 1 和 2。
    bad >> p;
    assert(!bad);
    assert(p.x == 1 && p.y == 2);
}

```

输入 1,2 成功后，点变为 (1,2)。输入 3;4 的分隔符不合法，流被标记为失败，点仍保持 (1,2)。这把输入解析和状态提交分开了。

原理剖析 错误状态也是接口的一部分

调用者不仅需要知道成功时得到什么，还需要知道失败如何被发现、旧状态是否保留。流的状态位、异常和显式错误返回值，都是表达这个约定的方式。

5.6 下标、检查访问与只读重载

当 b 保存一组整数时，希望既能读取 `int x=b[0]`，又能写入 `b[0]=7`。后者要求下标操作返回元素引用，而不是复制出来的整数。`operator[]` 的函数体负责把下标映射到对应元素。

通过 `const Buffer` 读取时，应返回 `const int&`，阻止这条只读访问路径用于赋值。至于下标是否合法，是另一项独立约定：本书 `[]` 不检查，`at` 会检查并在越界时抛异常。

`operator[]` 通常提供简洁的索引访问，`at` 则可以主动检查边界。若返回引用，调用者可以直接修改元素；`const` 对象则应得到只读访问。

代码示例 Buffer 的访问接口

```

std::size_t size() const noexcept { return size_; }
int* data() noexcept { return data_; }
const int* data() const noexcept { return data_; }
int& operator[](std::size_t i) noexcept { return data_[i]; }
const int& operator[](std::size_t i) const noexcept { return data_[i]; }
// 检查下标以后再返回元素引用；与不检查范围的 [] 区分。
int& at(std::size_t i) {
    if (i >= size_) throw std::out_of_range("Buffer::at");
    return data_[i];
}
const int& at(std::size_t i) const {
    if (i >= size_) throw std::out_of_range("Buffer::at");
    return data_[i];
}

```

代码示例 验证访问语义

```

#include "buffer.hpp"
#include <cassert>
#include <utility>
#include <type_traits>
int main() {
    Buffer b{1, 2};
    // [] 返回元素引用，因此赋值写回 Buffer 内部。
    b[0] = 7;
    static_assert(std::is_same_v<decltype(b[0]), int&>);
    static_assert(std::is_same_v<decltype(std::as_const(b)[0]), const int&>);
    // 长度为2，只存在下标0和1；at(2) 按约定抛异常。
    try { b.at(2); assert(false); }
    catch (const std::out_of_range&) {}
    assert(b.at(0) == 7);
}

```

`b[0]=7` 修改内部元素。`std::as_const(b)[0]` 的类型是只读引用。`b.at(2)` 对长度为 2 的缓冲区越界，因此抛出异常。

返回引用意味着暴露有效期

返回的引用并不拥有元素。容器销毁、重新分配或某些结构修改后，引用可能失效。一个语法上很方便的下标接口，背后仍然有生命周期契约。

5.7 前置与后置自增

先想清楚调用结果：执行 `old=c++` 时，`old` 应拿到增加前的值，`c` 自己增加 1；执行 `current=++c` 时，先增加，再返回当前对象。下面按这个顺序实现两个函数。

函数参数表里的那个 `int` 只用来区别后置版本。`Counter& operator++() &` 中有两处不同的 `&`：返回类型后的 `&` 表示返回引用，参数括号后的 `&` 则限制此成员函数用于左值对象。它们不要混在一起读。

前置自增通常先修改对象，再返回当前对象引用。后置自增通常保留旧值，修改当前对象，再返回旧值副本。后置版本的 `int` 参数是语法区分标记。

代码示例 用引用限定保持常规用法

```
#include <cassert>
class Counter {
    int value_;
public:
    explicit Counter(int n = 0) : value_(n) {}
    // 前置 ++: 先修改，再返回当前对象；末尾的 & 限定左值调用。
    Counter& operator++() & { ++value_; return *this; }
    // 参数 int 只用于区分后置形式 c++, 不是调用者传入的增量。
    Counter operator++(int) & {
        // 先留一份旧值，修改当前对象后把旧值按值返回。
        Counter old = *this;
        ++*this;
        return old;
    }
    int value() const { return value_; }
};

int main() {
    Counter c{3};
    Counter old = c++;
    assert(old.value() == 3 && c.value() == 4);
    Counter& current = ++c;
    assert(&current == &c && c.value() == 5);
}
```

`c++` 对应后置版本，`++c` 对应前置版本。例子中的成员函数带 `&` 引用限定，只允许对相应左值对象调用。因此不能再把返回的临时旧值当成普通可继续自增的左值使用。

易错点 不要把某个类的规则推广成全部重载规则

后置自增返回什么、是否允许对临时对象调用，取决于类的声明。语言不会强制所有自定义类型都完全模仿整数。讨论复杂表达式前，应先查看实际签名和引用限定。

5.8 调用操作与显式转换

`operator()` 让对象像函数一样被调用。对象中可以保存状态，调用操作根据状态完成计算。

代码示例 一个带状态的仿射函数

```
#include <cassert>
struct Affine {
    int scale;
    int offset;
    // 给对象定义括号调用语法，相当于把带状态的函数放进类里。
    int operator()(int x) const { return scale*x + offset; }
};
int main() {
    // 对象保存 scale=2、offset=3；因此 f(5) 按 2*5+3 计算。
    Affine f{2, 3};
    assert(f(5) == 13);
}
```

它与函数一章的捕获 Lambda 具有相似的概念结构：状态与行为被放在同一个对象中。

转换函数则改变对象在某个语境中的类型视角。例如句柄可以提供显式的布尔检查：

代码示例 允许条件检查，避免随意数值转换

```
#include <cassert>
class Handle {
    bool valid_;
public:
    explicit Handle(bool valid) : valid_(valid) {}
    // 转换目标写在 operator 后面；该函数没有另写返回类型。
    explicit operator bool() const noexcept { return valid_; }
};
int main() {
    Handle h{true};
    // 条件语境允许这个显式 bool 转换；不是任意数值转换都允许。
    if (h) { assert(static_cast<bool>(h)); }
}
```

`explicit operator bool()` 可以用于 `if(h)` 等布尔条件语境，也可以显式转换。不能笼统地说加上 `explicit` 后只有写出转换语法才能使用。

避免用符号制造意外语义

重载逻辑与、逻辑或以后，不能期待普通函数式重载仍像内置逻辑运算一样跳过另一个操作数。重载逗号、算术符号或箭头也可能让代码难以读懂。技术上能做到，与接口上值得这样做，是两回事。

第 6 章 继承与组合：表达关系，也承担约束

两个类有相似代码，不一定应该建立继承关系。继承会影响名字查找、访问权限、对象构造和隐式转换，还会让调用者期待派生对象能够履行基类的承诺。

本章先解释这些语言机制，再讨论什么时候更适合组合与委派。理解对象中的基类部分，比把继承简单理解成“把父类代码复制过来”更准确。

6.1 派生对象中的基类子对象

先把继承写成一个可以使用的程序。Employee 在已有 Person 的基础上增加工号，并继续使用姓名接口：

```
struct Person {
    std::string name;
    explicit Person(std::string n) : name(n) {}
};
struct Employee : public Person {
    int id;
    Employee(std::string n, int number) : Person(n), id(number) {}
};
Employee worker{"Lin", 7}; // 同时建立 Person 部分和 id 成员
Person& person = worker; // 借用同一对象的基类部分，没有复制出新人
```

冒号后的 `public Person` 指定公有继承；构造初始化中的 `Person(n)` 负责建立基类部分。worker 仍是一个完整的 Employee，person 只是通过 Person 接口看它。

公有继承允许一个派生对象在合适语境中作为基类使用。派生对象包含基类子对象，但这并不意味着每个基类成员都能被派生类代码直接访问。基类的私有表示仍由基类维护。派生类可以通过允许的接口操作它，而不能直接在自己的构造初始化列表中初始化基类内部的私有字段。

类型结构不等于固定字节布局

基类子对象如何摆放，可能受到对齐、空基类优化、多继承、虚继承和实现 ABI 的影响。不能把 `sizeof(Derived)` 简单写成基类大小加上新增字段大小，也不能假定所有上转型都只是保持地址数值不变。

原理剖析 视角改变，对象未必改变

把 `Derived&` 转成 `Base&`，通常是在访问同一个派生对象里的基类子对象，没有新建一个独立基类值。把派生对象按值初始化一个 `Base`，才会产生一个只具有基类类型的新对象。

6.2 访问权限与继承方式

继承方式	基类 public 成员	基类 protected 成员	外部隐式上转型
public	保持 public	保持 protected	在无歧义等条件下可用
protected	变为 protected	保持 protected	一般不能从外部使用
private	变为 private	变为 private	一般不能从外部使用

表 6.1 基类 private 成员的存储仍存在，但派生类不会因此获得直接访问权。

保护成员的访问还有对象类型方面的限制。派生类通常可以通过自己或同类派生对象访问相应保护成员，不能据此去修改任意独立基类对象的保护字段。

访问检查不会替代名字查找

如果两个基类都声明了同名成员，先出现的可能是查找二义性。把其中一个成员设为私有，并不总能消除这种二义性。应使用明确的限定、转发函数或更清楚的接口设计。

私有继承有其专门用途，但它通常不向外部表达“是一个基类”的可替换关系。若主要目的是使用一个组件的功能，组合常更容易表达。

6.3 名字隐藏、重载与覆盖

假设基类有 `print(int)`，派生类又写了 `print(const char*)`。看起来只是新增一种参数类型，但普通成员名字查找会先找到派生类的 `print`，并可能把基类同名集合隐藏。下面用 `using Base::print`；明确把原有重载引回，让两种参数都能调用。

派生类声明了同名成员时，可能隐藏基类中整个同名集合，即使参数不同。隐藏与虚函数覆盖是不同机制。

代码示例 把基类重载重新引入候选集合

```
#include <iostream>
struct Base {
    void print(int) const { std::cout << "int\n"; }
};
struct Derived : Base {
    // 把基类的同名重载放回候选集合，否则派生类的名字会隐藏它。
    using Base::print;
    void print(const char*) const { std::cout << "text\n"; }
};
int main() {
    Derived d;
    // 选择继承来的 int 版本；下一句选择派生类的字符串版本。
    d.print(3);
    d.print("hello");
}
```

输出为 `int` 与 `text`。 `using Base::print;` 使基类版本参与合适的查找。若删掉它，不能期待编译器无条件穿过派生类的同名声明去寻找另一个参数列表。

限定名表达明确选择

`d.Base::print(3)` 显式选择基类中的名字。对于虚函数调用，显式限定还会影响是否发生动态分派，下一章会进一步说明。

不要把所有同名现象都称为“覆盖”。只有满足相应签名等规则的虚函数，才构成覆盖关系。`override` 可以让编译器检查这个意图。

6.4 构造与销毁顺序

派生类构造时，先处理虚基类与直接基类，再初始化成员，最后执行派生类构造函数体。虚基类由最派生类负责初始化；普通直接基类按其声明顺序构造。

```
#include <iostream>
struct Base {
    explicit Base(int) { std::cout << "base\n"; }
    ~Base() { std::cout << "~base\n"; }
};
struct Member {
    explicit Member(int) { std::cout << "member\n"; }
    ~Member() { std::cout << "~member\n"; }
};
class Derived : public Base {
    Member member_;
public:
    Derived(int base_arg, int member_arg)
        : Base(base_arg), member_(member_arg) { // 先基类，再成员，最后进入函数体
        std::cout << "body\n";
    }
    ~Derived() { std::cout << "~body\n"; } // 函数体结束后再自动析构成员和基类
};
int main() { Derived object{1, 2}; }
```

该程序的预期输出依次为 `base`、`member`、`body`、`~body`、`~member`、`~base`。构造顺序不是由观察某次运行猜出，而是由语言规则规定；日志把这个规则显示出来。

析构大体沿相反方向展开：派生析构函数体先执行，再销毁派生成员和基类部分。把基类当作需要先建立的一部分，就容易理解这个顺序。

继承构造函数

代码示例 复用基类的构造入口

```
#include <cassert>
struct Base {
    int id;
    explicit Base(int n) : id(n) {}
};
struct Derived : Base {
    // 使用基类提供的构造入口建立基类部分。
    using Base::Base;
    // 派生类自己的成员仍按自己的默认成员初值建立。
    int tag = 7;
};
int main() { Derived d{3}; assert(d.id == 3 && d.tag == 7); }
```

`using Base::Base;` 让派生类可以使用相应基类构造入口，新增成员仍按自己的初始化规则建立。这不是把基类构造函数变成一个可以随意作用于任何对象的普通成员函数。

默认、复制和移动构造等特殊成员还各有规则。不能因为使用了继承构造，就认为派生类不再需要考虑自己的状态和不变式。

6.5 切片与基类部分赋值

`Base& view=derived` 只是建立引用；`Base copy=derived` 却要建立一个真正的 `Base` 对象。这个新对象只包含 `Base` 的状态，不能凭空容纳 `Derived` 额外的成员，这就是按值切片。

另一种容易混淆的情况是先用 `Base` 引用绑定某个 `Derived`，再对引用赋值。赋值操作作用于其中的 `Base` 子对象，不会把整个 `Derived` 的额外状态一起替换。下面用两组不同数值让变化一眼可见。

按值创建基类对象时，派生部分不会跟着成为新基类对象的一部分。这通常称为对象切片。

代码示例 复制基类值与只给基类部分赋值

```
#include <cassert>
struct Base {
    int base_value;
    explicit Base(int x) : base_value(x) {}
};
struct Derived : Base {
    int extra;
    Derived(int x, int y) : Base(x), extra(y) {}
};
int main() {
    Derived a{1, 10}, b{2, 20};
    // view 指向 b 内部的基类子对象，没有新建一个 Base。
    Base& view = b;
```

```

// 只给 b 的基类部分赋值，extra 仍为20。
view = a;
assert(a.base_value == 1 && a.extra == 10);
assert(b.base_value == 1 && b.extra == 20);
// 按值建立一个独立 Base，派生类 extra 不进入这个新对象。
Base copied = a;
assert(copied.base_value == 1);
}

```

view 绑定到 b 的基类部分。view=a 调用相应基类赋值，只修改 b.base_value，不会修改 b.extra。源对象 a 不变。

为什么这会破坏派生类的不变式

假设派生类要求基类字段与新增字段之间保持某种关系。只更新基类部分，可能让关系失效。此时应限制通过基类进行的公开赋值，或重新设计值语义与多态接口。

易错点 指针转换不能补出不存在的对象

把一个真实的 Base 对象地址强行解释成 Derived*，不会在后面自动增加派生字段。访问这些并不存在的成员可能导致未定义行为。某次运行恰好打印出一个数，不构成合法性证明。

如果需要复制“真实动态类型”的对象，应考虑明确的虚克隆接口，而不是依靠按值传入基类偷偷保留派生信息。

6.6 多继承与虚继承

多继承允许一个类具有多个直接基类。若各基类提供互不冲突的小接口，可以表达一个对象具备多种能力；若各自携带复杂共享状态，结构就更难分析。

菱形继承中，两条路径可能各有一份相同基类子对象。虚继承可以让它们共享相应虚基类子对象。

代码示例 最派生类初始化共享虚基类

```

#include <cassert>
struct Root { int value; explicit Root(int v) : value(v) {} };
// virtual 继承表示在最派生对象中共享同一个 Root 子对象。
struct Left : virtual Root { Left() : Root(1) {} };
struct Right : virtual Root { Right() : Root(2) {} };
// Leaf 负责建立共享虚基类，所以最终 Root 的值取9。
struct Leaf : Left, Right { Leaf() : Root(9) {} };
int main() {
    Leaf object;
    Root* via_left = static_cast<Left*>(&object);
    Root* via_right = static_cast<Right*>(&object);
    // 从左右两条路径转换，最终都到达同一份 Root。
    assert(via_left == via_right);
}

```

```
    assert(object.value == 9);  
}
```

Leaf 中只有一份相应 Root，由 Leaf 的构造指定值 9。单独创建 Left 或 Right 时，则由它们各自的构造决定虚基初始化。

复杂性来自关系，而不只是语法

虚继承会影响构造责任、布局和转换。它解决了共享基类子对象的问题，却不会自动解决接口冲突与业务语义冲突。只有确实需要这种关系时才值得引入。

6.7 组合与委派：复用功能而不承诺替换

组合把一个对象作为另一个对象的组成部分。委派则让一个对象把某项工作交给另一个对象完成。例如汽车可以保存一个动力组件，并把启动动作交给它；不必为了复用动力代码而让汽车继承发动机。

拥有关系不能仅从“字段是不是裸指针”判断。unique_ptr 成员可以表达拥有的组成部分，引用或普通指针可能只是关联。应看谁负责生命周期，而不只是字段语法。

工程实践 先判断关系，再选择机制

如果外部客户应该能够把派生对象当作基类使用，就必须认真考虑替换契约。若只是内部使用另一对象的功能，组合与委派通常更直接。后面的设计原则会用具体反例说明这种区别。

继承让我们有了统一的静态接口，但还没有解释为什么通过这个接口可以调用不同对象的不同实现。动态分派正是下一章的主题。

第 7 章 多态与运行时行为：同一个接口，不同的实现

一个图形列表中可以同时放圆形和矩形。调用者只想逐个绘制，不希望每增加一种形状就修改一串类型判断。虚函数让共同接口把具体行为交给实际对象。

但动态分派并不会接管全部语言规则。名字查找、访问权限、默认参数和许多重载选择仍由静态类型决定。把这两层分开，是读懂多态代码的关键。

7.1 静态类型与动态类型

如果要在一个循环里绘制圆和矩形，可以让它们都提供 Shape 的 draw 接口。通过 Shape& 调用时，程序再根据实际对象选择绘制方法，这就是这里需要的运行时多态。

先分清两个描述：变量 s 的声明类型是 Shape 引用，称为静态类型；它实际绑定到 Circle 对象，实际对象类型是动态类型。下面同时安排一个普通成员 kind 和一个 virtual 成员 draw，让相同引用产生不同的选择过程。

Shape& s=c; 中，表达式 s 的静态类型是 Shape 这一基类视角；它引用的完整对象可以是 Circle。静态类型决定哪些名字能够被找到，虚函数机制再选择相应的最终覆盖实现。

代码示例 普通成员与虚成员的选择不同

```
#include <iostream>
struct Shape {
    const char* kind() const { return "shape"; }
    virtual void draw() const { std::cout << "Shape::draw\n"; }
    virtual ~Shape() = default;
};
struct Circle : Shape {
    const char* kind() const { return "circle"; }
    void draw() const override { std::cout << "Circle::draw\n"; }
};
int main() {
    Circle c;
    // 引用的静态类型是 Shape，但实际对象始终是 Circle。
    Shape& s = c;
    std::cout << s.kind() << ' ' << c.kind() << '\n';
    // draw 是虚函数，运行时选择 Circle 的覆盖实现。
    s.draw();
    // 显式限定基类版本，本次调用不再做动态分派。
    s.Shape::draw();
}
```


运行追踪 三次观察

```
shape circle
Circle::draw
Shape::draw
```

`kind()` 不是虚函数，通过基类视角调用时选择基类版本。`draw()` 是虚函数，正常调用选择圆形实现。最后的显式基类限定则指定调用 `Shape::draw`。

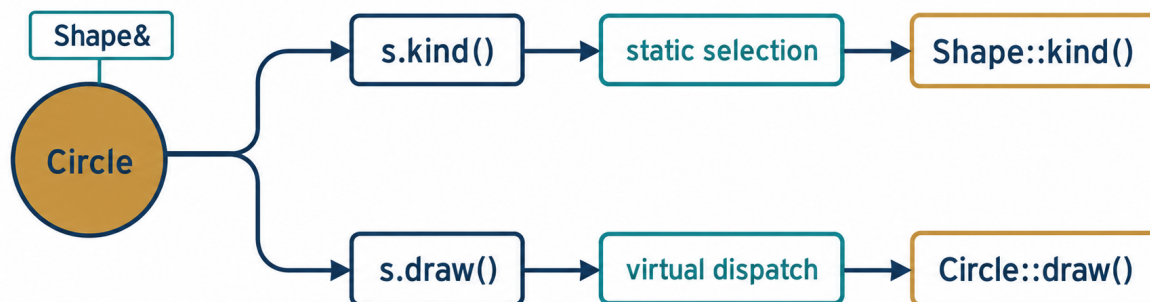


图 7.1 非虚成员的静态选择与虚成员的动态分派。它们可以作用于同一个完整对象，却遵循不同的选择规则。

虚函数不是“任何同名函数”

覆盖关系要求符合相应签名规则。漏写 `const`、参数类型不同或引用限定不同，可能产生隐藏或新的重载，而不是期待的覆盖。给派生实现加上 `override`，可以让编译器检查这一意图。

`final` 可以限制后续覆盖或继承。静态成员函数没有隐式当前对象，不能通过同样机制成为虚函数。

7.2 把查找、访问与分派分开

通过基类表达式调用成员时，先判断基类接口是否提供该操作、调用方是否有权访问，以及参数如何匹配。只有在形成合适的虚调用后，才根据对象选择实现。

代码示例 访问权限与默认参数由静态接口决定

```
#include <iostream>
struct Base {
    // 接口声明决定通过 Base 引用调用时采用的默认参数1。
    virtual void action(int value = 1) const = 0;
    virtual ~Base() = default;
```

```
};
struct Derived : Base {
private:
    // 实际执行体由动态类型选择; private 不阻止通过公开基类接口调用。
    void action(int value = 2) const override {
        std::cout << "Derived " << value << '\n';
    }
};
int main() { Derived d; const Base& b = d; b.action(); }
```

输出为 Derived 1。默认实参来自基类声明，所以传入 1。实际函数体来自派生类，所以输出 Derived。派生覆盖函数即使是 private，也不阻止已经通过基类公共接口形成的这次合法调用。

原理剖析 两层机制分别解决两个问题

静态检查回答“调用者能否发出这种请求”。动态分派回答“这个实际对象用哪个实现响应请求”。把二者混成一个步骤，就容易对访问权限、默认参数和同名成员产生误判。

普通基类成员函数内部调用虚成员，也可以发生动态分派，因为它仍通过当前对象调用。与之不同，显式限定基类名字可以抑制相应动态选择。

7.3 构造与析构期间的虚调用

基类构造时，派生部分尚未完成建立；基类析构时，派生部分已经进入或完成清理。此时不能像一个完整、正常使用中的派生对象那样向更派生实现分派。

代码示例 沿构造和析构阶段追踪调用

```
#include <iostream>
struct Base {
    // 执行基类构造阶段时，虚调用按基类阶段处理。
    Base() { speak(); }
    virtual ~Base() { speak(); }
    virtual void speak() const { std::cout << "Base\n"; }
};
struct Derived : Base {
    // 进入派生类构造函数体以后，才调用派生版本。
    Derived() { speak(); }
    ~Derived() override { speak(); }
    void speak() const override { std::cout << "Derived\n"; }
};
int main() {
    Derived d;
    const Base& b = d;
    // 完整对象已经建立，通过基类引用仍调用 Derived 的实现。
    b.speak();
}
```

运行追踪 输出顺序

```
Base
Derived
Derived
Derived
Base
```

第一行来自基类构造，第二行来自派生构造，第三行来自正常使用。销毁时先执行派生析构中的调用，再执行基类析构中的调用。

不要在基类构造中依赖派生状态

如果一个基类构造函数试图调用派生类实现来完成初始化，就容易违背构造阶段的规则。更清楚的设计是让构造参数提供必要数据，或在对象完整建立后，通过明确操作启动可扩展行为。

构造或析构期间的纯虚调用尤其需要避免。纯虚函数能否有类外定义，与是否允许在某个动态调用语境中调用它，是两个不同问题。

7.4 虚析构与多态拥有

假设 `std::unique_ptr<Shape>` 实际拥有一个 `Circle`。拥有者销毁时，必须完整销毁 `Circle` 的派生成员，再销毁 `Shape` 部分。因此，允许通过基类指针执行 `delete` 的接口，通常需要虚析构函数。

写 `virtual ~Shape()=default;` 表示保留编译器生成的析构逻辑，同时允许经基类销毁时进行正确的动态选择。`unique_ptr` 能自动调用删除操作，但不能替一个错误的基类析构契约自动补上虚分派。

如果对象将通过普通基类指针删除，基类通常需要公共虚析构函数。对常规多态删除情形，缺少所需虚析构会导致未定义行为，不能把结果描述为稳定地“只调用基类析构”。

```
struct Interface {
    // 纯虚接口要求具体类型提供行为，虚析构允许经接口完整销毁。
    virtual void run() = 0;
    virtual ~Interface() = default;
};
```

如果基类根本不允许通过基类指针删除，也可以使用受保护的非虚析构等设计来表达限制。选择应与拥有方式一致。

易错点 智能指针不会替你修正错误的删除接口

把 `Derived*` 放入使用普通基类删除方式的独占指针，并不能自动弥补基类缺少虚析构的问题。拥有类型只会执行已经约定的释放操作，释放操作本身必须合法。

共享指针的控制块可以在某些构造方式下保留实际删除器，但不应因此让多态接口依赖容易被误用的特殊构造路径。清楚的析构契约仍然重要。

7.5 抽象类与可扩展集合

有些接口只能说明“每种形状应能求面积”，却无法为所有形状提供一个通用公式。可以写 `virtual double area() const=0;` 末尾的 `=0` 标记纯虚函数，不是说面积函数返回 0。

Shape 因此不能直接建立对象；具体 Rectangle 与 Square 各自提供公式。main 中先建立具体对象，再把它们放入统一的拥有者列表，最后循环调用 area。按这个使用过程读，抽象类就不再只是一个类图中的空框。

抽象类规定操作接口，并把部分实现留给派生类。能否实例化，取决于是否仍有纯虚的最终覆盖函数。它可以拥有数据、构造函数和已实现的普通成员。

代码示例 用拥有型指针保存不同形状

```
#include <cassert>
#include <memory>
#include <vector>
struct Shape {
    // 纯虚函数规定所有具体形状都要提供求面积的接口。
    virtual double area() const = 0;
    virtual ~Shape() = default;
};
struct Rectangle final : Shape {
    double width, height;
    Rectangle(double w, double h) : width(w), height(h) {}
    double area() const override { return width * height; }
};
struct Square final : Shape {
    double side;
    explicit Square(double s) : side(s) {}
    double area() const override { return side * side; }
};
int main() {
    // 容器拥有各形状，但统一通过 Shape 接口访问。
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Rectangle>(2, 3));
    shapes.push_back(std::make_unique<Square>(4));
    double total = 0;
    // 循环不用区分具体类型；动态分派分别算出6和16。
    for (const auto& s : shapes) total += s->area();
    assert(total == 22);
}
```

列表拥有每个具体形状，遍历时只使用 Shape 接口。增加新的具体形状，可以在不改求总面积循环的情况下加入系统。

开放的扩展点仍有边界

这个设计方便增加形状种类，但如果要给所有形状增加一种全新操作，可能需要修改接口及每个实现。没有一种组织方式对所有变化方向都同样开放。后面的访问者模式会讨论另一种取舍。

接口还应表达行为契约。若一个基类承诺某个操作不会修改对象，派生实现就不能只满足函数签名，却悄悄破坏调用者对行为的期待。

7.6 复制多态对象：显式 clone

一个只收到 `Value&` 的函数，如何复制完整的实际对象？直接写 `Value copy=value` 既可能切片，也可能因为基类抽象而无法编译。可以让接口提供虚函数 `clone`，由具体类型自己建立一份同类型副本，再通过基类拥有指针返回。

将派生对象按值传给基类，会发生切片。若需要复制动态类型，可以让每个具体类型提供自己的克隆实现。

代码示例 复制实际类型并返回拥有者

```
#include <cassert>
#include <memory>
struct Value {
    virtual int get() const = 0;
    virtual std::unique_ptr<Value> clone() const = 0;
    virtual ~Value() = default;
};
struct Number : Value {
    int n;
    explicit Number(int value) : n(value) {}
    int get() const override { return n; }
    std::unique_ptr<Value> clone() const override {
        // 知道实际类型为 Number，因此能复制完整派生对象。
        return std::make_unique<Number>(*this);
    }
};
int main() {
    Number n{7};
    std::unique_ptr<Value> copy = n.clone();
    // 修改原对象后，克隆出来的新对象仍保存7。
    n.n = 9;
    assert(copy->get() == 7);
}
```

这里返回 `unique_ptr<Value>`，明确表示新对象的所有权。`Number` 知道自己的完整类型，因此能建立完整副本。

克隆与赋值不是一件事

克隆创建新对象；给某个现有基类引用赋值，仍可能只修改基类部分。定义一个名叫 `clone` 的虚函数，不会自动改变普通赋值运算符的语义。调用者必须使用这个明确的复制接口。

对于 `unique_ptr` 这样的类模板返回类型，不能直接把智能指针版本的不同类型返回当成语言内置协变返回。派生覆盖声明仍需遵循正确签名，再在函数体中完成允许的拥有指针转换。

7.7 RTTI：查询与检查动态类型

当一个基类接口不能表达某个特定操作时，有时需要检查对象是否属于某个派生类型。`dynamic_cast` 可以在适当多态层次中进行运行时检查。

代码示例 指针失败与引用失败

```
#include <cassert>
#include <typeinfo>
struct Base { virtual ~Base() = default; };
struct Derived : Base {};
struct Other : Base {};
int main() {
    Derived d;
    // 通过基类接口观察实际的 Derived 对象。
    Base& b = d;
    assert(typeid(b) == typeid(Derived));
    assert(typeid(42) == typeid(int));
    assert(dynamic_cast<Derived*>(&b) == &d);
    // 指针形式的检查失败用 nullptr 表示，不产生一个 Other 对象。
    assert(dynamic_cast<Other*>(&b) == nullptr);
    bool rejected = false;
    // 引用形式没有空引用可返回，因此检查失败会抛 bad_cast。
    try { (void)dynamic_cast<Other&>(b); }
    catch (const std::bad_cast&) { rejected = true; }
    assert(rejected);
}
```

失败的相应指针转换返回空指针，失败的相应引用转换抛出 `std::bad_cast`。示例没有把一个真实基类对象强行当成派生对象使用。

typeid 也可以查询非多态类型

`typeid(42)` 是合法的类型查询。对多态类型的适当 glvalue 表达式，`typeid` 才会进一步反映动态类型；其他情形通常依据静态类型等规则处理。因此不能把整个 `typeid` 功能限定为“只有带虚函数的类才能用”。

返回的类型信息名字属于实现相关表示，不适合当成稳定业务标识。需要持久化的类型标签，应由程序定义明确格式。

工程实践 频繁类型判断可能提示接口需要调整

如果每个调用者都在判断 `Circle`、`Rectangle`、`Triangle`，然后执行相似分支，可以考虑把行为放回共同接口。如果只有少数局部场景需要特定能力，受控的动态检查也可以是合理选择。

7.8 虚表是常见解释，不是语言强制布局

许多实现用虚表和对象中的相应指针支持动态分派。这有助于理解间接调用和部分存储开销，但标准不要求所有实现采用同一张表、相同字段位置或固定字节数。

编译器若能证明实际目标，还可能消除间接调用。性能判断应结合具体优化结果与测量，而不是只看到 `virtual` 就断言一次调用必定很慢。

多态的价值，是把变化放到清楚的接口之后。它与所有权、复制方式和生命周期一起设计，才能形成可维护的对象系统。

第 8 章 移动语义与类型转换：什么时候复制，什么时候交出资源

假设一个函数刚读完一个大文件，结果存在 `vector` 里，接下来要交给另一个对象处理。如果原数组还要继续使用，应当复制；如果原数组的内容已经不再需要，重新分配一份存储并复制全部元素，就可能是多余的工作。

移动语义处理的就是这种交接。本章先把复制与移动放进实际程序，再解释 `std::move`、右值引用和构造函数分别做了什么。理解它们的分工，比先背值类别名称更重要。

8.1 先用起来：保留一份副本，或把内容交给新对象

代码示例 复制与移动放在同一个程序里比较

```
#include <iostream>
#include <utility>
#include <vector>
int main() {
    std::vector<int> source{10, 20, 30};
    std::vector<int> backup = source;           // 复制：两边各有自己的元素
    backup[0] = 99;                             // 修改副本，不影响 source
    std::cout << source[0] << ' ' << backup[0] << '\n';

    std::vector<int> worker = std::move(source); // 选择 vector 的移动构造，把资源交给 worker
    std::cout << worker.size() << ' ' << worker[0] << '\n';

    // source 对象仍然存在；先清空再加入新值，不依赖移动后的旧内容
    source.clear();
    source.push_back(7);
    std::cout << source.size() << ' ' << source[0] << '\n';
}
```

预期输出为：

```
10 99
3 10
1 7
```

第一行输出来自复制。执行 `backup = source` 的初始化后，`backup` 与 `source` 各自拥有一组元素；把副本的第一个元素改成 99，原数组仍是 10。

接着，`worker = std::move(source)` 在建立一个新 `vector`。这里会选择 `vector` 的移动构造，让 `worker` 接管原来的资源，而不是要求保留两份独立内容。新对象因此有三个元素，第一个仍为 10。

最后两行操作原来的 `source`。它没有消失，仍然是一个 `vector` 对象。程序先调用 `clear()` 建立明确的空状态，再加入 7，所以最后输出 1 7。移动完成后，我们不再依赖它仍保存原来的三个数。

原理剖析 先决定程序还需不需要原内容

需要同时保留两份独立的值时用复制；原内容可以交给接收方时，才考虑移动。不要为了“可能更快”就在任意变量外面加 `std::move`，然后继续假定原内容没有变化。

移动的是资源责任，不是把变量名搬到另一行

`source` 和 `worker` 始终是两个不同的对象。对拥有动态存储的类型，可以改变的是“哪一个对象负责这块存储”。移动成功后，目标负责接管的资源，源对象留下一个允许销毁或重新赋值的状态。

源状态由具体类型规定。我们自己的 `Buffer` 会明确置空；许多标准库类型只承诺有效但值未指定。理解这个限制以后，使用方法很直接：不要读取旧内容来完成后续业务，先重新赋值或按该类型允许的方式建立新内容。

8.2 `std::move` 本身做什么：观察一次普通函数调用

仅看刚才的初始化，很容易误以为 `std::move(source)` 已经亲自把数组转交了。下面换一个接收方：它只打印信息，不建立新对象，也不修改数组。

代码示例 相同对象，可以选择不同的重载

```
#include <iostream>
#include <utility>
#include <vector>
void inspect(const std::vector<int>& values) {
    std::cout << "read-only reference\n";
}
void inspect(std::vector<int>&& values) {
    // 这个函数没有移动构造别的 vector，也没有修改 values
    std::cout << "rvalue reference\n";
}
int main() {
    std::vector<int> x{10, 20, 30};
    inspect(x);                // x 是左值表达式，选择第一个重载
    inspect(std::move(x));      // 改变参数的值类别，选择第二个重载
    std::cout << x.size() << ' ' << x[0] << '\n'; // 两个函数都没有修改 x
}
```

预期输出：

read-only reference

rvalue reference

3 10

两次调用传入的都是 `x` 这个对象。不同之处是，第二次用 `std::move(x)` 产生了一个右值形式的表达式，于是编译器选择参数为 `vector<int>&&` 的重载。这个函数没有改动数组，调用后 `x` 仍有三个元素，首元素仍为 10。

现在可以把上一节拆成两个动作：

1. `std::move(source)` 改变编译器看待这个表达式的方式，使它可以匹配接收右值引用的接口。
2. 新建 `worker` 时，`vector` 的移动构造函数被选中；这个构造函数才执行资源交接。

`std::move` 决定“怎样把这个对象交给接口”，接收方决定“实际对它做什么”。它不会自行释放原对象，不会调用一个隐藏的通用“搬运函数”，也不保证最终一定发生移动。

8.3 右值引用的语法：T&& 不代表自动搬运

已经熟悉的 `T&` 是左值引用，通常绑定到一个可指认的已有对象。这里的 `T&&` 是右值引用，可以绑定相应右值表达式：

```
std::vector<int> x{1, 2, 3};
std::vector<int>& ordinary = x;           // ordinary 是 x 的别名
std::vector<int>&& ready = std::move(x); // ready 仍是 x 的别名
```

这两行引用声明都没有新建 `vector`，也没有建立一份新的元素数组。`ready` 只把同一个对象绑定到右值引用。引用是否建立、接收方是否随后移动构造新对象，是两件事。

构造函数可以据此提供不同入口：

```
Buffer(const Buffer& other); // 从只读来源建立独立副本
Buffer(Buffer&& other);     // 可改变来源的资源状态，完成交接
```

这与普通函数重载相同：参数类型参与编译期选择。但两个入口的函数体由类的作者编写，语言不会因为写了 `&&` 就自动生成正确的资源处理逻辑。

有了名字的右值引用，再拿来用时是左值

表达式 `ready` 有名字，表示那个已有对象，所以

```
Buffer source{1, 2, 3};
Buffer&& ready = std::move(source); // 只建立引用
Buffer a(ready);                   // 按已有左值使用：选择复制
Buffer b(std::move(ready));         // 再显式给出右值形式：可选择移动
```

第一行并不意味着以后每次写 `ready` 都会自动消费它。如果真要交出资源，调用处仍需要表达这个意图。

const 会影响能否调用移动入口

移动通常需要修改源对象的资源状态。若源对象为 `const`，`std::move` 不会替你去掉 `const`。常见的 `T(T&&)` 无法接受 `const T&&`；若复制入口可用，就可能改为复制。

下面用两个计数器观察构造选择。它不是用来证明资源移动速度，而是把编译器选择的入口显示出来。

代码示例 按语句核对复制与移动次数

```
#include <cassert>
#include <utility>
struct Item {
    inline static int copies = 0, moves = 0;
    Item() = default;
    Item(const Item&) { ++copies; }
    Item(Item&&) noexcept { ++moves; }
};
int main() {
    Item x;
    // 这里只建立别名，没有调用任何 Item 构造函数。
    Item&& view = std::move(x);
    assert(Item::moves == 0);
    // 具名的 view 是左值表达式，选择复制入口。
    Item a(view);
    // 显式转成右值形式，选择可用的移动入口。
    Item b(std::move(view));
    const Item c;
    // const 仍然保留，不能绑定到 Item&&，这里退回复制入口。
    Item d(std::move(c));
    assert(Item::copies == 2 && Item::moves == 1);
}
```

建立 view 后，两种构造次数都为零。a(view) 使复制次数变为 1；b(std::move(view)) 使移动次数变为 1；最后 const 源使 d 选择复制，所以总计复制两次、移动一次。若某种类型既没有合适的移动入口，也不能复制，相应表达式会编译失败。

8.4 亲手实现资源交接：Buffer 的两个移动操作

Buffer 用 data_ 保存动态数组地址，用 size_ 保存长度。只要每块数组始终有且只有一个拥有者，析构时执行 delete[] 就不会重复释放。完整类定义见附录；这里把资源交接的步骤直接写出来。

代码示例 移动构造：新对象接管，源对象置空

```
Buffer(Buffer&& other) noexcept {
    size_ = other.size_;    // 接管原长度
    data_ = other.data_;    // 接管原数组地址，没有复制数组元素
    other.size_ = 0;        // 来源仍存在，但不再拥有那块数组
    other.data_ = nullptr;  // 来源析构时 delete[] nullptr 是安全的
}
```

移动构造时，目标是一个正在建立的新对象，还没有需要先释放的旧数组。执行后，两边的状态可以直接列成表：

时刻	来源 other	新目标
交接前	长度为 3，拥有数组地址 P	尚未拥有这块数组
交接后	长度为 0，地址为空	长度为 3，拥有原地址 P
析构时	不释放 P	负责释放 P

表 8.1 置空源指针是避免两个对象重复释放同一数组的关键步骤。

移动赋值的目标已经存在，可能拥有另一块数组。因此要先处理目标原来的资源：

代码示例 移动赋值：先释放旧资源，再接管新资源

```
Buffer& operator=(Buffer&& other) noexcept {  
    if (this != &other) {          // 自移动时保留当前状态  
        delete[] data_;           // 目标已有的旧数组先归还  
        size_ = other.size_;  
        data_ = other.data_;      // 接管来源的数组  
        other.size_ = 0;  
        other.data_ = nullptr;  
    }  
    return *this;                  // 返回目标自身，支持赋值表达式  
}
```

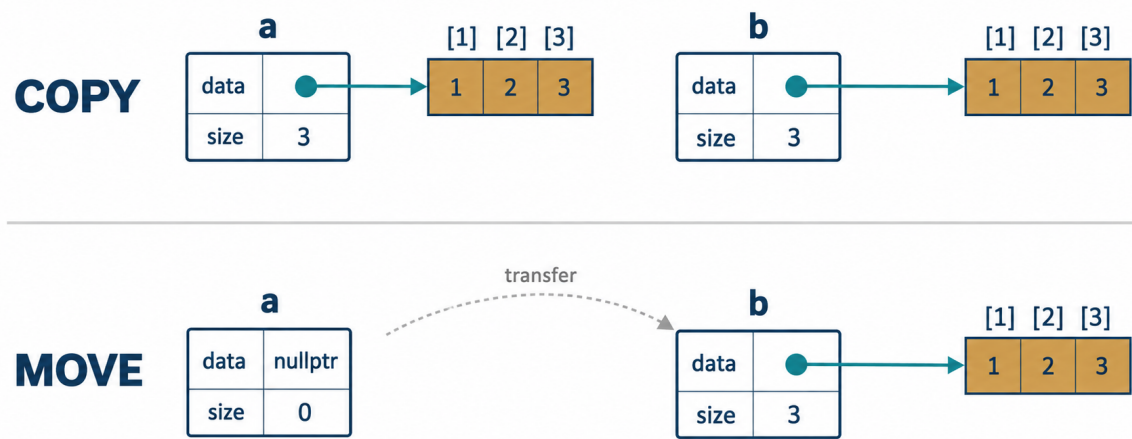


图 8.1 复制让两个 Buffer 各自拥有数组；移动让目标接管原数组。这里的源对象置空，是本类明确定义的行为。

代码示例 通过地址和内容分别检查交接

```
#include "buffer.hpp"
#include <cassert>
#include <utility>
int main() {
    Buffer source{1, 2, 3};
    // 保存地址用于比较，original 只是观察者，不拥有数组。
    int* original = source.data();
    // 移动构造接管数组地址；没有逐个复制三个整数。
    Buffer target = std::move(source);
    assert(target.data() == original && target.size() == 3);
    assert(source.data() == nullptr && source.size() == 0);
    // 给仍然存在的源对象写入新内容，不依赖它移动后的旧值。
    source = Buffer{9};
    assert(source[0] == 9);
    Buffer destination{5, 6};
    // 移动赋值还要先释放 destination 原来保存5、6的数组。
    destination = std::move(target);
    assert(destination.data() == original && target.size() == 0);
}
```

`original` 只是保存地址，不能负责释放。移动后目标的地址与它相同，说明目标接管了原存储。后面给源对象重新赋值，又说明移动没有结束源对象的生命周期。

`std::exchange(x,new_value)` 可以把“取出旧值并写入新值”合成一个标准库调用。完整类中若写 `data_=std::exchange(other.data_,nullptr)`，作用就是先取得旧地址，再把来源地址置空。先理解上面的分步形式，再读这种简写会容易得多。

8.5 值类别：给已经看到的调用规则命名

到这里，我们已经多次区分“直接写对象名字”和“用右值形式传入”。值类别就是编译器描述这些表达式性质的一套分类。它与数据类型是不同信息：两种表达式都可能涉及 `vector`，却选择不同的重载。

类别	本章的例子	阅读代码时的重点
左值 lvalue	<code>x</code> 、具名引用 <code>ready</code>	表示一个已有对象
纯右值 prvalue	<code>3+4</code> 、 <code>Buffer{1,2}</code>	计算一个值或初始化结果对象
亡值 xvalue	<code>std::move(x)</code>	仍指向已有对象，但允许匹配资源复用接口

表 8.2 纯右值与亡值合称右值。值类别不等于是否可修改。

一个 `const` 对象的名字仍是左值，却不能通过该路径修改。因此，“能放在等号左边”不是左值的完整定义。`decltype` 可以用于检查类别，但它是验证工具，不必用模板判断式代替对程序过程的理解。

代码示例 用编译期检查验证前面的结论

```

#include <type_traits>
#include <utility>
int main() {
    int x = 1;
    const int c = 2;
    // 声明类型是右值引用，但之后单独写名字 r 仍是左值表达式。
    int&& r = 3;
    static_assert(std::is_same_v<decltype((x)), int&>);
    static_assert(std::is_same_v<decltype((c)), const int&>);
    // 双括号按表达式检查，所以得到 int&。
    static_assert(std::is_same_v<decltype((r)), int&>);
    // move 改变表达式类别，不改变被指向对象原本的 int 类型。
    static_assert(std::is_same_v<decltype(std::move(x)), int&&>);
    static_assert(std::is_same_v<decltype(1+2), int>);
}

```

8.6 noexcept 与异常保证

vector 扩容时，需要把元素从旧存储转移到新存储。如果已经移走几项，第五项却抛出异常，旧元素可能已经改变，回滚就变得困难。若元素的移动明确承诺不抛异常，容器便能在相应情形下更容易保证迁移过程的安全。先有这个实际问题，再看 noexcept 的意义。

noexcept 表达函数不会把异常传播到调用者的承诺。若违反承诺，通常会触发终止，而不是让调用者照常捕获这个异常。因此它不是可以随意添加的性能标记。

对只交换指针和长度的 Buffer 移动操作，写成 noexcept 很自然。容器在重新分配元素时，也会利用类型的移动异常性质，决定如何满足自身的异常保证。

原理剖析 移动是否安全，影响外层容器的选择

如果移动可能失败，而且失败已经改变了源元素，外层容器就难以恢复原状态。可复制类型有时可以通过复制来保留旧元素。理解这种选择，应从异常保证出发，而不是只记“容器喜欢移动”。

类型特征如 `std::is_nothrow_move_constructible_v<T>` 可以查询相关性质。实际库算法还会结合可复制性、分配器与操作类别做决定。

8.7 复制消除：有时根本不需要复制或移动

在 C++17 及以后，一些相应纯右值初始化情形具有保证的直接构造语义。它们不只是“优化器可能帮忙省掉一步”。

代码示例 不可复制、不可移动的类型仍可这样返回

```
#include <cassert>
struct Token {
    int value;
    explicit Token(int v) : value(v) {}
    // 禁止复制和移动，便于确认返回表达式不需要这两个构造函数。
    Token(const Token&) = delete;
    Token(Token&&) = delete;
};
// C++17 以后，这种返回纯右值的情形直接建立结果对象。
Token make_token() { return Token{7}; }
int main() { Token t = make_token(); assert(t.value == 7); }
```

返回 `Token{7}` 可以直接建立最终结果对象。这与返回一个具名局部变量的 NRVO 情形需要区分，后者是否适用以及是否被实施，有自己的条件。

不要习惯性地对返回局部变量写 `move`

`return local;` 已经有适当的返回规则。无条件改成 `return std::move(local);` 可能阻止原本可用的 NRVO，还让代码更难读。应先理解返回语境，再决定是否需要显式转换。

构造日志只能说明某次合法执行中发生了什么。涉及未保证的复制消除时，应写出允许的情况，而不是把一个编译器的日志当成唯一语言结果。

8.8 四类显式转换分别表达什么

显式转换通常写成 `转换名 < 目标类型 >(表达式)`。例如把 `3.75` 转成整数，可以写 `static_cast<int>(3.75)`，结果为 `3`。目标类型说明要得到什么，转换名说明允许使用哪一类规则。不同问题应选不同入口，不能只要编译报错就随手换成最强的强制转换。

转换	主要用途与边界
<code>static_cast</code>	允许的数值、层次或其他静态转换；不自动验证所有运行时前提
<code>dynamic_cast</code>	在适当多态层次中检查类型关系；失败行为取决于指针或引用形式
<code>const_cast</code>	调整特定 <code>const/volatile</code> 限定；不改变对象原本是否可写
<code>reinterpret_cast</code>	低层类型重新解释；访问仍受对齐、生命周期和类型访问规则约束

表 8.3 显式转换让意图可见，但不能把本来无效的对象访问自动变合法。

代码示例 数值转换、访问路径与位表示

```
#include <cassert>
#include <bit>
#include <cstdint>
#include <type_traits>
int main() {
```

```
double d = 3.75;
// 明确做数值转换，丢弃可表示浮点数的小数部分。
int n = static_cast<int>(d);
assert(n == 3);
int mutable_value = 7;
const int* read_only_path = &mutable_value;
// 原整数本来可写，只是访问路径只读；原对象若为 const 则不能照做。
*const_cast<int*>(read_only_path) = 9;
assert(mutable_value == 9);
static_assert(sizeof(float) == sizeof(std::uint32_t));
// 复制对象表示为等大的无符号整数，不是把浮点数值转换成1。
auto bits = std::bit_cast<std::uint32_t>(1.0f);
auto original = std::bit_cast<float>(bits);
assert(original == 1.0f);
}
```

3.75 在可表示范围内转换成整数 3。去掉 `read_only_path` 的 `const` 限定后写入是因为原对象本来就是可修改整数；若原对象定义为 `const`，结论就不同。

`std::bit_cast` 在满足大小和类型条件时转换对象表示，避免用错误的指针别名方式读取另一类型。示例明确检查了所用平台上两个类型大小相同，并只验证往返恢复，不假定一个普遍的浮点字节常数。

8.9 让类型变化服务清楚的设计

频繁强制转换往往值得停下来检查：接口是否接受了不合适的类型？所有权是否被隐藏？基类是否缺少真正需要的操作？消除错误的抽象边界，通常比继续增加转换更可靠。

移动、复制与转发并不是三个互相竞争的技巧。它们分别表达独立取值、允许复用和保留调用方信息。把这些意图分清，模板和标准库代码就更容易理解。

第 9 章 模板与编译期抽象：让算法依赖能力而非固定类型

一个求最大值的算法不必只处理整数，一个固定长度数组也不必为每种元素类型重写。模板把类型或编译期值作为参数，让一份定义描述一族具体实体。

这里的关键不是“把类型名字替换一下”这么简单。推导、名字查找、重载选择和实例化都在编译期发生。理解这些阶段，才能看懂泛型代码的错误信息，也能设计真正适用的抽象。

9.1 函数模板与实参推导

如果先写一个接收 `int` 的 `larger`，再写一个接收 `double` 的 `larger`，两份函数体几乎一样。函数模板把这里变化的“类型”变成参数：`template<class T>` 声明类型参数 `T`，后面的函数用 `T` 代替固定类型名。

先对比两种调用：

```
larger_value(3, 5);           // 根据实参推导 T=int
larger_value<double>(3, 5.5); // 直接指定 T=double
```

模板不是一个运行时把类型当普通变量传入的函数，而是一份用于形成具体函数的定义。第一种调用使用推导，第二种调用通过尖括号明确给出类型参数。

模板参数在使用时可以显式给出，也可以从函数实参中推导。推导关注参数模式与实参类型之间的匹配，不会任意猜一个“大家都能转过去的类型”。

代码示例 同类型参数与独立类型参数

```
#include <cassert>
#include <type_traits>
// T 是类型参数；同一次实例化中，两个实参必须按规则匹配同一个 T。
template<class T> T larger_value(T a, T b) { return a < b ? b : a; }
template<class T, class U> auto add(T a, U b) { return a + b; }
int main() {
    assert(larger_value(3, 5) == 5);
    // 显式指定 T=double，两个数按 double 参数接收。
    assert(larger_value<double>(3, 5.5) == 5.5);
    // 两个独立类型参数允许 int 与 double 组合，返回类型由 a+b 推导。
    static_assert(std::is_same_v<decltype(add(1, 2.5)), double>);
}
```

`larger_value(3,5)` 能推导 `T` 为 `int`。若直接传入整数和 `double`，同一个 `T` 会得到冲突信息；显式指定 `larger_value<double>` 则先确定 `T`，再按相应参数类型处理转换。

`add` 使用两个独立模板参数，并根据表达式推导返回值。它不再把返回值强行限制成第一个实参的类型。但这也意味着必须分析加法表达式本身的类型与语义。

值参数、引用参数与数组

对 `f(T)`，常见推导会丢掉顶层 `const` 和引用，并处理数组或函数到指针的调整。对 `f(T&)`，则从引用绑定的角度推导，能够保留更多信息。对转发引用，还要结合前一章的引用折叠规则。

原理剖析 推导保留哪些信息，由参数模式决定

模板函数的参数不是一个空白容器。写成 `T`、`T&`、`const T&` 或 `T&&`，已经表达了不同的接口要求。先确定需要独立值、借用还是转发，再选择模式。

9.2 转发引用与完美转发

现在已经知道模板参数怎样由实参推导，再看包装函数。`relay` 不想决定调用者的对象是否被消费，只想把参数原样转交给 `select`。若不做处理，参数一旦有了名字，写出这个名字就是左值；若一律使用 `move`，又会把原本的左值也当成可消费对象。

`std::forward<T>(value)` 根据推导得到的 `T` 选择类别。对下面的调用：`relay(x)` 推导 `T` 为 `int&`，转发后仍是左值；`relay(3)` 推导 `T` 为 `int`，转发后保留右值类别。先看这两次具体选择，再理解“完美转发”这个名称。

包装函数收到一个参数后，若直接用参数名字调用下一层，名字是左值表达式，原来的右值性质可能丢失。`std::forward` 根据推导得到的类型，有选择地保留调用方的值类别。

代码示例 把调用方的类别传给下一层

```
#include <cassert>
#include <utility>
int select(const int&) { return 1; }
int select(int&&) { return 2; }
template<class T>
// 按调用方推导的 T 还原类别，不把所有调用都强制当成右值。
int relay(T&& value) { return select(std::forward<T>(value)); }
int main() {
    int x = 3;
    // 传入具名变量，下一层收到左值，因此选择返回1的版本。
    assert(relay(x) == 1);
    // 传入临时值，下一层保留右值类别，因此选择返回2的版本。
    assert(relay(3) == 2);
}
```

传入左值 `x` 时，模板参数推导与引用折叠使下一层看到左值；传入临时整数时，下一层可以选择右值版本。

不是所有 `T&&` 都是转发引用

这里的 `T` 需要处于相应类型推导语境，且满足转发引用的条件。一个类模板中已经确定的 `T`，或者 `const T&&`，不能不加分析地套用同一规则。

常见引用折叠可以记成：只要其中一层是左值引用，结果通常折叠为左值引用；两个右值引用折叠为右值引用。这个规则解释了转发模板为何能够保存调用方的信息。

易错点 一个参数不应被无意消费多次

完美转发不是可以反复对同一个对象无代价使用的操作。如果第一次调用已经移动走资源，第二次转发可能处理的是改变后的源状态。包装器应明确调用次数和所有权行为。

9.3 模板与普通函数如何竞争

普通函数和函数模板可以同时参与重载。不能简单认为“先找普通函数，找不到才找模板”。转换匹配质量依然重要。

代码示例 精确匹配与同等级选择

```
#include <iostream>
void pick(int) { std::cout << "ordinary int\n"; }
template<class T> void pick(T) { std::cout << "template\n"; }
// 依次选择普通 int、模板 double，以及显式要求的模板 int 版本。
int main() { pick(1); pick(1.5); pick<>(1); }
```

运行追踪 输出

```
ordinary int
template
template
```

整数调用中，普通函数在相应同等级比较中胜出。double 调用中，模板提供精确类型匹配，比转成 int 更好。显式写 `pick<>` 则把意图指向模板。

推导失败与实例化错误

某些替换失败可以让候选退出重载集合，但并不是模板中的所有错误都能被悄悄忽略。错误发生在什么语境、是否进入函数体实例化，会影响它是候选不可用还是整个程序需要诊断。

清楚的约束可以让错误更早、位置更明确。C++20 Concepts 正是把一部分能力要求写到接口上的机制，后面会单独讨论。

9.4 类模板、非类型参数与 CTAD

函数模板形成具体函数，类模板形成具体类型。`Fixed<int,3>` 指定元素类型为 int、长度为 3；`Fixed<double,3>` 则是另一种类型。尖括号里的 3 不是给构造函数的运行时参数，而是类型的一部分。

`Box b{2.5}` 使用类模板实参推导(CTAD)：编译器根据构造方式推导 b 应属于 `Box<double>`。这里是省略一部分能推导出来的类型写法，类模板仍然需要确定的具体参数才能建立对象。

类模板描述一族类型。`Fixed<int,3>` 与 `Fixed<int,4>` 是不同类型，长度不仅是运行时字段，也进入了类型身份。

代码示例 让元素类型与长度都成为参数

```

#include <array>
#include <cassert>
#include <cstddef>
#include <type_traits>
// T 控制元素类型, N 控制编译期长度, 两者共同决定具体类型。
template<class T, std::size_t N>
class Fixed {
    std::array<T, N> values_{};
public:
    void fill(const T& x) { values_.fill(x); }
    const T& at(std::size_t i) const { return values_.at(i); }
    static constexpr std::size_t size() { return N; }
};
template<class T> struct Box { T value; explicit Box(T x) : value(x) {} };
int main() {
    // 把模板参数填成 int 和3, 才建立一个具体数组对象。
    Fixed<int, 3> a;
    a.fill(7);
    assert(a.at(2) == 7);
    static_assert(!std::is_same_v<Fixed<int, 3>, Fixed<int, 4>>);
    // 根据构造实参2.5推导模板参数, 得到 Box<double>。
    Box b{2.5};
    static_assert(std::is_same_v<decltype(b), Box<double>>);
}

```

非类型模板参数需要满足相应编译期实参规则。一个普通运行时读取的整数, 不能仅因为“之后不再修改”就充当这里的长度参数。

类模板实参推导

`Box b{2.5};` 可以从构造入口推导得到 `Box<double>`。这个过程称为类模板实参推导, C++17 引入了相关机制。复杂类型还可以提供推导指引。

推导结果需要符合设计意图。例如一个传入字符串字面量的构造参数, 可能推导出指针类型, 而不是自动拥有的字符串类型。是否保存副本、指针还是引用, 仍需要类设计决定。

不同特化的静态成员

不同类模板特化的静态成员应分别理解。`Tag<int>::value` 与 `Tag<double>::value` 不是一个整数。模板统一了定义方式, 并没有把所有具体类型的状态合并。

9.5 特化与类型特征

假设模板想根据“是不是指针”选择实现, 可以写一个在编译期给出 `true` 或 `false` 的小类型。`IsPointer<int>` 使用默认模板, 结果为 `false`; `IsPointer<int*>` 能匹配 `T*` 这个形式, 于是使用专门的偏特化, 结果为 `true`。

`std::false_type` 与 `std::true_type` 提供名为 `value` 的编译期布尔常量。继承它们，让两个分支用统一的 `IsPointer<T>::value` 方式读取结果。先把这种“输入一个类型，得到一个编译期答案”的使用方法掌握，再看更多标准类型特征。

主模板提供一般情况，特化为特定类型或模式提供不同实现。类模板可以进行全特化或偏特化；函数模板不能按相同方式进行偏特化，常用重载或其他分派方式表达需求。

代码示例 一个简单的指针类型特征

```
#include <type_traits>
// 默认情况 value 为 false; 下面的偏特化单独匹配指针形式。
template<class T> struct IsPointer : std::false_type {};
// 若类型能写成某个 T*, 匹配这一版, value 为 true。
template<class T> struct IsPointer<T*> : std::true_type {};
template<class T> struct Tag { inline static int value = 0; };
int main() {
    static_assert(IsPointer<int*>::value);
    static_assert(!IsPointer<int>::value);
    static_assert(IsPointer<std::remove_cv_t<int* const>>::value);
    // Tag<int> 和 Tag<double> 是不同特化, 各自拥有静态成员。
    Tag<int>::value = 3;
    return Tag<double>::value;
}
```

模式 `T*` 匹配相应指针类型。若还要处理顶层 `const` 等情况，应明确进行类型变换或增加相应定义。工程中通常优先使用标准库已提供并处理了边界情况的类型特征。

核心概念 类型也可以成为计算对象

`std::is_same_v` 返回类型关系，`std::remove_cv_t` 生成调整后的类型，`std::conditional_t` 根据编译期条件选择类型。它们让泛型实现可以根据类型性质做清楚的决定。

不要让特化成为隐藏的例外集合

如果每加入一种类型就必须增加一个完全不同的特化，原来的抽象可能并没有抓住共同能力。特化适合少数真正不同的表示或优化路径，而不是代替一份清楚的接口要求。

9.6 变参模板与折叠表达式

想让 `sum` 接收任意个参数，可以用 `class... Ts` 声明类型参数包，用 `Ts... values` 声明对应的函数参数包。`sum(1,2,3)` 中，参数包包包含三个实参；`sum()` 中则为空。

折叠表达式用一个运算符连接参数包。把下面的例子实际展开：`(0+...+values)` 相当于 `((0+v1)+v2)+v3`；`(...-values)` 与 `(values-...)` 则把括号放在不同方向，因此减法结果不同。

参数包表示一组数量可变的模板参数或函数参数。包展开将某种模式应用到每个参数；折叠表达式则把一系列二元运算组织成一个表达式。

代码示例 空包、左折叠与右折叠

```
#include <cassert>
// 初始0使空参数包也有结果；有参数时按括号方向逐项展开。
template<class... Ts> auto sum(Ts... values) { return (0 + ... + values); }
template<class... Ts> auto left_subtract(Ts... values) { return (... - values); }
template<class... Ts> auto right_subtract(Ts... values) { return (values - ...); }
int main() {
    assert(sum() == 0 && sum(1, 2, 3) == 6);
    // 左折叠相当于 (30-5)-2，结果23。
    assert(left_subtract(30, 5, 2) == 23);
    // 右折叠相当于 30-(5-2)，结果27。
    assert(right_subtract(30, 5, 2) == 27);
}
```

左折叠减法相当于 $(30-5)-2$ ，得到 23；右折叠相当于 $30-(5-2)$ ，得到 27。结合方向会影响非结合运算的结果。

初始值决定空包行为，也参与类型推导

$(0 + \dots + \text{values})$ 为数字求和提供了空包时的零。但若目标是字符串连接，初始值就应该与字符串运算相容。泛型代码不能把“零”当成所有类型共享的默认单位元。

`sizeof...(Ts)` 查询参数包大小。包展开也可用于初始化列表、基类列表或函数实参等语境，但不同位置具有不同规则，不能把省略号任意放置。

原理剖析 写出代数要求，才知道模板能泛化多远

求和依赖单位元和二元操作；排序依赖比较关系；去重依赖等价关系。语法上能写出这些操作，只解决了一部分问题。它们是否满足算法需要的规律，还要由类型和调用者保证。

9.7 if constexpr 与按需实例化

有时同一模板针对不同类型需要不同代码。整数的“两倍”可以用乘法，字符串的“两倍”可以用拼接。普通 if 的条件即使总为真，通常也不能让另一支中的类型错误消失；if constexpr 按编译期条件选择相应实例化分支。

因此，twice(3) 需要整数乘法，twice(string) 需要字符串相加。把这两个调用分别代入下面的函数，比只记“编译期 if”更容易理解它解决了什么问题。

代码示例 根据类型选择实现路径

```
#include <cassert>
#include <string>
#include <type_traits>
template<class T> auto twice(const T& x) {
    // 条件在实例化时判断；算术类型使用乘法，其他类型使用相加。
    if constexpr (std::is_arithmetic_v<T>) return x * 2;
    else return x + x;
}
```

```

}
int main() {
    assert(twice(3) == 6);
    // 字符串不需要支持乘法，只实例化被选中的 x+x 分支。
    assert(twice(std::string{"a"}) == "aa");
}

```

算术类型使用乘法，字符串类型使用加法。对某个具体实例，不被选择的依赖分支可以不按另一类型的要求实例化。

但 `if constexpr` 不是预处理器。代码仍需被解析，一些不依赖模板参数的错误也不能靠一个永远为假的分支隐藏起来。它解决的是编译期分支选择，不是任意非法文本的屏蔽。

成员并不总在同一时刻全部生成

类模板的使用可能要求类定义完整，却不一定立即实例化每个普通成员函数体。成员函数、静态成员和显式实例化各有触发规则。读诊断时，要找出到底是哪一次使用要求了某个定义。

9.8 依赖名字与定义可见性

`std::vector<int>::value_type` 明确表示 `int`；但模板中的 `Container` 还没确定，编译器初读 `Container::value_type` 时不能无条件假定它是类型。前面的 `typename` 正是在说明：这里需要把这个依赖名字按类型解析。

示例函数只适用于非空容器，因为它会解引用 `begin`。名字查找语法检查通过，与某次调用满足运行前提，是两项不同条件。

编译器第一次读模板时，还不知道某些名字最终代表类型还是值。`typename` 可以在适当位置明确说明依赖名字代表类型。

代码示例 依赖类型与依赖基类成员

```

#include <cassert>
#include <vector>
template<class Container>
// 告诉编译器 value_type 是类型；传入 vector<int> 时它就是 int。
typename Container::value_type first_value(const Container& c) {
    return *c.begin();
}
template<class T> struct Base { int value() const { return 7; } };
template<class T> struct Derived : Base<T> {
    // 通过当前实例查找依赖基类中的成员。
    int read() const { return this->value(); }
};
int main() {
    assert(first_value(std::vector<int>{3, 4}) == 3);
    assert(Derived<int>{}.read() == 7);
}

```

`Container::value_type` 依赖模板参数。`this->value()` 则明确通过依赖于当前实例的基类关系查找成员。这样的语法提示有助于编译器在正确阶段完成判断。

模板定义为什么常放在头文件

隐式实例化通常需要看到模板定义。若只有声明在头文件，实现被藏在某个普通源文件里，另一个翻译单元可能无法生成需要的实例。也可以采用显式实例化等组织方式，但要明确提供支持的类型集合。

多翻译单元中的同一定义规则仍然适用。模板并没有取消 ODR。关于头文件、链接和构建的具体做法，会在多文件工程中接着讨论。

9.9 泛型抽象的收益与代价

模板让算法依赖能力，而不是依赖某个固定继承体系。它可以保留具体类型，便于静态检查和优化，也可能增加编译时间、诊断复杂度与代码体积。

运行时多态适合在运行中通过统一接口处理不同对象；模板适合在编译时为不同类型生成相应实现。真实软件可以同时使用二者，不必把它们视为只能二选一的编程风格。

第 10 章 容器与迭代器：数据结构背后的使用契约

标准容器让我们不用反复实现动态数组、链表和映射。但选择容器并不是挑一个能编译的名字。存储方式会影响访问成本，修改操作会影响引用是否有效，元素类型还会影响复制、移动和异常行为。

本章围绕三个问题展开：数据怎样组织，算法怎样访问，以及修改后哪些访问途径仍然有效。

10.1 容器、迭代器与算法的分工

先看怎样真正保存和访问一组数：

```
std::vector<int> scores{70, 80}; // 建立两个元素
scores.push_back(90);           // 在尾部增加一个元素
int first = scores[0];           // 读取第一个元素，得到70
for (int score : scores) {       // 按顺序遍历70、80、90
    std::cout << score << ' ';
}
```

vector 负责保存元素。若把遍历位置单独表示出来，就得到 begin/end 这样的迭代器接口；算法再接收这些位置，实现查找或排序。这个分工让算法不必知道元素背后是动态数组还是其他结构。

容器负责存储和生命周期；迭代器描述访问位置与移动方式；算法根据迭代器和可调用对象完成操作。三者分开，才能让同一个算法作用于不同存储结构。

这种抽象并不意味着所有容器支持相同操作。链表没有随机下标，优先队列不提供任意遍历接口，关联容器的键也不能像普通数组元素一样随意修改。

核心概念 算法依赖能力，也依赖语义

一个类型能写出 ++it 和 *it，还不足以说明它满足某个完整迭代器契约。多趟遍历、相等比较、有效范围和复杂度等条件，同样影响算法是否适用。

10.2 vector：大小、容量与重新分配

一个空 vector 的 size 是 0，表示没有可访问的元素。reserve(8) 只要求准备容纳至少 8 个元素的空间；它不会让 v[0] 突然变合法。resize(3) 才会把元素数量改成 3。

可以把 size 看成“已经建立的元素数”，capacity 看成“现有存储在重新分配前能容纳的数量”。下面故意把容量填满，再追加一次，观察重新分配的边界。

size() 表示当前元素数量，capacity() 表示在需要重新分配之前可容纳的相应数量。reserve 调整存储准备，不建立同样数量的元素；resize 才改变元素数量。

代码示例 容量不是可直接写入的元素范围

```
#include <cassert>
#include <vector>
int main() {
    std::vector<int> v;
    // 只准备存储容量，不建立8个元素，当前 size 仍为0。
    v.reserve(8);
    assert(v.empty() && v.capacity() >= 8);
    // 建立3个整数元素，新增元素取值0。
    v.resize(3);
    assert(v.size() == 3 && v[2] == 0);
    const auto old_capacity = v.capacity();
    while (v.size() < old_capacity) v.push_back(1);
    // 此前已经填满容量，这次追加必须重新分配存储。
    v.push_back(2);
    assert(v.capacity() > old_capacity);
    // Use fresh iterators/references after reallocation.
    assert(v.back() == 2);
}
```

reserve(8) 后容器仍为空。不能因为容量足够，就对不存在的 v[0] 赋值。例子先用 resize 建立元素，再把当前容量填满，最后一次追加必然需要更大的容量。

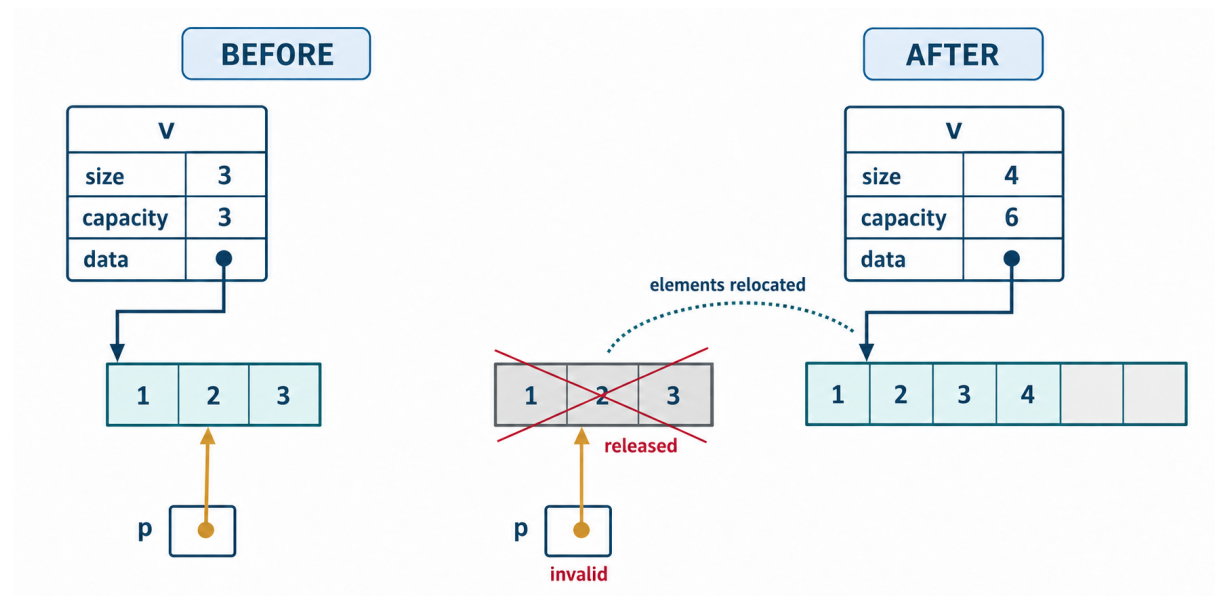


图 10.1 一种重新分配情形。vector 转向新存储，指向旧元素的指针不会自动跟随。图中的容量变化只是示例，不规定实现必须按两倍增长。

摊还常数时间是什么意思

多数尾部追加只构造新元素；少数追加还需要分配更大存储并搬迁已有元素。单次操作可能很贵，但按照合适增长策略观察一长串追加，其平均代价可以保持常数级。

这不是最坏情况下每次都只做常数工作。实时系统关注单次延迟时，仍需要考虑重新分配的突发成本。

提前 reserve 与错误的 reserve 用法

已知大致数量时，提前准备容量可以减少搬迁。若每次追加前都 `reserve(size()+1)`，反而可能破坏实现原本的增长策略，造成频繁重分配。

`shrink_to_fit` 也不应被当成每个实现都必须立刻释放全部多余容量的硬保证。需要稳定的内存行为时，应看具体接口契约并测量。

10.3 不同容器的组织方式

容器	适合的操作	主要取舍
vector	连续存储、随机访问、尾部追加	中间插删通常要移动元素
deque	两端增删、随机访问	存储不要求整体连续，失效规则更复杂
list	已知位置的插删、节点转接	查找位置仍可能线性，局部性较弱
array	固定长度连续存储	长度进入类型，不支持运行时扩容
map/set	按有序键查找与遍历	通常有节点及比较开销
unordered_map/set	基于哈希的平均快速查找	依赖哈希质量，最坏情况与重哈希需考虑

表 10.1 容器选择应围绕真实访问模式，而不是只比较一个操作的复杂度。

代码示例 双端队列、链表与优先队列

```
#include <algorithm>
#include <cassert>
#include <deque>
#include <list>
#include <queue>
#include <vector>
#include <functional>
int main() {
    // 双端队列允许两端插入；最终顺序是1、2、3、4。
    std::deque<int> d{2, 3}; d.push_front(1); d.push_back(4);
    assert(d[2] == 3);
    // 链表使用自己的成员 sort，不要求随机访问迭代器。
    std::list<int> l{3, 1, 2}; l.sort();
    assert(l.front() == 1 && l.back() == 3);
    // greater 使较小元素位于队首；接口只暴露当前最高优先级元素。
    std::priority_queue<int, std::vector<int>, std::greater<int>> q;
    for (int x : {3, 1, 2}) q.push(x);
    // top 只查看，pop 只删除，因此先保存再删除。
    int smallest = q.top(); q.pop();
```

```
    assert(smallest == 1 && q.top() == 2);
}
```

链表的成员排序能够利用自身结构，而普通 `std::sort` 需要随机访问迭代器。优先队列只承诺访问最高优先级元素；`pop()` 删除它，不返回它，所以需要先取值再弹出。

易错点 链表插入快，不等于整个任务快

已知有效位置时，链单元元素插入可以是常数级；如果先要从头找到位置，整体仍可能是线性。节点分配和缓存局部性也会影响实际性能。复杂度应覆盖完整任务链。

10.4 关联容器：键、比较器与哈希

若要按单词统计次数，数组下标不再方便，可以用 `map<string,int>` 把单词映射到计数。`counts["apple"]` 找到对应整数；键不存在时，它会先插入值初始化的整数 0，因此 `++counts["apple"]` 能直接开始计数。

如果只是想知道键是否存在，用 `find` 或 `contains`。`find` 返回迭代器，没找到时等于 `end`；不要为了查询而用 `[]`，否则可能在读取逻辑里改变容器。下面的例子把查询、插入和重复插入的结果分别检查。

有序映射按比较器组织键。两个键是否被视为等价，由比较关系决定，不一定直接调用 `operator==`。无序映射则同时依赖哈希和相等关系，相等的键必须产生相容的哈希结果。

代码示例 查询与插入的区别

```
#include <cassert>
#include <map>
#include <string>
int main() {
    std::map<std::string, int> counts;
    assert(counts.find("a") == counts.end() && counts.empty());
    // 不存在的键先插入值初始化的整数0，再加1。
    ++counts["a"];
    assert(counts.at("a") == 1);
    // 结构化绑定接收位置与是否插入成功；已有键不会被99覆盖。
    auto [it, inserted] = counts.try_emplace("a", 99);
    assert(!inserted && it->second == 1);
    assert(counts.contains("a"));
}
```

`find` 查不到时不插入，`operator[]` 对缺失键会建立相应值，`at` 则在缺失时报告错误。选择哪一个，取决于是否希望查询改变容器。

不要在只读逻辑中意外增加键

统计频次时 `++counts[key]` 很自然；检查某个用户是否存在时，使用下标可能悄悄添加一个默认记录。相同语法在不同业务语境中具有不同风险。

映射的键通常通过 `const` 键接口暴露，以避免随意修改后破坏组织规则。需要改变键时，应采用容器支持的相应提取、插入等机制，而不是通过强制转换偷偷写入。

10.5 半开区间与迭代器能力

迭代器可以先理解成“当前访问位置”。`begin` 指向第一个元素，`end` 表示尾后位置；`*it` 读取当前位置，`++it` 前进一步：

```
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << ' '; // 只有 it != end 时才读取元素
}
```

这种范围写成 `[first,last)`：包含 `first`，不包含 `last`。若两者相等，范围为空，循环一次也不执行。`end` 是结束标记，不是“最后一个元素的迭代器”。

`vector` 的迭代器还能快速跳跃；`list` 的迭代器需要逐步移动。因此，迭代器像统一的访问接口，却不保证所有操作都具有相同能力和复杂度。

常见算法使用 `[first,last)`：包含 `first` 指向的元素，不包含 `last` 所表示的位置。空范围可以自然表示为 `first==last`，但末端位置本身通常不能解引用。

能力	关键含义
输入	按相应单趟读取语义获取值
输出	通过赋值等协议输出值，是另一条能力分支
前向	支持相应多趟遍历语义
双向	还能向前一个位置移动
随机访问	能以常数复杂度按偏移定位
连续	在随机访问基础上，元素满足连续存储关系

表 10.2 输入与输出不是简单的大小排序。C++20 的范围还可以让终止哨兵与迭代器使用不同类型。

代码示例 复制元素、引用元素与只读迭代器

```
#include <cassert>
#include <iterator>
#include <list>
#include <vector>
int main() {
    std::vector<int> v{1, 2, 3};
    // 按值遍历得到副本，循环中的乘2不改变原元素。
    for (int x : v) { x *= 2; (void)x; }
    assert(v[0] == 1);
    // 按引用遍历直接修改原元素。
    for (int& x : v) x *= 2;
    assert(v[0] == 2);
    // const 限制迭代器对象不能移动，不限制它指向的 int 是否可写。
    const auto it = v.begin();
```

```

    *it = 7; // iterator object is const, element is not
    // cbegin 给出只读元素访问；这个迭代器本身仍可以移动。
    auto read_only = v.cbegin();
    assert(*read_only == 7);
    std::list<int> l{4, 5, 6};
    // 链表只能逐步前进；next 在这里走两步到值6。
    assert(*std::next(l.begin(), 2) == 6);
}

```

`for(int x:v)` 操作副本，`for(int& x:v)` 操作元素。`const auto it=v.begin()` 限制迭代器对象本身，未必限制所指元素；`cbegin()` 则提供只读元素访问。

不要把随机访问当成全部迭代器的能力

链表迭代器不能用任意加法下标或大小比较来遍历。`std::next` 可以适应多种迭代器，但在链表上移动多个位置仍需要逐步前进，不会自动变成常数时间。

10.6 失效规则：修改结构后重新检查访问途径

`vector` 发生重新分配后，指向原元素的指针、引用和迭代器失效。没有重分配的插入、删除也可能使相应位置及之后的访问途径失效。过去保存的 `end()` 也不能随意继续使用。

链表和许多节点式关联容器在插入时通常有更强的稳定性，但删除元素仍使指向该元素的访问途径失效。无序容器的重哈希会影响迭代器，与对元素引用和指针的影响要分别看。

代码示例 删除时使用返回的新位置

```

#include <cassert>
#include <vector>
int main() {
    std::vector<int> v{1, 2, 3, 4, 5};
    for (auto it = v.begin(); it != v.end(); ) {
        // erase 使旧位置失效，并返回后继位置；不要再额外递增一次。
        if (*it % 2 == 0) it = v.erase(it);
        // 保留当前元素时才前进一步，否则会跳过需要检查的元素。
        else ++it;
    }
    assert((v == std::vector<int>{1, 3, 5}));
}

```

删除后不再增加旧迭代器，而是接收 `erase` 返回的新位置。保留元素时才执行自增。这个控制流同时避免失效访问和跳过紧邻元素。

工程实践 把容器修改视为有效期边界

每次插入、删除、扩容或交换后，都重新确认先前保存的指针、引用、迭代器和视图是否仍有效。不要凭“地址看起来没变”代替契约检查。

10.7 元素构造、复制与移动

插入容器并不总是复制。传入左值可能复制，传入适当右值可能移动，原地构造接口可以直接用参数建立元素。具体选择还受重载、异常保证和分配器等规则影响。

代码示例 容器可以保存移动专用拥有者

```
#include <cassert>
#include <memory>
#include <utility>
#include <vector>
int main() {
    std::vector<std::unique_ptr<int>>> items;
    auto value = std::make_unique<int>(7);
    // 容器元素接管独占所有权，value 随后为空。
    items.push_back(std::move(value));
    // 这里仍先创建了一个临时 unique_ptr，再把它交给新元素。
    items.emplace_back(std::make_unique<int>(9));
    assert(value == nullptr && *items[0] == 7 && *items[1] == 9);
}
```

独占指针不能复制，但可以移动进入 vector。容器随后通过元素的生命周期管理每个资源。

emplace 不会让所有开销消失

即使新元素能原地构造，容器仍可能重分配并搬迁旧元素。若实参本来就是一个临时拥有对象，也要分析这个临时对象怎样产生。接口名字不能代替对整条构造路径的分析。

`vector<bool>` 等特殊表示还可能返回代理对象，而不是普通 `bool&`。写泛型遍历时，不应无条件假定每个解引用结果都是裸引用或每个 `vector` 都能提供对应的普通元素指针。

10.8 选容器时先回答实际问题

数据主要按顺序扫描，还是反复按键查询？插入位置是否已知？是否需要稳定引用？元素是否昂贵或不可复制？这些问题决定了容器选择的主要方向。

标准库为我们保存了实现经验，但不会替我们回答业务的访问模式。下一章将进一步讨论算法如何使用这些能力，以及违反算法前提为什么可能产生远离出错位置的问题。

第 11 章 标准算法：把意图写清楚，也把前提说清楚

标准算法让代码直接表达查找、复制、筛选、排序或累计的意图。它们的价值不只是省掉一个循环，还在于提供了清楚的接口、复杂度和行为约定。

不过，算法不会自动修复错误范围、失效迭代器或不合格的比较器。使用算法之前，先确认它需要什么能力和语义，比只记住函数名更重要。

11.1 一个算法由什么组成

先写最常见的用法，观察参数各负责什么：

```
std::vector<int> values{4, 1, 3};
std::sort(values.begin(), values.end()); // 原地排序，得到1、3、4
auto it = std::find(values.begin(), values.end(), 3);
if (it != values.end()) {
    std::cout << *it << '\n';           // 找到时才读取，预期输出3
}
```

前两个参数划定范围，find 的第三个参数说明要找什么。算法通常不替容器管理所有存储变化；后面的 copy、transform 和 remove 会进一步说明这种边界。

很多算法接收输入范围、输出位置以及一个可调用对象。输入范围决定处理哪些值，输出位置决定把结果放到哪里，可调用对象决定局部行为。

```
std::transform(first, last, output, operation);
```

这个接口不必知道具体容器类型。但输出空间必须足够，相关迭代器必须有效，操作也必须满足算法规定的要求。抽象屏蔽了不需要知道的细节，没有取消使用者的责任。

11.2 查找：返回末端时不要解引用

`std::find(first, last, value)` 找到时返回相应位置，找不到时返回传入的 `last`。这个 `last` 可能只是某个子范围的末端，不一定等于整个容器的 `end`。

易错点 子范围末端仍表示查找失败

在 `[v.begin()+1, v.end()-1)` 中查找失败，返回 `v.end()-1`。这个位置碰巧可能指向容器的有效元素，但它不属于本次搜索范围，不能把该元素当作找到了目标。

有序查找利用额外结构

代码示例 下界、上界与重复元素

```
#include <algorithm>
#include <cassert>
#include <iterator>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 2, 4};
    // lower_bound 找到第一个不小于2的位置，即第一个2。
    auto first = std::lower_bound(v.begin(), v.end(), 2);
    // upper_bound 找到第一个大于2的位置；两个位置围成重复值区间。
    auto last = std::upper_bound(v.begin(), v.end(), 2);
    assert(std::distance(first, last) == 2);
    assert(!std::binary_search(v.begin(), v.end(), 3));
}
```

下界定位第一个不小于目标的位置，上界定位第一个大于目标的位置。二者之间的半开区间给出全部相应重复元素。调用需要范围满足相关比较分区前提；保持排序是最常见的满足方式。

比较次数不等于全部运行成本

二分查找可以减少比较次数。但若在非随机访问迭代器上不断推进到中点，迭代器移动仍可能很贵。关联容器自己的查找接口可以利用内部结构，通用算法只能利用传给它的访问能力。

11.3 复制、筛选与输出迭代器

希望把每个元素平方后放进新 vector，可以写循环并逐次 `push_back`，也可以用 `transform`。难点是告诉算法结果写到哪里：普通输出迭代器表示已经存在的位置，`back_inserter(result)` 则把一次输出赋值适配成一次尾部插入。

因此，空 `result` 也能配合 `back_inserter` 使用；若直接传 `result.begin()`，却没有建立足够元素，就违反输出范围要求。下面先完成删除，再把剩余元素变换到新数组。

`std::copy` 向现有输出位置赋值，不会因为目标是 vector 就自动建立缺失元素。目标只 `reserve` 容量而 `size` 仍为零时，不能把 `begin` 当作可写入的已有元素范围。

`std::back_inserter` 把输出赋值适配成 `push_back`，让算法可以向尾部增加新元素。它展示了一种重要思想：保持算法接口不变，通过适配对象改变具体动作。

代码示例 删除范围与输出转换

```
#include <algorithm>
#include <cassert>
#include <iterator>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 2, 3};
    // remove 把保留元素移到前面，只返回新的逻辑终点。
```

```

auto end_of_values = std::remove(v.begin(), v.end(), 2);
assert(v.size() == 4);
// 真正缩小 vector 的 size, 并销毁尾部不再需要的元素。
v.erase(end_of_values, v.end());
assert((v == std::vector<int>{1, 3}));
std::vector<int> squared;
// back_inserter 把每次输出变成 push_back, 空容器也能接收结果。
std::transform(v.begin(), v.end(), std::back_inserter(squared),
               [](int x) { return x*x; });
assert((squared == std::vector<int>{1, 9}));
}

```

remove 为什么没有真正缩小容器

通用 `remove` 只知道迭代器范围，不拥有容器本身。它把应保留元素移动到前面，返回新的逻辑末端。容器仍保持原 `size`，直到调用容器的 `erase`。

逻辑末端之后的值不应再被当成一份有明确语义的删除结果。代码只依赖前面的保留区间，再一次性删除尾部。这就是 `erase-remove` 用法的核心。

C++20 还提供某些更直接的容器删除辅助函数，例如相应的 `std::erase_if`。选择哪种写法，应考虑语言版本和所用容器接口。

复制范围重叠时要检查方向

普通 `copy`、`copy_backward` 和移动版本有各自的重叠前提。不能把任意重叠范围都交给同一个方向的循环。对非平凡对象，也不能随意用字节复制代替对象赋值。

11.4 排序：比较器决定关系

比较器接收两个元素，返回“第一个是否应该排在第二个前面”。按分数降序就返回 `a.score > b.score`。它不是返回交换命令，也不是返回三个可能值中的一个。

同分时若需要保留原相对顺序，使用 `stable_sort`；若还要按编号决定次序，就把编号加入比较规则。下面两次排序分别展示这两种需求。

代码示例 稳定排序与显式打破平局

```

#include <algorithm>
#include <cassert>
#include <tuple>
#include <vector>
struct Student { int id; int score; };
int main() {
    std::vector<Student> students{{1, 90}, {2, 80}, {3, 90}};
    // 按分数降序；同分时保留原顺序，所以编号1在3前面。
    std::stable_sort(students.begin(), students.end(),
                    [](const auto& a, const auto& b) { return a.score > b.score; });
}

```

```

assert(students[0].id == 1 && students[1].id == 3);
std::sort(students.begin(), students.end(), [](const auto& a, const auto& b) {
    // 先比 score, 相同再比 id, 形成明确的多关键字顺序。
    return std::tie(a.score, a.id) < std::tie(b.score, b.id);
});
assert(students[0].id == 2);
}

```

第一次稳定排序只按分数，分数相同者保留原来的相对次序。第二次把分数和编号组成字典序，明确规定平局时怎样比较。

严格弱序应怎样理解

一个合适的严格比较器不应认为一个元素小于自己；若 a 小于 b ，不能同时认为 b 小于 a ；传递关系与比较等价关系也应保持一致。许多算法依赖这些规律组织内部控制流。

```

return a.score < b.score;    // suitable basic ordering
// return a.score <= b.score; // not a strict comparison

```

原理剖析 比较器是算法收到的规约

排序器不会逐一证明比较器是否满足所有关系性质。若传入的关系自相矛盾，算法就失去了原来的前提。编译通过只能说明调用形式合适，不能证明比较语义正确。

浮点 NaN、动态变化的外部权重和不稳定的随机比较，都需要特别谨慎。容器与算法使用的等价关系也可能与 `operator==` 不同，应避免混用判断标准。

11.5 算法能力：为什么 list 不能直接用 std::sort

通用 `std::sort` 要求随机访问等能力。链表只能按节点前后移动，不满足这个接口要求。它提供自己的成员 `sort`，利用不同结构完成排序。

为了看清能力与算法的关系，可以写一个只依赖前向遍历的教学排序：

代码示例 用前向迭代器完成冒泡排序

```

#include <algorithm>
#include <cassert>
#include <forward_list>
#include <functional>
#include <iterator>
template<class It, class Compare = std::less<>>
void bubble_sort(It first, It last, Compare less = {}) {
    if (first == last) return;
    bool changed = true;
    while (changed) {
        changed = false;
        // 只需维护两个相邻位置，不使用下标或迭代器加整数。
    }
}

```

```

    auto left = first, right = std::next(first);
    for (; right != last; ++left, ++right) {
        if (less(*right, *left)) {
            // 交换两个迭代器所指向的元素，而不是交换迭代器对象。
            std::iter_swap(left, right);
            changed = true;
        }
    }
}
}

int main() {
    std::forward_list<int> values{4, 1, 3, 2};
    // forward_list 没有随机访问能力，但满足本算法的逐步前进需求。
    bubble_sort(values.begin(), values.end());
    assert(std::is_sorted(values.begin(), values.end()));
}

```

这个版本能够作用于 `forward_list`，但比较与遍历成本为平方级。它用于理解泛型要求，不能因为适用范围广就认为比标准库的专门算法更好。

正确性与复杂度一起属于接口

相同功能可以由不同算法完成。随机访问能力让某些算法高效，节点操作让另一些算法方便。选择时应同时看功能、能力前提和实际数据规模。

11.6 累计：初始值也决定类型

代码示例 整数初值与浮点初值

```

#include <cassert>
#include <numeric>
#include <vector>

int main() {
    std::vector<double> values{1.5, 2.5};
    // 初值0的类型为 int，每一步的累计状态都回到 int。
    auto integer_sum = std::accumulate(values.begin(), values.end(), 0);
    // 改为 double 初值后，小数能够保留。
    auto real_sum = std::accumulate(values.begin(), values.end(), 0.0);
    assert(integer_sum == 3 && real_sum == 4.0);
}

```

整数初值使累计状态采用整数类型。1.5 加入后变成整数 1，再加入 2.5 后得到整数 3。浮点初值则保留小数，结果为 4.0。不是最后一步统一截断，而是每一步累计状态的类型都参与转换。

折叠顺序影响浮点与非结合操作

`accumulate` 具有相应顺序累计语义。允许重排的归约算法可能用不同组合顺序，浮点舍入结果也可能变化。把一个操作用于并行或自由重排归约前，应检查结合性等要求，而不只看它能否接受两个参数。

工程实践 明确选择累计状态

统计数量可以选择足够宽的整数；累计浮点数要考虑范围和数值误差；连接字符串需要字符串初始值。一个习惯性写下的 0，可能改变整个算法的结果类型。

11.7 可调用对象的状态与复制

算法常按值接收可调用对象。若对象内部有统计状态，调用者应确认最终修改发生在哪一份对象中。

代码示例 `for_each` 返回处理后的函数对象

```
#include <algorithm>
#include <cassert>
#include <vector>
struct Sum {
    int value = 0;
    void operator()(int x) { value += x; }
};
int main() {
    std::vector<int> data{1, 2, 3};
    Sum original;
    // 算法按值接收函数对象，累计结果从返回对象中取回。
    Sum result = std::for_each(data.begin(), data.end(), original);
    // 原对象仍是0；不能误以为算法一定直接修改了 original。
    assert(original.value == 0 && result.value == 6);
}
```

原始对象保持 0，返回对象累积到 6。这里不能仅凭传入了一个变量，就认为算法一定在原变量内部更新计数。

需要借用某个函数对象时，可以考虑显式引用包装等机制；需要保持可重用和可并行性质时，则应减少依赖隐含副作用。对于不同算法和执行策略，要分别确认调用次数、顺序和状态要求。

当算法链使状态与错误处理更难看清时，保留清楚的循环更好。下一章用 Ranges 简化范围传递，并继续分析生命周期问题。

第 12 章 Concepts 与 Ranges：把泛型要求放到接口上

模板代码经常在函数体深处才暴露类型不合适的问题。调用者本来只是传错一个参数，却可能看到许多层实例化诊断。Concepts 让一部分能力要求直接出现在接口上。

Ranges 则把常见的迭代器对进一步组织成范围，并提供可组合视图。它减少样板代码，也要求我们更加清楚地分析拥有、借用与惰性求值。

12.1 requires 表达式描述可用操作

模板函数写了 `a+b`，就隐含要求 `T` 支持加法。concept 让这个要求有名字，并放到接口附近。先从最简单的形式读起：

```
template<class T>
concept CanAdd = requires(T a, T b) { a + b; }; // 检查这段表达式能否使用
template<CanAdd T>
auto add(T a, T b) { return a + b; }           // 只接受满足条件的 T
```

`requires` 表达式中的 `a`、`b` 用来描述检查对象，不会在运行时执行一段隐藏测试。调用时若类型不满足约束，这个模板就不能按相应规则成为可行候选。

```
template<class T>
concept Addable = requires(T a, T b) {
    // 不仅要求表达式存在，还要求其结果满足可转换到 T 的约束。
    { a + b } -> std::convertible_to<T>;
};
```

这里检查表达式是否成立，以及结果是否满足指定要求。`a`、`b` 是用于表达检查条件的局部参数，不是在运行时创建两个对象，也不会仅凭这种写法要求 `T` 一定有默认构造函数。

可以检查类型名字、表达式、异常性质或组合约束。但约束应尽量准确描述算法真正需要的能力，既不要漏掉必要条件，也不要无故排除本来可用的类型。

核心概念 满足语法约束，不等于满足全部数学性质

一个类型能做加法，不代表加法满足交换律；一个比较器能返回 `bool`，不代表它构成严格弱序。Concepts 能检查部分形式要求，许多语义约定仍需要设计者和使用者维护。

12.2 用约束表达一个真正泛化的求和接口

普通求和常把结果类型写成 `int`，这既限制输入，也可能溢出。更清楚的接口允许调用者提供累计状态类型，并要求该状态能够接收范围中的元素。

代码示例 输入范围与累计能力分别约束

```

#include <cassert>
#include <concepts>
#include <ranges>
#include <string>
#include <vector>
template<class T>
// 声明一个编译期条件: a+b 有效, 且结果可转换回 T。
concept Addable = requires(T a, T b) { { a + b } -> std::convertible_to<T>; };
template<std::ranges::input_range R, class Acc>
// 外层 requires 添加约束; 内层 requires 表达式检查 a+=x 能否使用。
requires requires(Acc a, std::ranges::range_reference_t<R> x) { a += x; }
Acc sum(R&& range, Acc initial) {
    // 按范围提供的引用方式访问元素, 再累计到调用者指定的状态类型。
    for (auto&& x : range) initial += x;
    return initial;
}
int main() {
    static_assert(Addable<int> && Addable<std::string>);
    // OLL 指定 long long 累计值; 另一例用空字符串决定累计形式。
    assert(sum(std::vector<int>{1, 2, 3}, 0LL) == 6);
    assert(sum(std::vector<std::string>{"a", "b"}, std::string{}) == "ab");
}

```

`sum(vector<int>{1,2,3},0LL)` 使用较宽的整数累计, 字符串示例则用字符串初值。结果类型从初值参数获得, 不需要编译器从一个仅出现在返回类型中的参数凭空推导。

约束参与候选选择

当约束不满足时, 相应模板通常不能作为可行候选。它有助于把错误放到接口附近, 也能让多个受约束重载表达不同能力层次。

但是, 增加两个看起来相似的 `requires` 条件, 不保证编译器一定能按人类直觉判断哪个“更强”。约束的形式与归约规则也会影响重载关系。实际接口应保持简洁, 并用代表性类型检查选择结果。

12.3 范围不是新的容器继承体系

范围表示能够按相应规则获得起点与终点的一组元素。`vector`、数组、`span` 和许多视图都可以是范围。`std::ranges` 是命名空间, 不是让这些类型继承的共同基类。

传统算法常接收迭代器对, `Ranges` 算法还可以直接接收范围对象。有些范围的终止哨兵与迭代器不是同一类型, 这为某些序列提供了更自然的结束条件。

投影让“按哪个字段”显式化

代码示例 按分数排序，按编号查找

```
#include <algorithm>
#include <cassert>
#include <functional>
#include <ranges>
#include <vector>

struct Student { int id; int score; };

int main() {
    std::vector<Student> values{{1, 80}, {2, 95}, {3, 70}};
    // 先取每个 Student 的 score 字段，再按 greater 做降序比较。
    std::ranges::sort(values, std::greater<>{}, &Student::score);
    assert(values.front().id == 2);
    // 查找投影字段 id，而不是把整个 Student 与整数1比较。
    auto it = std::ranges::find(values, 1, &Student::id);
    assert(it != values.end() && it->score == 80);
}
```

投影说明先从元素中取得哪个值，再进行比较或匹配。成员指针在这里通过相应调用机制发挥作用，不需要为每个字段重复写一份近似 Lambda。

投影是调用规则，不意味着算法一定先把所有投影结果缓存起来。若投影很昂贵，仍需考虑实际调用次数与算法成本。

12.4 视图管道：描述怎样访问，而不立即建立结果数组

假设输入为 1、2、3、4、5、6，希望保留偶数、求平方、取前两个结果。可以写一个循环逐个处理，也可以用视图管道描述同一条数据流。

这里的 | 连接范围与范围适配器，不是对整数做按位或。按逻辑顺序读：filter 让 2、4、6 通过，transform 对通过的元素求平方，take(2) 限制最多取两个。因此最终保存 4、16。这些中间序列用于说明规则，不表示库一定建立了对应的中间数组。

代码示例 筛选、变换与截取

```
#include <cassert>
#include <ranges>
#include <vector>

int main() {
    std::vector<int> data{1, 2, 3, 4, 5, 6};
    // 管道描述访问规则；并不为每一步建立完整中间数组。
    auto query = data
        | std::views::filter([](int x) { return x % 2 == 0; })
        | std::views::transform([](int x) { return x*x; })
        | std::views::take(2);
    std::vector<int> result;
    // 真正遍历时才逐个筛选、平方，再把前两个结果保存下来。
}
```

```

for (int x : query) result.push_back(x);
assert((result == std::vector<int>{4, 16}));
}

```

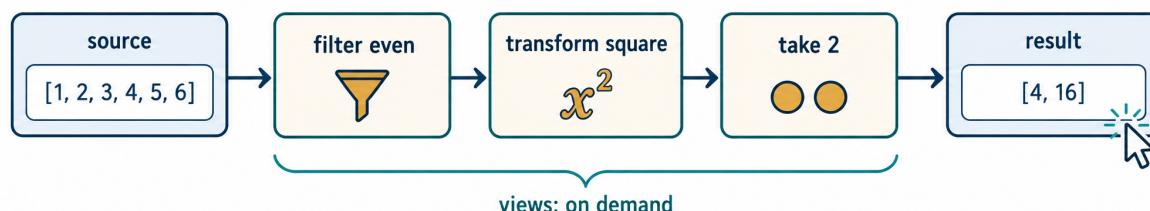


图 12.1 这条管道描述按需访问的转换规则。中间视图并不自动生成三份中间数组，最终容器由后面的遍历显式建立。

`filter` 保留偶数，`transform` 求平方，`take(2)` 只向后续使用者提供前两个相应元素，所以最终结果为 4 和 16。管道中的竖线由相应重载支持，用来组合范围适配器。

顺序改变，结果也可能改变

先取两个元素再筛选，与先筛选再取两个元素通常不同。先平方再按某个条件筛选，也可能改变谓词的含义。管道短并不意味着可以省略对顺序的分析。

12.5 惰性求值与重复访问

代码示例 视图没有提前保存变换后的全部值

```

#include <cassert>
#include <ranges>
#include <vector>
int main() {
    std::vector<int> values{1, 2, 3};
    // 建立一条读取时求平方的规则，这里不立即生成平方数组。
    auto squared = values | std::views::transform([](int x) { return x*x; });
    // 底层数据先改为10，之后访问视图时才计算得到100。
    values[0] = 10;
    assert(*squared.begin() == 100);
}

```

视图建立后，修改底层第一个元素；随后访问时得到平方 100。这个例子采用借用现有 `vector` 的变换视图，结果反映当前底层值。

不要依赖一个想象中的调用次数

不同视图可能进行缓存，也可能在重复解引用或遍历时重复计算。迭代器推进还可能调用筛选谓词。不要仅根据最终拿到几个元素，就断言谓词或变换一定恰好执行相同次数。

工程实践 让筛选与变换保持清楚的值语义

尽量让管道中的操作表达纯粹的判断或变换。必须产生副作用时，用明确的遍历或专门操作表达。这样更容易理解重复访问、调试和性能行为。

12.6 拥有、借用与悬垂

`std::span` 可以看成“数组起点 + 长度”的借用接口。它不负责申请或释放底层元素，也不因为自身还存在就让原数组延寿。`std::span{data}` 从现有数组建立这类访问描述。

视图同样需要追踪它究竟拥有数据，还是只保存对别处的引用。下面分别查找临时 `vector` 和临时 `span`：前者销毁后元素也随之消失；后者销毁后，本地 `data` 数组仍然存在。返回迭代器能否继续使用，必须连同这条存储关系一起判断。

视图不统一等于“不拥有”。具体类型可能借用，也可能以某种方式保存数据或其他范围。关键是追踪这个实例到底把哪些对象保存了下来。

把局部 `vector` 的借用视图返回以后，局部 `vector` 已经销毁。视图对象自己还活着，也不能补救底层数据的生命周期。

易错点 两层寿命都要检查

一层是底层元素是否仍然存在，另一层是迭代器是否依赖某个仍然存活的视图对象。仅看到范围来自一个活着的 `vector`，并不自动证明从临时复合视图中保存下来的迭代器总有效。

代码示例 `dangling` 与 `borrowed_range` 的差别

```
#include <algorithm>
#include <cassert>
#include <ranges>
#include <span>
#include <type_traits>
#include <vector>

int main() {
    // 临时 vector 随语句结束销毁，返回类型阻止继续拿悬垂迭代器访问。
    auto not_an_iterator = std::ranges::find(std::vector<int>{1, 2, 3}, 2);
    static_assert(std::is_same_v<decltype(not_an_iterator), std::ranges::dangling>);
    ;
    int data[]{1, 2, 3};
    // span 临时对象结束后，底层 data 数组仍然存在。
    auto it = std::ranges::find(std::span{data}, 2);
    assert(*it == 2);
}
```

对临时 `vector` 的查找返回 `dangling` 占位类型，避免把它误当成一个可继续访问的迭代器。临时 `span` 则可以在自身销毁以后留下仍指向数组的迭代器，因为数组仍由调用者持有。

borrowed 不表示数据永远存在

它描述迭代器能否在范围对象自身结束后继续独立于该对象使用，并不让底层存储永久有效。数组或拥有容器结束生命周期后，相应访问仍会失效。

12.7 视图也受算法能力限制

一个筛选视图可能不再具有原 `vector` 的随机访问能力。一个变换视图可能产生新值，而不是可交换的引用。因此不能因为底层来自 `vector`，就认为所有组合后的视图都能直接排序或写回。

范围修改还要考虑谓词、缓存和迭代器有效性。若修改破坏了视图赖以成立的前提，应重新构建合适访问结构，而不是继续复用旧状态。

显式物化结果

需要稳定拥有的结果时，可以像示例一样遍历并写入容器。较新标准还提供额外的范围转容器设施；例如使用 `std::ranges::to` 时，需要确认所用标准版本与库支持，不能直接当作所有 C++20 环境都有的接口。

12.8 用现代接口减少错误，而不是隐藏责任

Concepts 让能力要求更靠近接口，Ranges 让数据流更容易组合。它们没有改变 C++ 的基本事实：对象有寿命，借用有边界，算法有前提，类型承担语义。

到这里，语言与标准库的主要工具已经建立。后续将把它们放进多文件工程、测试和设计变化中，观察一个程序怎样逐步成为可维护的软件。

第 13 章 多文件工程：从一个程序文件到可构建的项目

当程序变大，把所有代码放进一个文件会让修改与协作变得困难。拆成多个文件以后，又会遇到新的问题：谁提供声明，谁提供定义，编译器怎样找到头文件，链接器怎样找到实现？

本章用一个小型统计库把这些步骤连起来。它包含公共头文件、实现、命令行程序和测试，能够独立构建，后面讨论测试与模块边界时也会用到这套结构。

13.1 翻译单元、编译与链接

先做一个只有三个文件的小程序：头文件声明求平均值的接口，一个源文件提供实现，另一个源文件包含 `main` 并调用它。拆分以后，调用者不必看到函数体，但编译器仍需看到声明，链接器仍需找到实现。下面的例子会把这三个文件都写出来。

源文件经过预处理后形成相应翻译单元。编译器根据它能看到的声明和定义检查代码，并生成目标文件。链接阶段再组合目标文件与库，解析跨文件的相应符号关系。

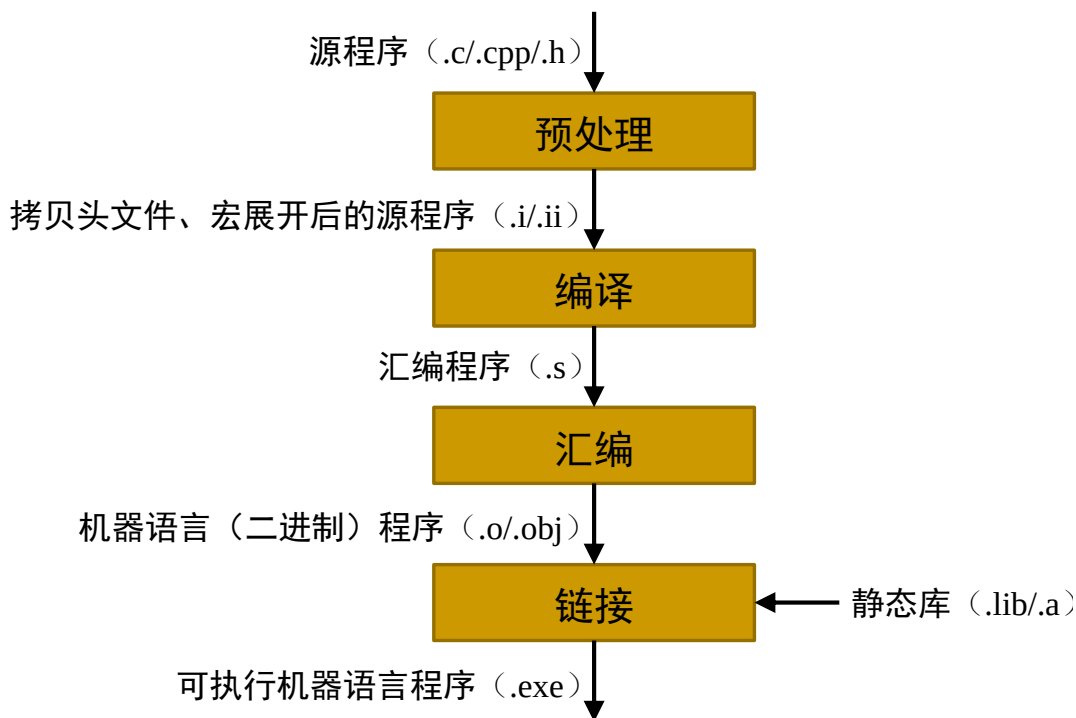


图 13.1 传统 C++ 工具链的概念阶段。实际工具可以合并部分步骤；文件后缀与可执行格式也随平台变化。（课程第 12 讲，第 2 页）

头文件通常通过包含进入翻译单元，而不是靠一个特殊的“全局共享区域”自动让所有文件看见。`#include` 解决可见文本的问题，链接库解决实现组合的问题，二者不能互相替代。

核心概念 区分两种常见错误

编译阶段的“名字未声明”通常表示当前翻译单元缺少必要声明。链接阶段的“未定义引用”常表示调用已经通过类型检查，却没有找到所需定义，或相应目标文件没有参与链接。应先判断错误发生在哪个阶段。

13.2 把接口与实现分开

建立 `include/notes/stats.hpp`。头文件中的保护宏避免同一翻译单元重复包含时反复处理相同内容；`namespace notes` 为项目接口提供名字范围，因此调用时写 `notes::mean`。

配套工程的公共头文件只暴露求平均值的接口：

代码示例 include/notes/stats.hpp

```
#ifndef NOTES_STATS_HPP
#define NOTES_STATS_HPP
#include <span>
namespace notes {
// 只声明调用接口；span 借用连续数据，const 禁止通过该接口改元素。
double mean(std::span<const double> values);
}
#endif
```

`std::span<const double>` 表示短期借用的一组只读值，不强迫调用者使用某一种容器。函数不保存这段借用，因此调用者只需让数据在调用期间有效。

实现文件提供算法与失败条件：

代码示例 src/stats.cpp

```
#include "notes/stats.hpp"
#include <numeric>
#include <stdexcept>
namespace notes {
double mean(std::span<const double> values) {
    // 空范围没有本接口定义的平均值，先检查再计算。
    if (values.empty()) throw std::invalid_argument("empty input");
    // 用 double 初值累计，最后除以实际元素数量。
    return std::accumulate(values.begin(), values.end(), 0.0)
        / static_cast<double>(values.size());
}
}
```

使用者只依赖头文件中的接口。平均值内部换成另一种适当的数值算法时，不必把实现复制到每个调用文件。

调用程序保存在 `src/main.cpp`，它只包含公共头文件，不包含实现源文件：

```
#include "notes/stats.hpp"
#include <array>
```

```
#include <iostream>
int main() {
    // 调用方拥有三个数，被调用函数只在调用期间借用它们。
    const std::array values{2.0, 4.0, 6.0};
    // 预期输出4，说明头文件声明与另一个源文件的定义成功连接。
    std::cout << notes::mean(values) << '\n';
}
```

在配套 `examples/project` 目录中，可以先手动编译这个最小工程：

```
clang++ -std=c++20 -Iinclude src/main.cpp src/stats.cpp -o stats_app
./stats_app
```

`-Iinclude` 把 `include` 加入头文件搜索路径；两份 `cpp` 都参与最终程序，输出预期为 4。若只编译 `main.cpp`，编译器虽然看得到声明，链接时却会缺少 `mean` 的实现。

头文件应尽量自包含

头文件使用了 `std::span`，就直接包含对应标准头文件，不依赖调用者碰巧先包含了别的头文件。头文件保护避免同一翻译单元重复处理相关定义，但不负责解决整个程序的全部定义冲突。

13.3 ODR、inline 与名字空间

单一规则（ODR）是一组规则，而不是“所有名字在整个程序里只能写一次”。普通具有外部链接的非 `inline` 函数与变量，通常需要唯一的相应定义。类、模板和 `inline` 实体可以在满足规则时出现在多个翻译单元中。

因此，把普通非 `inline` 函数定义放进会被多个源文件包含的头文件，可能造成重复定义。模板定义常放在头文件中，则是另一类有明确条件的用法。

// `counter.hpp`：只声明名字与类型，不在每次包含时重新定义存储。

```
extern int request_count;
```

// `counter.cpp`：在一个实现文件中提供实体与初值。

```
int request_count = 0;
```

`extern` 声明让其他文件知道这个变量；定义负责提供实体。C++17 的 `inline` 变量提供了另一种头文件组织方式，但仍需遵循一致定义等要求。

易错点 inline 不保证机器码内联

`inline` 与跨翻译单元定义规则有关。编译器是否把一次调用展开，是另一个优化决定。不能靠给函数加上 `inline`，就保证调用开销必然消失。

名字空间用于组织接口并减少名字冲突。公共头文件中尽量避免宽泛的 `using namespace`，否则会影响所有包含它的调用方。只在某个实现文件使用的辅助对象，可放在合适的内部范围中。

13.4 用构建系统表达目标和依赖

手动列出两份源文件已经能建立这个程序。项目变大以后，需要记录哪些文件组成库、哪些程序依赖它，以及接口需要什么编译标准。CMake 用目标表达这些关系，随后生成具体构建工具需要的文件。

`add_library(stats ...)` 建立库目标；`add_executable(stats_app ...)` 建立程序目标；`target_link_libraries(stats_app PRIVATE stats)` 表示程序依赖这个库。先按“库、调用者、依赖”阅读下面的配置，再看测试目标。

这个工程把算法做成库目标，再让程序和测试链接它：

代码示例 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(notes_stats LANGUAGES CXX)
# 把实现组成可复用的库，并向使用者公开头文件路径。
add_library(stats src/stats.cpp)
target_include_directories(stats PUBLIC include)
target_compile_features(stats PUBLIC cxx_std_20)
# 程序目标只需链接库目标，不必重复库的编译配置。
add_executable(stats_app src/main.cpp)
target_link_libraries(stats_app PRIVATE stats)
# 为同一份库实现建立独立测试入口。
include(CTest)
if(BUILD_TESTING)
    add_executable(stats_test tests/stats_test.cpp)
    target_link_libraries(stats_test PRIVATE stats)
    add_test(NAME mean_contract COMMAND stats_test)
endif()
```

`PUBLIC` 表明头文件路径和相关编译要求需要传给使用这个库的目标。`PRIVATE` 表明某项依赖只服务当前目标。与把所有选项塞进全局变量相比，目标级依赖更容易追踪。

在工程目录运行：

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

CMake [5] 描述并生成相应构建系统，不只能够生成 Makefile。构建目录与源文件分开，可以减少生成文件对源码的干扰。

增量构建依赖于正确的依赖图

修改一个实现文件，通常只需重新编译受影响部分；修改公共头文件，可能影响许多翻译单元。减少不必要的包含依赖，也是在控制修改成本和构建时间。

13.5 文件与流：状态检查属于处理过程

读取文本文件的流程是：打开文件，检查是否成功，再逐行或逐字段读取，最后区分正常结束与输入错误。

```
std::ifstream input("scores.txt"); // 需要 <fstream>, 文件名相对于运行目录
if (!input) throw std::runtime_error("cannot open file");
auto records = read_records(input); // 下面的解析函数可接收文件流或字符串流
```

把解析函数写成接收 `std::istream&`，就能用同一套逻辑处理文件与内存字符串。测试时不必先制造临时文件，可以直接把几行文本放入 `istringstream`。

文件流与字符串流共享许多接口，因此解析逻辑可以接收 `std::istream&`。测试时使用字符串流，实际使用时传入文件流，就不必为同一格式维护两套解析代码。

代码示例 逐行读取记录并检查完整格式

```
#include <cassert>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
struct Record { std::string name; int score; };
std::vector<Record> read_records(std::istream& input) {
    std::vector<Record> result;
    std::string line;
    // 每次成功读取一整行才进入循环，不在读取前猜测 eof。
    while (std::getline(input, line)) {
        if (line.empty()) continue;
        std::istringstream row{line};
        Record record;
        std::string extra;
        // 两个字段都要读到，之后还不能有多余字段。
        if (!(row >> record.name >> record.score) || (row >> extra))
            throw std::runtime_error("bad record");
        if (record.score < 0 || record.score > 100)
            throw std::runtime_error("score out of range");
        result.push_back(record);
    }
    // 区分正常读到文件末尾与真正的读取失败。
    if (input.bad() || (input.fail() && !input.eof()))
        throw std::runtime_error("read failure");
    return result;
}

int main() {
    std::istringstream input{"Alice 90\nBob 80\n"};
    auto rows = read_records(input);
    assert(rows.size() == 2 && rows[1].score == 80);
    std::istringstream bad{"Alice 90 extra\n"};
    bool rejected = false;
```

```
try { (void)read_records(bad); }  
catch (const std::runtime_error&) { rejected = true; }  
assert(rejected);  
}
```

例子规定每行是一个不含空白的名字和一个 0 至 100 的整数分数。先解析并验证完整行，再把记录加入结果。格式错误与底层读取错误都通过明确路径报告。

不要用 eof 预测下一次读取是否成功

到达文件末尾通常是在读取尝试后才被状态记录。`while(!stream.eof())` 容易多执行一次或重复使用旧数据。应让读取操作本身控制循环，并在结束后区分正常末尾与其他失败。

文件流析构会关闭资源，但如果必须确认写入是否成功，应在业务需要的时机显式检查 `flush`、`close` 或流状态。资源会被释放，不等于数据已经成功持久化。

13.6 二进制存储不是随意写出对象字节

直接写出一个结构体的内存，可能包含填充字节、平台相关的整数表示或端序，也可能包含仅在本进程中有效的指针。`std::string` 的对象字节更不等于字符串内容。

需要长期保存或跨平台交换的数据，应定义明确格式：字段顺序、长度、编码、数值表示及版本。短期实验中的字节快照，与稳定文件格式，是两种不同需求。

工程实践 让工程能被别人重复构建

记录所需标准、构建命令、输入格式与测试入口。构建不应依赖某个 IDE 中未写入项目的隐藏选项。一个能在干净构建目录完成编译和测试的小工程，才是后续重构与协作的可靠起点。

第 14 章 开发过程与设计原则：控制变化带来的成本

程序完成一次计算，软件还要在需求变化、多人协作和长期维护中保持可理解。设计的作用，是决定责任放在哪里、模块怎样交流，以及一项变化会影响多大范围。

本章保留开发过程与设计原则中最常用的判断方法。它们不要求把每个小程序都变成复杂框架，而是帮助我们在具体规模与风险下选择合适的结构。

14.1 从功能到可观察的完成标准

假设要做“按日期导出收支报告”。只写这句话，程序员还不知道边界日期是否包含、没有记录时输出什么、文件写入失败怎样处理。可以先确定一个小例子：输入三条带日期的记录，选择某个闭区间，应输出哪两条、总额是多少。

需求落到可观察结果以后，才有办法讨论接口和测试。随后把变化分别分给筛选、统计、格式化与写出组件，便能看到设计原则在具体问题中服务什么。

需求应说明用户需要什么结果，而不只是列出计划使用的技术。一个任务是否完成，最好有可检查的条件：输入怎样处理，输出怎样验证，错误如何报告，测试是否通过。

瀑布式过程强调阶段产物与计划，迭代过程通过多次反馈逐步修正方向。敏捷 [7] 强调可工作的软件、协作与对变化的响应，并不表示可以省略设计、文档和测试。

短迭代的意义是缩短反馈路径

尽早做出一个能验证关键假设的小版本，可以发现接口不合适、数据拿不到或性能目标不现实等问题。只有在阶段末尾才合并与测试，往往把多个问题叠在一起。

结对编程和代码审查提供不同形式的反馈。前者在实现过程中共同推理，后者让另一个人检查已经形成的变更。它们的价值来自发现盲点与共享理解，而不是简单增加参与人数。

14.2 模块与架构：先分责任，再画框

架构描述系统中的重要元素、关系和约束。分层结构常把界面、业务逻辑与存储分开，让不同变化有相对清楚的落点。

MVC 将模型、视图与控制相关职责分开。具体框架的细分方式可能不同，不能只根据名字判断系统是否解耦。应追踪一项用户操作如何修改状态，谁负责展示结果，谁决定业务规则。

信息隐藏与内聚、耦合

内聚关注一个模块里的职责是否围绕共同目标；耦合关注模块之间需要知道多少事情。信息隐藏希望把容易变化的设计决定放在稳定边界后面，例如数据表示、存储方式或计算策略。

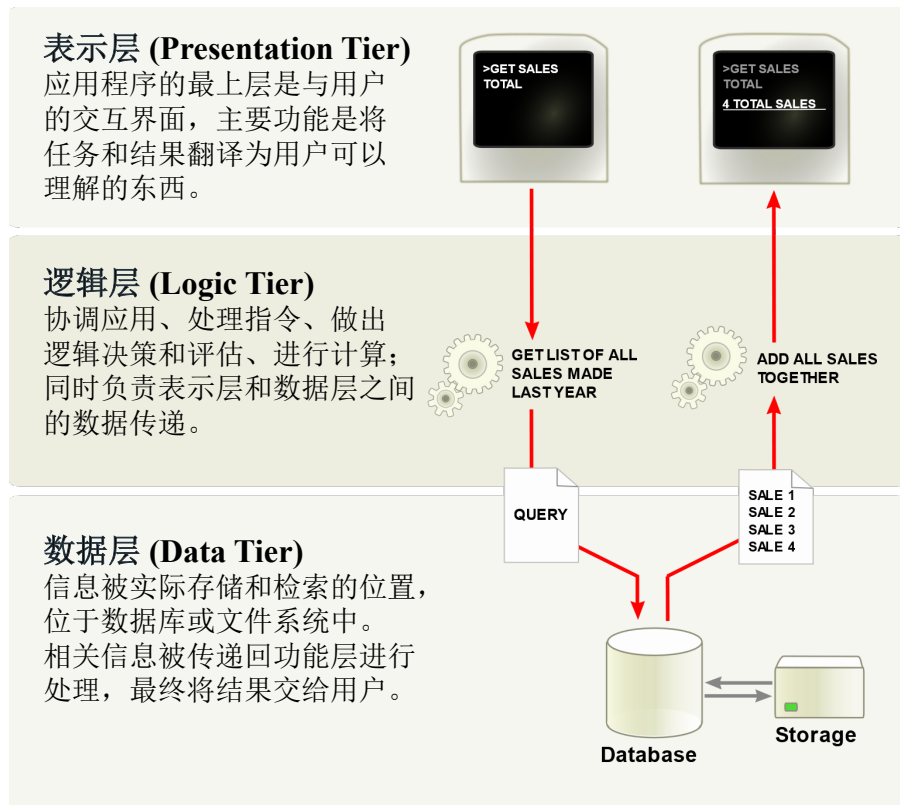


图 14.1 表示、业务逻辑与数据层是一种常见分工。箭头与边界应反映实际职责；把类放进三个文件夹，并不自动得到良好分层。（课程第 14 讲，第 16 页）

低耦合不等于没有依赖。一个系统必须协作。目标是让依赖清楚、方向合理，并让调用者只需要知道完成任务所必需的信息。

原理剖析 好的边界隔离变化，而不只是隔离文件

若更换存储方式必须修改界面、计算逻辑和许多调用方，存储决定很可能泄漏到了多个模块。把字段改成 `private` 还不够，还要检查接口是否继续暴露底层细节。

14.3 SOLID：把原则落到行为与变化上

仍用报告导出理解这组原则：统计金额的函数不应因为“PDF 改成网页”而改变；新增一种格式时，可以给稳定导出流程提供新的格式化实现；替换实现仍应遵守原接口对输入、结果与失败的约定。

如果测试报告流程必须真正发邮件或连接数据库，说明高层过程可能与具体设施绑得太紧。通过一个小接口传入真实或测试实现，就能在保持业务行为的同时替换这些依赖。下面的缩写是对这类问题的归纳，不需要为了凑缩写而增加类。

单一职责不是“每个类只允许一个方法”。计算发票金额、存数据库和排版打印可能由不同变化驱动，适合分开；围绕同一不变式的一组操作则可能应该放在一起。

开放封闭也不意味着任何未来需求都无需修改已有代码。应围绕可信的变化方向设置扩展点。过早抽象会增加接口与理解成本，完全不抽象则会让重复分支不断扩散。

原则	可以直接问自己的问题
SRP 单一职责	哪些不同的需求方会迫使这个模块改变？
OCP 开放封闭	对预期变化，能否增加实现而少改稳定流程？
LSP 里氏替换	派生对象是否保持基类对调用者作出的行为承诺？
ISP 接口隔离	客户是否被迫依赖它根本不需要的操作？
DIP 依赖倒置	高层规则能否依赖稳定抽象，而不绑定具体设施？

表 14.1 原则是分析工具，不是看到每个名词就增加一层类的理由。

替换关系要看契约

数学上的正方形是矩形，不代表带有独立宽高修改接口的 Square 一定适合继承 Rectangle。若基类承诺修改宽度不影响高度，派生类同时改掉高度就违背了调用者的预期。

第七章让 Square 与 Rectangle 平级实现 Shape，就是保留共同面积接口，同时避免承诺不合适的独立宽高修改行为。**继承关系能编译，只说明类型规则允许，不说明行为替换合理。**

14.4 依赖倒置与可测试性

按钮按下以后需要开启一个设备。如果 Button 直接构造某一种硬件驱动，测试也会被硬件绑定。下面把“能够开关”的能力提取成接口，再在建立 Button 时传入对象。

先读 main：建立测试设备，把它交给按钮，按下按钮，最后检查设备状态变为 on。这条使用路径就是依赖注入的具体含义；接口的价值在于同一 Button 可以使用其他合格设备。

下面的按钮不直接操作某一种灯泡，而是依赖它真正需要的开关能力。

代码示例 把设施替换成可验证的协作者

```
#include <cassert>
struct Switchable {
    virtual void set_on(bool value) = 0;
    virtual ~Switchable() = default;
};
class Button {
    // Button 借用一个满足开关接口的对象，不决定其具体设备类型。
    Switchable& target_;
public:
    explicit Button(Switchable& target) : target_(target) {}
    // 业务动作只表达开机要求，具体怎样开机由接收者实现。
    void press() { target_.set_on(true); }
};
struct FakeDevice : Switchable {
    bool on = false;
    void set_on(bool value) override { on = value; }
};
int main() {
```

```
FakeDevice device;  
// 创建时把依赖交进来；测试替身与真实设备使用同一个接口。  
Button button{device};  
button.press();  
assert(device.on);  
}
```

测试用 FakeDevice 记录状态，验证按钮确实发出了开启请求。生产系统可以提供其他实现，而按钮逻辑保持不变。

这里使用引用表示借用，调用者负责让设备活得比按钮的使用期更久。接口抽象没有取消前半部分建立的生命周期责任。

接口不要过宽

如果某个设备只能打印，不应强迫它实现扫描、传真和联网等无关操作。可以拆成客户真正需要的小接口。接口拆分应依据使用方式，而不是把每个成员函数都单独变成一个抽象类。

14.5 计划、风险与版本控制

任务依赖图说明哪些工作必须先完成，甘特图等工具帮助展示时间安排。估计应保留不确定性，并随着反馈调整；表格画得精确，不代表估计本身精确。

风险管理关注可能发生、并会影响目标的事情。可以先用“发生概率乘影响”的粗略尺度排序，再选择验证、缓解或备用方案。最重要的是让高影响的不确定性尽早得到反馈，而不是只登记风险名称。

Git [6] 记录提交之间的历史与项目快照关系。分支、合并和代码审查让多人工作能够有序汇合。提交粒度应围绕可理解的变化，避免把功能、格式调整和大规模重构混成一个难以检查的提交。

工程实践 把协作成本放进设计判断

稳定接口、可重复构建、自动化测试和清楚的提交说明，会减少团队成员之间需要口头记住的信息。工具支持这些约定，但不能代替共同理解。

一个合适的设计，应让常见变化成本可控，也让错误更容易被发现。下一章从输入边界和测试入手，讨论怎样检查这些承诺是否真正成立。

第 15 章 健壮性与测试：把“应该正确”变成可检查的条件

一个程序可能在正常输入下工作，却在空文件、非法数字或失败的资源申请下失去控制。健壮性要求我们为这些情况定义行为，而不是等它们发生后依赖偶然结果。

测试则把需求转成可执行的检查。它需要能够触发问题的输入，也需要足够敏感的判断标准。只运行过代码，与验证代码做对了事，是两回事。

15.1 规约、故障与可观察失败

正确性需要相对于某个规约讨论：允许什么输入，成功时返回什么，失败时有什么保证。健壮性进一步关注面对异常条件时，系统是否以受控方式响应。

一个代码缺陷要表现为外部失败，通常还需要执行到它、产生错误状态，并让错误状态传播到可观察结果。这解释了为什么一段有缺陷的代码也可能长期“看起来没问题”。

原理剖析 覆盖代码只是第一步

测试必须有机会到达缺陷，还要使它造成差异，并用合适的断言观察差异。只检查程序没有崩溃，可能漏掉内容错误、重复处理或状态悄悄损坏。

Verification 关注实现是否满足规定要求，Validation 关注这些要求及最终系统是否解决用户真正的问题。两种检查都需要，不能只靠通过单元测试来替代需求确认。

15.2 输入边界与错误处理

假设命令行参数应当是非负数量。输入字符串“42”应得到整数 42，但“12x”不能因为前两个字符能解析就算成功。还要处理空串、负数和超出 int 范围的数字。

from_chars 同时返回停止位置和错误码。因此下面不仅检查错误码，还检查是否恰好读到末尾，最后再检查数量必须非负。语法有效与业务有效分成两步，更容易发现遗漏。

外部输入应在明确边界上检查。解析器既要检查能否读出一个值，也要检查是否消费了整个输入，以及该值是否符合业务范围。

代码示例 完整解析一个非负计数

```
#include <cassert>
#include <charconv>
#include <stdexcept>
#include <string_view>
int parse_count(std::string_view text) {
    if (text.empty()) throw std::invalid_argument("empty count");
```

```
int value{};
const char* last = text.data() + text.size();
// 同时取得停止位置与错误码；解析出前缀数字还不算完整成功。
auto [end, error] = std::from_chars(text.data(), last, value);
// 必须无错误、用完整个输入，并满足非负数量的业务约束。
if (error != std::errc{} || end != last || value < 0)
    throw std::invalid_argument("invalid count");
return value;
}

int main() {
    assert(parse_count("0") == 0 && parse_count("42") == 42);
    // 覆盖空串、负数、尾部垃圾、空格和溢出，而不只测试正常整数。
    for (auto text : {"", "-1", "12x", " 3", "99999999999999999999"}) {
        bool rejected = false;
        try { (void)parse_count(text); }
        catch (const std::invalid_argument&) { rejected = true; }
        assert(rejected);
    }
}
```

这个接口明确拒绝空串、负数、多余字符、前导空格和超出整数表示范围的输入。如果产品允许前导空格，应把规范化作为有意的规则加入，而不是偷偷接受一部分错误格式。

断言不能替代外部输入检查

标准 `assert` 的行为受 `NDEBUG` 等配置影响，不是看到 `Debug` 或 `Release` 名称就自动决定。断言适合暴露程序内部不应发生的状态，不能承担必须始终执行的用户输入验证。

易错点 断言中不要放必要副作用

`assert(read_next());` 可能在禁用断言后连读取都不执行。应先完成必要操作，再检查结果。测试框架中的检查与生产代码中的调试断言，也应按各自约定使用。

把已验证数据与外部原始数据分开，可以减少重复检查。但内部函数仍可能被新调用方使用，接口前提应通过类型、封装和文档明确表达。

15.3 异常路径与状态保证

异常可以跨越多层调用，把失败交给能够处理它的边界。通常按 `const` 引用捕获标准异常，避免不必要复制与切片。重新传播当前异常可使用 `throw;`，而不是无条件重新抛出某个按基类类型保存的值。

异常传播过程中，RAII 负责清理已建立的局部资源。至于失败后的业务状态，还需要算法设计保证。

第四章 `Buffer` 的复制赋值先建立新副本，再通过不抛异常的交换提交，展示了强保证的常见结构。对文件写入或跨系统操作，则还需要考虑部分完成、恢复和重试，不能仅靠析构回滚全部外部影响。

保证	含义
基本保证	失败后仍保持约定的不变式，不泄漏所负责资源
强保证	失败时对约定的可观察状态不产生改变
不抛异常保证	操作不向调用者传播异常

表 15.1 这些保证回答不同问题。某个操作不泄漏，并不意味着它失败后保持原值。

15.4 黑盒、白盒与边界值

黑盒测试从规约选择输入，白盒测试利用实现结构选择路径。二者关注点不同，并不互相替代。

若分数有效范围为 0 至 100，并以 60 为通过线，可考虑非法区间、合法不通过区间和合法通过区间，再检查 -1, 0, 1, 59, 60, 61, 99, 100, 101 等关键位置。边界值应从真实规约推导，不是机械地为每个参数列同一组数字。

覆盖率观察什么

语句覆盖关注语句是否执行，分支覆盖关注判定的不同结果，条件覆盖关注组成条件的取值。短路求值会影响哪些子表达式实际被执行。循环还要关注零次、一次、多次与终止边界。

高覆盖率可以发现测试遗漏，却不能证明程序没有缺陷。错误的断言、不正确的规约、未检查的输出和不可行路径，都需要另外分析。

工程实践 先写清楚测试预言

测试预言就是判断结果对错的依据。对于排序，只检查结果有序会漏掉“排序时丢了一个元素”。还应检查元素是否仍构成原输入的排列。

15.5 参数化测试与性质测试

只测排序输入 {3,1,2} 得到 {1,2,3}，能确认一个例子，却很容易漏掉其他错误。可以把需求写成更一般的性质：结果有序；元素及其出现次数不变；已经排序的结果再次排序仍相同。

下面不是试图用几条性质证明任意程序永远正确，而是用它们检查不同角度。一个错误实现即使输出了有序序列，也可能偷偷丢掉重复元素，因此必须同时检查排列保持。

一次测试通常包含安排输入、执行操作和检查结果。需要限制适用域时，先写出假设；但不能用过强假设把真正需要检查的边界全部排除。

代码示例 用一般性质检查集合与排序

```
#include <algorithm>
#include <set>
#include <stdexcept>
#include <vector>

void require(bool condition) {
    if (!condition) throw std::runtime_error("property failed");
}
```



```
int main() {
    for (int x = -8; x <= 8; ++x) {
        std::set<int> values;
        // 先插入一次，再插入相同值，检查集合大小不应继续增加。
        values.insert(x);
        auto count = values.size();
        values.insert(x);
        require(values.contains(x) && values.size() == count);
        values.erase(x);
        require(!values.contains(x));
    }

    std::vector<int> input{3, 1, 2, 1};
    auto sorted = input;
    std::sort(sorted.begin(), sorted.end());
    require(std::is_sorted(sorted.begin(), sorted.end()));
    // 排序不能丢失或增加元素，只检查有序还不足以证明正确。
    require(std::is_permutation(input.begin(), input.end(), sorted.begin()));
    auto again = sorted;
    std::sort(again.begin(), again.end());
    // 已经排好的结果再次排序，应保持相同内容。
    require(again == sorted);
}
```

集合插入同一值两次应保持相应幂等性质。排序后应有序、保持元素集合，并且再次排序不改变已经排序的结果。

这些小循环只检查有限样本，不构成对任意输入的形式化证明。它们的价值是把规约中的关系写成可执行检查，从而比单个硬编码结果覆盖更多情况。

测试应能定位失败

把复杂场景拆成有名字的测试，分别检查边界、错误路径和关键性质。失败报告应让人知道哪个条件没有满足，而不是只留下一个笼统的“运行错误”。

15.6 建立短而可靠的反馈循环

单元测试检查局部行为，集成测试检查模块组合，系统或验收测试检查更完整的使用场景。资源检查工具、编译警告和 Sanitizer 可以提供额外线索，但只覆盖它们能够观察的性质和实际执行路径。

修复缺陷时，先保留一个能复现问题的测试，再调整实现并运行相关回归测试。这样，测试不仅说明今天运行成功，也记录了以后不能重新引入的行为错误。

测试和设计相互影响：可以注入协作者的模块更容易独立测试，明确返回错误的接口更容易覆盖失败路径。下一章的设计模式，正是在组织这些变化点和协作关系。

第 16 章 复用与创建模式：给变化一个合适的位置

同一任务可能有多种算法，旧组件可能使用不同接口，一个对象也可能需要按配置创建。设计模式 [8] 总结了这些反复出现的问题及其常见组织方式。

理解模式时，先找变化在哪里，再看对象之间怎样协作。一个模式的收益常伴随额外接口、对象和间接调用，只有这些代价换来了实际需要的灵活性，设计才有意义。

16.1 模式的名字应指向设计意图

遇到一个变化需求时，先写出调用者希望怎样使用它。例如求和算法希望更换“每项怎样计算”，日志模块希望接入一个旧接口，通知器希望叠加前缀。下面每种模式都从这样的需求出发，再看对象之间为什么需要那种关系。

适配器、装饰器和代理都可能保存一个对象并转发调用，但目的不同：适配器转换接口，装饰器组合附加行为，代理控制访问或推迟工作。只看一张类图的形状，未必能判断使用的是哪种模式。

复用也不只意味着继承。函数、模板、组合与委派都可以复用已有能力。库通常由应用调用，框架则常控制主流程，再调用应用提供的扩展点。控制权放在哪里，会影响整个系统的组织方式。

16.2 策略与模板方法：变化算法，还是变化步骤

第二章的求和循环已经展示了变化点：同样遍历 1 到 n ，每项可以是 k 、平方或立方。策略版本把每项公式作为一个对象传进来；模板方法版本在基类中固定求和流程，再让派生类实现每项公式。先用同一个平方和结果比较它们，才能看清结构差异。

策略把可变行为放进协作者中。模板方法把稳定流程放在基类，让派生类覆盖某些步骤。第二章的求和问题可以同时演示两种结构。

代码示例 同一个求和流程的两种组织方式

```
class SumStrategy {
    // 策略对象保存一个可替换的每项计算规则。
    std::function<int(int)> term_;
public:
    explicit SumStrategy(std::function<int(int)> term) : term_(std::move(term)) {}
    int total(int n) const {
        int result = 0;
        // 稳定的求和循环把变化的计算委派给策略。
        for (int k = 1; k <= n; ++k) result += term_(k);
        return result;
    }
}
```

```

    }
};
class SumTemplate {
public:
    int total(int n) const {
        int result = 0;
        for (int k = 1; k <= n; ++k) result += term(k);
        return result;
    }
    virtual ~SumTemplate() = default;
protected:
    // 模板方法把变化步骤作为虚钩子，由派生类提供。
    virtual int term(int k) const = 0;
};
class SquareSum : public SumTemplate {
    int term(int k) const override { return k*k; }
};

```

SumStrategy 保存可替换的可调用对象，SquareSum 则通过虚钩子定义每一项。配套程序对二者输入 5，都得到平方和 55。

```

SumStrategy strategy{[](int x) { return x*x; }}; // 把平方公式传进去
SquareSum inherited;                          // 用派生类提供平方公式
assert(strategy.total(5) == 55);
assert(inherited.total(5) == 55);

```

策略版本在循环中调用 term_ 保存的函数对象；模板方法版本在循环中调用虚函数 term。结果相同，变化是通过组合传入，还是通过继承提供。

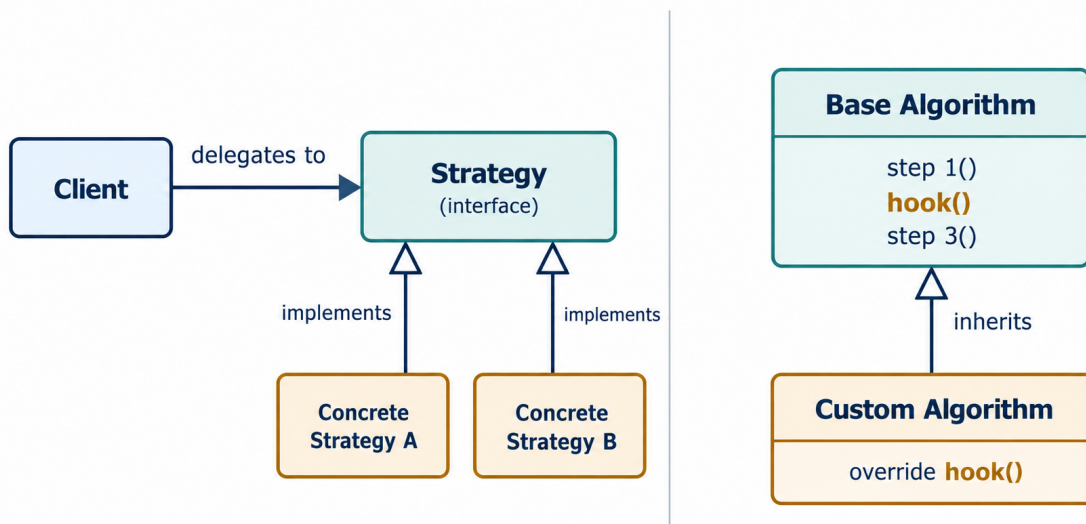


图 16.1 策略通过委派改变协作者；模板方法通过继承改变稳定流程中的钩子。二者都应保留清楚的行为契约。

如何选择

希望同一个对象在运行中更换算法，或让多个独立策略自由组合时，委派通常更灵活。流程本身稳定，而且派生类确实代表合理的特化关系时，模板方法比较直接。

策略不一定需要一整套虚类层次。本例用 `std::function` 表达运行时可替换行为；模板参数也可以表达编译期策略。选择语言机制时，应结合是否需要类型擦除、状态与运行时切换。

16.3 适配器：保留旧组件，提供新接口

假设旧日志组件接收零结尾字符串，新接口接收 `string_view`。适配器需要处理这两种接口之间的语义差异。

代码示例 适配不只改一个函数名

```
struct LegacyLog {
    std::string last;
    void write(const char* message) { last = message; }
};

struct Logger {
    virtual void log(std::string_view message) = 0;
    virtual ~Logger() = default;
};

class LogAdapter : public Logger {
    LegacyLog& legacy_;
public:
    explicit LogAdapter(LegacyLog& log) : legacy_(log) {}
    void log(std::string_view message) override {
        // 按视图长度建立自有字符串，保证末尾存在零终止符。
        std::string terminated{message};
        // 旧接口立即复制字符串，不保存即将失效的临时指针。
        legacy_.write(terminated.c_str());
    }
};
```

`string_view` 可能只是一个较长字符串的子段，`data()` 后面不一定立刻是零终止字符。这里先建立准确长度的字符串，再传给旧接口；旧接口在调用中复制内容，不保存临时指针。

```
LegacyLog legacy;
LogAdapter adapter{legacy};           // 适配器借用旧组件
std::string input = "hello-extra";
adapter.log(std::string_view{input}.substr(0, 5));
assert(legacy.last == "hello");       // 只写出视图包含的部分
```

这个文本示例假设消息没有内嵌零字符。若要支持任意二进制数据，就应让旧接口也接收长度，而不是只依赖零终止。

原理剖析 边界转换要核对数据与寿命

适配层可能还需要处理单位、编码、错误形式和所有权。函数名能对应，不代表两个接口的全部契约自然一致。

16.4 装饰器与外观：组合行为，或简化入口

装饰器维持相同接口，并在内部委派给另一个同接口对象。多个装饰器可以逐层组合。下面用返回字符串记录调用链，不进行实际网络发送。

代码示例 可组合的通知包装

```
struct Notifier {
    virtual std::string send(std::string_view text) const = 0;
    virtual ~Notifier() = default;
};

struct BasicNotifier : Notifier {
    std::string send(std::string_view text) const override {
        return "base:" + std::string{text};
    }
};

class PrefixDecorator : public Notifier {
    std::unique_ptr<Notifier> inner_;
    std::string prefix_;
public:
    PrefixDecorator(std::unique_ptr<Notifier> inner, std::string prefix)
        // 装饰器接管被包装对象，形成明确的拥有链。
        : inner_(std::move(inner)), prefix_(std::move(prefix)) {
        if (!inner_) throw std::invalid_argument("missing component");
    }

    std::string send(std::string_view text) const override {
        // 先加入自己的前缀，再把请求交给下一层。
        return inner_>send(prefix_ + std::string{text});
    }
};
```

```
auto notifier = std::make_unique<PrefixDecorator>(
    std::make_unique<PrefixDecorator>(
        std::make_unique<BasicNotifier>(), "inner:"), "outer:");
auto result = notifier->send("hello"); // 从最外层进入调用链
```

客户只调用最外面的 send。外层先形成“outer:hello”，内层再加“inner:”，最后基础对象加“base:”。

若外层添加 outer:，内层添加 inner:，基础实现再添加 base:，结果为：

```
base:inner:outer:hello
```

包装顺序可能改变行为，不能把装饰器组合无条件看成可交换。

外观模式则给多个子系统提供更简单的入口。例如文献管理软件的“导出报告”入口，可以协调查询、格式转换和写出，而不让界面逐个调用全部内部模块。

模式	主要变化	常见代价
适配器	客户所需接口与旧接口不同	多一层转换与错误映射
装饰器	同一接口下组合附加行为	调用链更长，顺序与调试更复杂
外观	客户不想了解多个子系统	外观可能逐渐积累过多职责
代理	需要控制访问或推迟工作	隐藏成本、失败与同步问题

表 16.1 相似的委派结构可以服务不同意图，应从客户需求判断。

16.5 工厂：把创建决定与使用流程分开

假设置配置文件给出 “text” 或 “json”，导出流程需要相应写出器。先把产品接口和两种具体实现定义清楚：

```
struct Writer {
    virtual std::string format() const = 0; // 本例用字符串表示所选格式
    virtual ~Writer() = default;
};
struct TextWriter : Writer {
    std::string format() const override { return "text"; }
};
struct JsonWriter : Writer {
    std::string format() const override { return "json"; }
};
```

下面的工厂接收配置并创建其中一种。调用者可以写 `auto writer=make_writer("json");`，再调用 `writer->format()` 得到 “json”，不必在每个使用位置重复配置分支。

一个普通工厂函数可以集中处理配置到具体类型的映射：

代码示例 简单工厂

```
std::unique_ptr<Writer> make_writer(const std::string& kind) {
    // 创建细节集中在工厂中，调用者只接收 Writer 的拥有指针。
    if (kind == "text") return std::make_unique<TextWriter>();
    if (kind == "json") return std::make_unique<JsonWriter>();
    // 未知配置明确失败，不能返回一个含糊的半初始化对象。
    throw std::invalid_argument("unknown writer");
}
```

调用者只接收 `Writer` 接口与明确的拥有者。新增格式时仍要修改这个集中分支，因此集中创建逻辑不等于完全消除了修改。

工厂方法把创建步骤作为可覆盖的钩子，稳定流程调用这个钩子：

代码示例 创建步骤参与动态分派

```
class ExportJob {
public:
    // 稳定流程通过虚工厂方法取得具体产品，再使用共同接口。
```



```

        std::string run() const { return create_writer()->format(); }
        virtual ~ExportJob() = default;
protected:
        virtual std::unique_ptr<Writer> create_writer() const = 0;
};
class JsonJob : public ExportJob {
    std::unique_ptr<Writer> create_writer() const override {
        // 派生任务只替换创建步骤，复用 ExportJob 的稳定流程。
        return std::make_unique<JsonWriter>();
    }
};

```

这里的工厂方法与模板方法具有共同结构：主流程保持稳定，创建哪个具体对象由特化实现决定。

抽象工厂组织产品族

如果需要一起创建相互匹配的按钮和对话框，可以让一个工厂提供整组创建操作：先定义两种产品接口，以及同一主题的两个具体产品：

// 两种产品分别有自己的接口；工厂负责把同一主题的实现配套创建。

```

struct Button { virtual std::string theme() const = 0; virtual ~Button() = default; };
struct Dialog { virtual std::string theme() const = 0; virtual ~Dialog() = default; };
struct DarkButton : Button { std::string theme() const override { return "dark"; } };
struct DarkDialog : Dialog { std::string theme() const override { return "dark"; } };

```

接下来用一个工厂把它们组织到一起：

代码示例 同一主题的相关控件

```

struct WidgetFactory {
    // 工厂接口组织多个相关产品，而不是只返回一种产品。
    virtual std::unique_ptr<Button> button() const = 0;
    virtual std::unique_ptr<Dialog> dialog() const = 0;
    virtual ~WidgetFactory() = default;
};
// 具体工厂提供同一主题的一整组控件。
struct DarkFactory : WidgetFactory {
    std::unique_ptr<Button> button() const override { return std::make_unique<
DarkButton>(); }
    std::unique_ptr<Dialog> dialog() const override { return std::make_unique<
DarkDialog>(); }
};

```

客户先建立 `DarkFactory factory;`，再分别调用 `factory.button()` 与 `factory.dialog()`，即可取得主题一致的两个产品。抽象工厂让客户保持同一产品族的创建来源，但增加新产品种类时，通常要修改工厂接口及多个实现。

工程实践 不要为了一个构造表达式建立整个框架

一个简单工厂函数已经能表达需求时，可以先保留简单结构。需要改变创建方式、隔离设施或维持产品族一致性时，再增加对应抽象。模式的价值应由真实变化证明。

模式选择也需要测试

测试不应只检查类是否继承了某接口，还要检查替换与组合后的行为。策略是否保持相同输入契约？适配器是否保留错误信息？装饰器是否恰好委派一次？工厂是否对未知配置给出明确结果？

第 17 章 维护与演化：扩展行为，并守住已有语义

软件维护不仅是修复错误，也包括适应新环境、增加功能和改善结构。变化持续发生时，原来合理的分支和依赖可能逐渐变得难以理解。

本章选取观察者、状态和访问者三种机制，分别处理通知、状态相关行为和对象结构上的新操作。随后讨论怎样通过小步重构改善结构，而不顺手改变原有功能。

17.1 观察者：把状态变化通知给多个客户

例如一个数据源发生变化时，表格视图和日志都要更新。如果数据源直接保存每种具体视图的名字，增加一种显示方式就要修改数据源。观察者接口把“接收新值”的操作统一起来：客户先订阅，发布者更新时逐个通知。

下面先关注 attach、publish、detach 三个使用动作。弱引用、快照和回调时机是为这条通知流程服务的生命周期设计，不是观察者模式必须一开始就背下来的附加术语。

当表格、图表和日志都需要响应同一份数据变化时，可以让它们订阅通知。发布者依赖共同接口，而不直接写死每一种具体视图。

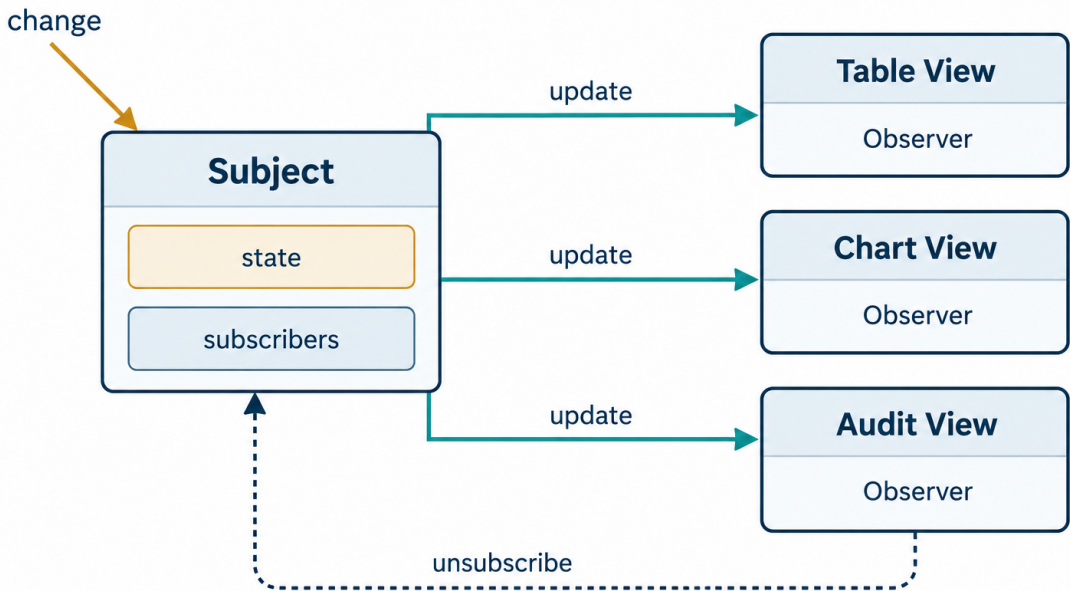


图 17.1 发布者向已订阅的观察者通知变化。订阅关系不必等于拥有关系，取消订阅和对象销毁都需要明确处理。

代码示例 弱引用订阅与每轮快照

```

struct Observer { virtual void update(int value) = 0; virtual ~Observer() = default
    ; };
class Subject {
    // 订阅列表不延长观察者寿命，使用时再临时取得强引用。
    std::vector<std::weak_ptr<Observer>> observers_;
    int state_ = 0;
public:
    void attach(const std::shared_ptr<Observer>& o) { observers_.push_back(o); }
    void detach(const std::shared_ptr<Observer>& o) {
        std::erase_if(observers_, [&](const auto& w) { return w.expired() || w.lock
        () == o; });
    }
    int current() const noexcept { return state_; }
    void publish(int value) {
        state_ = value;
        // 固定本轮遍历对象；回调修改原订阅列表不破坏当前迭代器。
        auto snapshot = observers_;
        for (const auto& weak : snapshot)
            // 目标仍存活才通知，并在本次回调期间保持其寿命。
            if (auto observer = weak.lock()) observer->update(value);
    }
};
struct Recorder : Observer { int last = 0; void update(int x) override { last = x;
    } };

```

外部拥有者管理观察者生命周期，Subject 保存弱引用。发送前临时锁定弱引用，可以避免访问已经销毁的观察者。每轮复制订阅表，则避免回调修改原表时直接破坏正在使用的遍历结构。

```

Subject subject;
auto view = std::make_shared<Recorder>();
subject.attach(view);           // 开始订阅
subject.publish(7);             // 回调 update, view->last 变为7
subject.detach(view);           // 结束订阅
subject.publish(9);             // 不再通知这个 view, 它仍保存7

```

publish 先改数据源状态，再按本轮订阅快照尝试取得观察者并调用 update。客户既可以显式取消，也可以销毁观察者；弱引用让数据源不会因此保留一个本应结束的界面对象。

通知策略必须说清楚

这个单线程示例使用发布开始时的订阅快照，因此本轮回调中取消的订阅，通常从下一轮起生效。回调异常、重复注册、递归发布和并发修改也需要明确策略，不能只靠“松耦合”三个字略过。

配套程序检查了订阅后收到值、取消后不再收到后续值，以及观察者销毁后不再访问它。事件流越隐蔽，记录清楚的通知约定和测试就越重要。

17.2 状态模式：让状态决定行为

播放器收到同一个 Pause 事件，在“播放中”应切换到暂停，在“已停止”则可以保持停止。先把“当前状态 + 事件 → 下一状态”列成表，再决定怎样实现。状态模式把各状态的响应放入对应对象，使播放器本身只负责转交事件与替换当前状态。

这并不意味着任何两三项状态都必须写成一组类。状态对象在每种状态有较多行为、规则需要独立维护时才更有价值。这个小例子用于看清交接流程。

当许多操作都根据同一状态写分支时，可以把各状态的行为集中到相应对象中。以播放器为例：停止状态可以播放，播放状态可以暂停或停止，暂停状态可以恢复或停止。

当前状态	Play	Pause	Stop
Stopped	Playing	保持	保持
Playing	保持	Paused	Stopped
Paused	Playing	保持	Stopped

表 17.1 先定义转移表，再选择 enum、表驱动或多态状态对象来实现。

状态对象返回下一状态，Context 在本次调用结束后提交替换：先给出事件、状态接口以及三种响应。相互引用的具体状态先声明 next，再在类型定义齐全后提供函数体：

```
// 事件类型限定了外部可以发给播放器的请求。
enum class Event { Play, Pause, Stop };
struct State {
    virtual std::unique_ptr<State> next(Event event) const = 0;
    virtual std::string_view name() const = 0;
    virtual ~State() = default;
};
struct Stopped : State {
    std::unique_ptr<State> next(Event) const override;
    std::string_view name() const override { return "stopped"; }
};
struct Playing : State {
    std::unique_ptr<State> next(Event) const override;
    std::string_view name() const override { return "playing"; }
};
struct Paused : State {
    std::unique_ptr<State> next(Event) const override;
    std::string_view name() const override { return "paused"; }
};
std::unique_ptr<State> Stopped::next(Event e) const {
    // 处于 stopped 时收到 Play，产生一个 Playing 状态对象。
    if (e == Event::Play) return std::make_unique<Playing>();
    return {};
}
std::unique_ptr<State> Playing::next(Event e) const {
    if (e == Event::Pause) return std::make_unique<Paused>();
    if (e == Event::Stop) return std::make_unique<Stopped>();
}
```

```

    return {};
}
std::unique_ptr<State> Paused::next(Event e) const {
    if (e == Event::Play) return std::make_unique<Playing>();
    if (e == Event::Stop) return std::make_unique<Stopped>();
    return {};
}

```

代码示例 把状态计算与状态替换分开

```

class Player {
    std::unique_ptr<State> state_ = std::make_unique<Stopped>();
public:
    void handle(Event event) {
        // 先让旧状态完整执行 next，此时它还没有被销毁。
        auto replacement = state_->next(event);
        // 取得新状态后再替换拥有者，避免成员函数执行中销毁自身。
        if (replacement) state_ = std::move(replacement);
    }
    std::string_view mode() const { return state_->name(); }
};

```

这样，当前状态执行自己的成员函数时仍然存在。创建下一状态失败时，也还没有替换旧状态。完整状态类和转移测试保存在配套程序中。

工程实践 小状态机不必强行使用类层次

状态少、操作简单时，enum 加一个清楚的转移函数可能更易维护。状态模式适合状态相关行为已经足够多，且需要把这些行为聚合起来的情况。

```

Player player;
player.handle(Event::Play);    // 从停止变为播放
player.handle(Event::Pause);   // 从播放变为暂停
player.handle(Event::Stop);    // 从暂停变为停止

```

对第一个事件，Player 把 Play 交给 Stopped::next，取得 Playing 对象，再替换 state_。下一次调用因此会进入 Playing 的响应规则，而不是继续调用 Stopped。

17.3 访问者：给稳定的对象结构增加操作

考虑表示 $2+3$ 的小语法树：根是 Add，两个孩子是 Number。现在想对这棵树增加求值、打印等不同操作。如果节点种类相对稳定，可以把这些操作各自放到访问者对象中，再由节点接待访问者。

先看调用路径：客户对根调用 accept，根把自己的具体类型交给访问者的 visit，访问者决定怎样处理并继续访问孩子。下面的 Number、Add 是数据结构，Evaluator 则是其中一种操作。

表达式树可能有数字节点、加法节点和更多节点类型。求值、打印和统计节点数都作用于同一棵树。访问者把这些操作从节点的主要职责中分离出来。

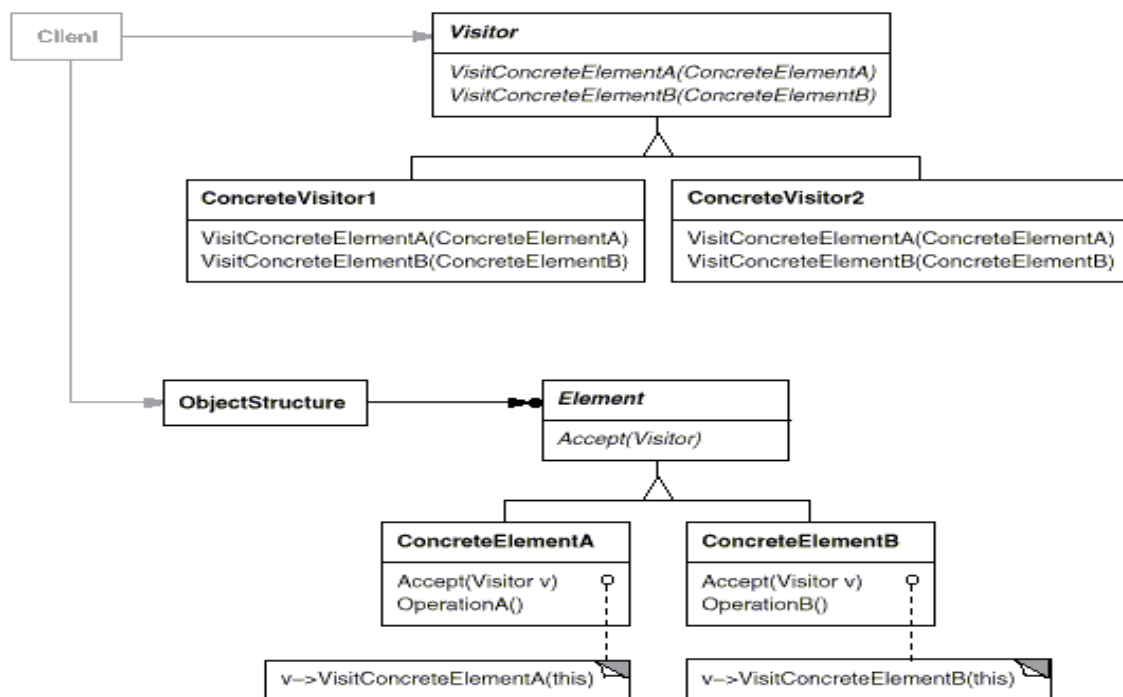


图 17.2 访问者与元素各有一组具体类型。元素通过 accept 将自身的准确类型交给访问接口。（课程第 18 讲，第 44 页）

代码示例 表达式树的访问入口

```

struct Number; struct Add; // 先声明节点类型，下面的接口才能引用它们。
struct Visitor {
    virtual void visit(const Number&) = 0;
    virtual void visit(const Add&) = 0;
    virtual ~Visitor() = default;
};

struct Node {
    virtual void accept(Visitor&) const = 0;
    virtual ~Node() = default;
};

struct Number : Node {
    int value;
    explicit Number(int n) : value(n) {}
    // 在 Number 的函数体内，*this 的静态类型为 Number，选择对应 visit 重载。
    void accept(Visitor& v) const override { v.visit(*this); }
};

struct Add : Node {
    std::unique_ptr<Node> left, right;
    Add(std::unique_ptr<Node> a, std::unique_ptr<Node> b)
        : left(std::move(a)), right(std::move(b)) {
        if (!left || !right) throw std::invalid_argument("missing child");
    }
    void accept(Visitor& v) const override { v.visit(*this); }
};
  
```

第一次虚分派选择具体节点的 `accept`。在该实现内部，`*this` 的静态类型明确，因此选中相应 `visit` 重载；这次虚调用再选择具体访问者实现。不能把整个过程简单理解成“C++ 自动按两个动态类型做任意重载选择”。

下面给这套接口加入求值操作，再实际访问 $2 + 3$ ：

```
struct Evaluator : Visitor {
    int result = 0;
    void visit(const Number& n) override { result = n.value; }
    void visit(const Add& n) override {
        // 先递归求左子树的值并保存，避免被右子树结果覆盖。
        n.left->accept(*this); int a = result;
        // 再求右子树，用保存的左值与新右值相加。
        n.right->accept(*this); result = a + result;
    }
};

int main() {
    Add expression{std::make_unique<Number>(2), std::make_unique<Number>(3)};
    Evaluator visitor;
    // 从树的根开始访问，预期把2和3相加得到5。
    expression.accept(visitor);
    assert(visitor.result == 5);
}
```

根节点 `Add::accept` 中的 `*this` 类型为 `Add`，所以调用访问者的 `Add` 重载。求值器先访问左边 `Number` 得到 2，并保存这个值；再访问右边得到 3；最后把两者相加。只有接口而没有这段调用过程，很容易看不出 `accept` 与 `visit` 为什么需要配合。

它对哪种变化友好

增加新的访问者，通常不必修改每个具体节点。增加新的节点种类，却通常需要扩展 `Visitor` 接口并检查各个访问者。所以它适合元素种类相对稳定、操作经常增加的场景。

配套例子使用独占指针拥有子节点，并实现求值访问者，验证 $2 + 3 = 5$ 。数据结构的拥有关系仍由节点负责，访问者只执行相应操作。

17.4 重构：小步改变结构，保持外部行为

原来的统计函数接收独立的 `start`、`end` 两个整数，每个调用点都要记住它们一起表达日期范围。可以提取 `DateRange`，把“是否包含某天”的规则放到范围对象里，再让统计循环调用 `contains`。

这里的目标是改变职责的位置，同时保持已有输入的结果。若还想规定 `start` 不能大于 `end`，应另作行为变更；把两件事混在一起，测试失败时就不容易判断发生了什么。

重构 [9] 在不改变约定外部行为的前提下改善内部结构。提取函数、引入参数对象、移动行为和拆分职责，都是常见做法。功能修改与重构混在一起，会让失败原因更难定位。

代码示例 把一组相关参数变成对象

```
struct DateRange {
    int start, end;
    // 把日期范围判断放回日期范围对象，保留原来的闭区间语义。
    bool contains(int day) const { return start <= day && day <= end; }
};

int after(const std::vector<Entry>& entries, DateRange range) {
    int sum = 0;
    for (const auto& e : entries)
        // 调用方表达筛选意图，不再重复拆开比较起止值。
        if (range.contains(e.day)) sum += e.amount;
    return sum;
}
```

原来用两个独立整数判断日期范围，现在把判断放到 `DateRange` 中。测试把多个起止值组合代入重构前后两份实现，比较结果一致性。

这里没有顺便改变起止反转时的行为。如果原程序对这种输入有某个既定结果，新增拒绝规则就是另一个功能决定，应单独讨论和测试。

代码味道是线索，不是自动判决

重复代码、长函数、长参数列表和过度消息链，可能提示职责没有放在合适位置。但数据记录类本身并不一定是坏设计；传输数据或表示输入格式时，只携带数据可能正是职责所在。

重构应围绕具体维护问题展开。为了让每个类都看起来更“面向对象”而增加层次，可能反而让关系更难理解。

17.5 度量与审查：让判断有依据

代码行数和圈复杂度可以提示检查重点。对满足相应图模型条件的单一连通控制流图，圈复杂度可写为 $M = E - N + 2$ ；结构化代码也常用判定节点数加一估算。短路表达式、异常边和工具规则会影响具体计数。

WMC、DIT、CBO、LCOM 等面向对象度量分别观察方法规模、继承深度、耦合和内聚等方面。它们不是一个可以代替设计判断的总分。为了降低某项数字而拆出大量无意义函数，也可能增加理解成本。

代码审查应围绕正确性、接口契约、资源与错误路径、测试和可读性给出具体反馈。小而目的清楚的变更更容易检查。发现问题以后，还要确认修正与回归结果，而不只是留下评论。

维护的目标，是让下一次变化仍然可以被理解、实现和验证。好的模式与重构方法，应当降低这个成本，而不是增加只有原作者才看得懂的结构。

第 18 章 性能与并发：先测量，再改变执行方式

性能优化应从具体目标出发。响应时间、吞吐量、峰值内存和资源利用率，描述的是不同问题。一个修改让单次计算更快，也可能增加内存、延迟或维护成本。

并发则进一步改变执行顺序。多个任务可以交错推进，也可以在多个处理器上并行执行。程序必须在允许的执行方式下保持正确，而不是只在某次测试的时间安排下成功。

18.1 测量与瓶颈

先确定有代表性的工作负载，再测量耗时、调用次数、分配和 I/O 等指标。一次运行的时间不是复杂度证明，性能分析器观察到的热点也只代表相应工作负载。

应分别观察启动、预热、稳态和异常路径，避免把调试构建与优化构建的数字混在一起。测量后每次只改变清楚的一项，再检查功能与性能是否都符合预期。

原理剖析 局部优化的收益受整体比例限制

若原运行时间中可优化部分占比例 p ，这部分加速 s 倍，忽略额外开销后的整体加速比为

$$S = \frac{1}{(1-p) + p/s}.$$

如果瓶颈并不在你正在改的那一段，即使把它变得很快，整体收益也可能很小。

从减少工作开始

优先检查是否反复计算相同结果、是否进行了不必要复制、是否能提前结束搜索，以及数据结构是否匹配访问模式。连续存储的局部性、分配次数和间接访问，也可能比一条算术表达式的写法更重要。

18.2 惰性计算与代理

惰性计算把工作推迟到确实需要时。它可以省掉从未使用的计算，也会把首次使用的成本和失败推迟到另一个位置。

代码示例 单线程的延迟加载代理

```
#include <cassert>
#include <memory>
struct Image {
    inline static int loads = 0;
    Image() { ++loads; }
```

```

    int draw() const { return 7; }
};
class ImageProxy {
    std::unique_ptr<Image> image_;
public:
    int draw() {
        // 第一次真正 draw 时才建立 Image，之后复用同一对象。
        if (!image_) image_ = std::make_unique<Image>();
        return image_>draw();
    }
};
int main() {
    ImageProxy p;
    // 只建立代理还不应触发加载。
    assert(Image::loads == 0);
    // 两次调用都返回7，但实际对象只构造一次。
    int a = p.draw(), b = p.draw();
    assert(a == 7 && b == 7 && Image::loads == 1);
}

```

对象建立时不加载图像，第一次 draw 才建立真实对象，之后复用。这里用计数器模拟加载行为，没有进行文件或网络操作。

缓存也需要契约

结果什么时候过期？数据变化后怎样失效？多个线程同时首次访问时谁负责建立？这些问题决定缓存是否正确。第十二章的惰性视图与这里的惰性缓存也不同：按需计算不一定意味着结果会被永久保存。

写时复制是一种可用的设计策略，但不能假定任意类型或现代 `std::string` 都采用这种实现。共享表示会影响引用、修改和同步契约，必须整体设计。

18.3 单例：唯一入口不等于没有依赖

代码示例 函数局部静态对象

```

#include <cassert>
class Settings {
    Settings() = default;
public:
    // 不允许复制出另一份 Settings 实例。
    Settings(const Settings&) = delete;
    Settings& operator=(const Settings&) = delete;
    static Settings& instance() {
        // 首次调用时建立唯一的函数局部静态对象，后续调用返回它。
        static Settings value;
        return value;
    }
};

```

```

    }
    int mode() const { return 3; }
};
int main() {
    // 比较地址，确认两次调用取得同一个对象。
    assert(&Settings::instance() == &Settings::instance());
    assert(Settings::instance().mode() == 3);
}

```

相应函数局部静态初始化具有线程安全规则，但这只解决初始化本身。对象的后续可变成员操作仍可能需要同步。

单例还会带来全局访问、销毁顺序与测试替换方面的成本。若某个服务只是需要统一管理生命周期，显式创建并注入依赖也可能更清楚。不要只为减少几次构造，就引入难以替换的全局状态。

18.4 线程生命周期与数据竞争

建立 `std::jthread worker([]{...})` 时，一个新执行线程开始运行传入的可调用对象，原线程也可以继续向下执行。任务结束与线程对象离开作用域是不同事件，拥有者必须安排等待。

下面使用一个内部作用域保存四个工作线程，作用域结束时等待它们完成，再在外层读取结果。那一对额外的花括号不是装饰，而是把“等待全部工作完成”的时机写进程序结构。

普通 `std::thread` 对象销毁时若仍可加入，程序会终止。必须明确安排 `join` 或合适的其他责任。C++20 的 `std::jthread` 在适用情况下请求停止并加入，便于用对象生命周期管理线程。

请求停止不是强制中断线程。任务需要配合停止机制，或者像下面例子一样自然完成有限工作。

代码示例 原子计数与自动加入

```

#include <atomic>
#include <cassert>
#include <thread>
#include <vector>
int main() {
    // 共享的是一个原子计数器，读改写不能拆成普通读后普通写。
    std::atomic<int> count{0};
    {
        std::vector<std::jthread> workers;
        for (int i = 0; i < 4; ++i)
            // 创建4个并发任务，它们引用同一个仍存活的 count。
            workers.emplace_back([&] {
                for (int k = 0; k < 1000; ++k)
                    // 这里只统计次数，不借此发布其他数据，因此使用 relaxed。
                    count.fetch_add(1, std::memory_order_relaxed);
            });
    } // all jthreads joined
}

```



```
// 内层作用域结束会等待线程完成，此时再检查总数4000。  
assert(count.load() == 4000);  
}
```

内层作用域结束时，所有工作线程已经加入；外部的 `count` 仍然存在，因此引用捕获的目标寿命足够长。每次递增是原子读改写，最后计数为 4000。

计数示例使用 `relaxed`，因为它只需要原子累计，不借这个计数发布另一组数据。若要让一个线程据标志读取另一个线程写入的其他状态，就必须分析相应同步与可见性关系。

数据竞争与普通的不确定顺序不同

两个线程同时访问同一内存位置，至少一个是写，并且缺少所需同步、也不满足相应原子规则时，可能构成数据竞争。在 C++ 中，这会导致未定义行为，不能只把输出列成几个可能的“丢失更新”结果。

即使代码没有数据竞争，操作组合也可能逻辑错误。例如两个独立原子操作组成的“先检查、再修改”，并不会自动成为一个不可分割事务。

18.5 用互斥量保护共同不变式

如果一次记账既增加余额，又追加记录，这两个字段应当一起变化。只保证每个整数读写独立安全，还不能保证“余额等于所有记录之和”。因此下面把整个更新过程放到同一把锁保护的区域中。

`lock_guard lock{mutex_}` 在建立时加锁，在离开作用域时解锁。其他遵守同一互斥量的访问要等它释放以后才能进入。读取一致性关系时也使用这把锁，才能观察完整状态。

如果余额与记录列表必须一起变化，就应在同一个临界区内维护它们，而不是分别保护两个字段。

代码示例 保护跨字段关系

```
#include <cassert>  
#include <mutex>  
#include <thread>  
#include <vector>  
class Ledger {  
    std::mutex mutex_;  
    int balance_ = 0;  
    std::vector<int> entries_;  
public:  
    void add(int amount) {  
        // 离开作用域时自动解锁，异常路径也包含在内。  
        std::lock_guard lock{mutex_};  
        // 先完成可能分配失败的操作，再更新余额，避免只改一半。  
        entries_.push_back(amount);  
        balance_ += amount;  
    }  
}
```

```
// 读取也遵守同一把锁，检查记录之和与余额的关系。
bool consistent() {
    std::lock_guard lock{mutex_};
    int sum = 0;
    for (int amount : entries_) sum += amount;
    return sum == balance_;
}
};

int main() {
    Ledger ledger;
    {
        std::jthread a{[&] { for(int i=0;i<100;++i) ledger.add(1); }};
        std::jthread b{[&] { for(int i=0;i<100;++i) ledger.add(2); }};
    }
    // 前面的内层作用域已结束，两条 jthread 都已完成。
    assert(ledger.consistent());
}
```

这里先向列表加入记录，再更新余额。若插入抛出异常，余额还没改变；锁守卫则负责离开作用域时释放互斥量。示例使用小整数，重点是演示同步边界与状态关系。

锁保护的是约定，而不只是变量名

所有读写路径都必须遵守同一同步协议。持锁调用未知回调可能导致死锁或重入；更强的原子内存顺序也不能弥补寿命错误。应优先减少共享，必要时再明确数据的拥有者、同步方式与线程结束时机。

附录 A 贯穿案例与自检

下面几组问题把语言规则、设计与测试连接起来。先预测行为，再编译或运行对应例子。遇到未定义行为时，应分析违反了哪项前提，不能用某次输出代替结论。

A.1 对象、复制与移动

一个 `move` 表达式建立了几个新对象

```
Buffer a{1, 2, 3};
Buffer&& alias = std::move(a); // 只建立 a 的别名
Buffer b = alias;             // 具名别名是左值，这里建立独立副本
Buffer c = std::move(alias);  // c 接管 a 原来的资源
```

建立 `alias` 时没有移动构造新 `Buffer`。具名的 `alias` 表达式是左值，所以 `b` 独立复制内容；`c` 则接管 `a` 原来拥有的数组。按照本书 `Buffer` 的约定，最后 `a` 为空，`b` 和 `c` 拥有不同数组。

可以检查元素内容、长度和拥有的地址关系，但不要通过已经失效的旧指针读取来“证明”移动是否发生。完整行为验证见 `08_buffer_move.cpp` 与 `08_move_is_cast.cpp`。

复制一个拥有指针的类，需要先回答什么

先确定复制应表示独立值、共享实体，还是根本不允许。再检查成员类型能否表达这种语义。复制了地址却让两个对象各自无条件释放，通常说明所有权责任没有设计清楚。

A.2 容器与视图

`reserve` 以后能否直接用下标写入

不能仅凭容量足够判断。下标需要指向已经存在的元素。`reserve(100)` 后 `size` 仍可能为零；可以先 `resize`，或使用 `push_back`、`emplace_back` 等建立元素。

一个视图能否安全返回

不要只看返回类型名字里有没有 `view`。应追踪它保存的是值、拥有者还是引用，以及迭代器是否依赖视图对象自身。返回局部容器的借用访问通常会悬垂；返回一个正确拥有数据的结果则是另一种情况。

排序测试需要检查什么

除了“输出有序”，还应检查元素是否丢失或重复、比较器是否符合关系要求，以及稳定性是否属于本次约定。属性测试中的排列保持与幂等性，是有用的补充，但仍不能覆盖所有可能缺陷。

A.3 设计变化与行为契约

增加一种算法，为什么不总要增加派生类

策略可以用模板参数、函数对象或类型擦除包装表达。若变化只是一小段行为，直接传入可调用对象可能足够。只有确实需要稳定的运行时接口或类职责时，才需要相应类层次。

状态切换时，旧对象何时结束生命周期

先让当前状态返回下一状态，再由 Context 替换拥有者，可避免在当前成员函数尚依赖自身时销毁它。还应检查下一状态创建失败时，旧状态是否仍满足不变式。

为什么重构要避免顺手修改输入规则

结构改善与功能变化一起发生，会让测试失败难以归因。引入 DateRange 可以先保持旧行为，再单独决定是否增加非法范围检查。每一步的目标清楚，回归判断才更可靠。

A.4 Buffer 的完整定义

下面是 Buffer 的完整定义，可保存为头文件供其他示例包含。先注意两种构造方式：

```
Buffer zeros(3); // 按数量构造：三个值为0的元素
Buffer one{3};   // 按元素列表构造：一个值为3的元素

#ifndef SOFTWARE_NOTES_BUFFER_HPP
#define SOFTWARE_NOTES_BUFFER_HPP
#include <algorithm>
#include <cstddef>
#include <initializer_list>
#include <stdexcept>
#include <utility>
class Buffer {
    std::size_t size_ = 0;
    int* data_ = nullptr;
public:
    // 按数量建立数组；这里的成员初始化器在构造函数体之前执行。
    explicit Buffer(std::size_t n = 0)
        : size_(n), data_(n ? new int[n]{} : nullptr) {}
    // 花括号元素列表先确定长度，再把每个初始值放入数组。
    Buffer(std::initializer_list<int> values) : Buffer(values.size()) {
        if (size_ != 0) std::copy(values.begin(), values.end(), data_);
    }
    // 对象离开作用域时归还自己拥有的数组；空指针可安全删除。
    ~Buffer() { delete[] data_; }
    // 复制构造先申请自己的新数组，再复制元素，双方互不共享所有权。
    Buffer(const Buffer& other) : Buffer(other.size_) {
        if (size_ != 0) std::copy_n(other.data_, size_, data_);
    }
    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
```

```

        // 先复制成功再改目标；若分配失败，目标原内容保持不变。
        Buffer temporary(other);
        // 交换后 temporary 拿到目标的旧数组，随后由它的析构函数释放。
        swap(temporary);
    }
    return *this;
}

void swap(Buffer& other) noexcept {
    std::swap(size_, other.size_);
    std::swap(data_, other.data_);
}

// exchange 取出来源旧值并同时置空；目标接管原数组。
Buffer(Buffer&& other) noexcept
    : size_(std::exchange(other.size_, 0)),
      data_(std::exchange(other.data_, nullptr)) {}

Buffer& operator=(Buffer&& other) noexcept {
    if (this != &other) {
        // 赋值目标已有旧数组，接管新数组之前先释放旧数组。
        delete[] data_;
        size_ = std::exchange(other.size_, 0);
        data_ = std::exchange(other.data_, nullptr);
    }
    return *this;
}

std::size_t size() const noexcept { return size_; }
int* data() noexcept { return data_; }
const int* data() const noexcept { return data_; }
int& operator[](std::size_t i) noexcept { return data_[i]; }
const int& operator[](std::size_t i) const noexcept { return data_[i]; }
// 检查下标以后再返回元素引用；与不检查范围的 [] 区分。
int& at(std::size_t i) {
    if (i >= size_) throw std::out_of_range("Buffer::at");
    return data_[i];
}

const int& at(std::size_t i) const {
    if (i >= size_) throw std::out_of_range("Buffer::at");
    return data_[i];
}
};
#endif

```

A.5 配套程序的使用

examples 目录包含独立示例、预期编译失败反例，以及一个多文件工程。buffer.hpp 是贯穿资源管理、运算符和移动语义的完整实现。

在讲义目录运行：

```
python3 examples/check.py
```

```
python3 examples/check.py 08_ 17_  
cmake -S examples/project -B .build/project  
cmake --build .build/project  
ctest --test-dir .build/project --output-on-failure
```

第二条命令只验证指定文件名前缀，适合修改某一部分后检查。编译失败反例以 `fail_` 开头，测试脚本期待编译器拒绝它们；不要把它们与正常程序一起合成一个可执行文件。

学习资源管理时，可以用 AddressSanitizer 与 UndefinedBehaviorSanitizer 帮助发现相应内存或未定义行为问题。工具只能检查它实际观察到的执行路径，仍需结合对象寿命与接口约定分析。

工程实践 怎样使用这些例子学习

每次只改一个条件，例如去掉 `const`、改成传值、提前销毁拥有者，或换一个容器。先写下自己的预测，再观察编译诊断或测试结果。确认行为来自语言规则、库契约还是具体实现，是比单纯记住输出更重要的能力。

参考文献

- [1] 谢涛、马郢. 软件设计实践. 北京大学, 2025 年春季课程课件, 00-20.
- [2] 软件设计实践 2025: 知识点总结. 本地参考笔记 `software.html`.
- [3] ISO C++ 工作草案 N4861. 2020. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/n4861.pdf>
- [4] C++ Core Guidelines. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>
- [5] CMake 官方文档: Tutorial. <https://cmake.org/cmake/help/latest/guide/tutorial/>
- [6] Scott Chacon, Ben Straub. Pro Git, 2nd edition. <https://git-scm.com/book/en/v2>
- [7] Manifesto for Agile Software Development. 2001. <https://agilemanifesto.org/iso/zhchs/manifesto.html>
- [8] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [9] Martin Fowler. Refactoring: Improving the Design of Existing Code, 2nd edition. 2018. <https://refactoring.com/>